

北京邮电大学

硕士学位论文



题目： PDF 内容提取系统设计与实现

学 号： 2019180064

姓 名： 徐志辉

专 业： 电子与通信工程

导 师： 孙学斌

学 院： 信息与通信工程学院

2022 年 5 月 29 日

中国 · 北京

密级： 保密期限：

北京邮电大学

硕士学位论文



题目： PDF 内容提取系统设计与实现

学 号： 2019180064

姓 名： 徐志辉

专 业： 电子与通信工程

导 师： 孙学斌

学 院： 信息与通信工程学院

2022 年 5 月 29 日

Beijing University of Posts and Telecommunications

Thesis for Master Degree



**Topic: Design and implementation of
PDF content extraction system**

Student ID: 2019180064

Cnadidate: Zhihui Xu

Subject: Electronics and Communications Engineering

Supervisor: Xuebin Sun

Institute:School of Information and Communication Engineering

May 29, 2022

PDF 内容提取系统设计与实现

摘 要

随着互联网时代的到来,人们传递信息的方式由纸质材料变成了电子文件。PDF 文件因为具有跨平台、体积小、不会出现排版混乱等优势,一经推出便坐稳了电子文件格式界的头把交椅。PDF 文件分为数字 PDF 文件和图片 PDF 文件,数字 PDF 文件可以由 Word、Excel 等文件转换得到,图片 PDF 文件可以由手机或者扫描仪扫描纸质文档得到。由于图片 PDF 文件生成方式特殊,用户使用它时会有一些障碍。首先,其内容本质为一张张图片,用户无法复制或者编辑文件中的文字。其次,由于手机拍摄角度或者光照不均等问题,生成的 PDF 文件经常会出现大片阴影,影响用户的阅读体验。最后,被拍摄的纸质文档往往存在折叠、褶皱等畸变,增加了内容提取的难度。

目前最新的图像处理技术几乎都是针对自然场景的,对文档类图像的研究很少,并且市面上现有的 PDF 文件处理软件并没有阴影去除这个功能。因此,本课题从用户需求和社会实用价值出发,基于深度学习等技术实现了一套 PDF 内容提取系统。主要工作内容包含以下几个方面:

1. 结合课题的研究背景和国内外研究现状,明确系统的研究方向和目标。对系统进行技术选型,通过分析系统需求,完成功能模块的划分,之后给出系统的详细设计。

2. 使用了 BEDSR-Net 模型和相关技术实现了 PDF 文件内容阴影去除功能, BEDSR-Net 模型是第一个也是目前唯一一个专门为文档图像阴影消除而设计的深度学习模型,解决了传统启发式手工设计特征去除图像阴影的弊端。

3. 提出了文档文字图像数据集的制作方法,供文字检测和文字识别模型训练使用。爬取互联网公开的 PDF 文件并随机选取其中的页面,由于这些 PDF 文件是数字生成的,我们需要使用相关库把文件里的页面转为图像,然后根据文本行坐标裁剪图像即可得到高质量的文本行图像。最后对这些数据进行图像增强,提高模型的泛化能力。

4. 使用工具库操作文档对象实现了数字 PDF 文件中文本或者表格的提取功能。使用基于深度学习的 OCR 技术实现了图片 PDF 文件中文本或者表格的提取功能。根据用户的不同需求，系统可以只处理部分范围页面，并且生成不同格式的文件。文中给出了详细的处理流程图并且展示了提取结果。

5. 实现了系统的前端页面，降低了系统使用门槛，让用户可以轻松的使用本系统。从系统实用性角度出发，增加了一些常用的附加功能，例如 PDF 合并、PDF 拆分、PDF 旋转、PDF 加水印等。最后从系统的界面、功能、性能方面进行了测试，并给出了测试的结果。

关键词 PDF 文件 图像阴影去除 OCR 技术 合成数据集

DESIGN AND IMPLEMENTATION OF PDF CONTENT EXTRACTION SYSTEM

ABSTRACT

With the advent of the Internet age, the way people transmit information has changed from paper materials to electronic files. Due to the advantages of cross-platform, small size, and no typesetting confusion, PDF files have become the top spot in the field of electronic file formats once they are launched. PDF files are divided into digital PDF files and image PDF files. Digital PDF files can be converted from Word, Excel and other files, and image PDF files can be obtained by scanning paper documents with mobile phones or scanners. Due to the special way of generating image PDF files, there are some obstacles for users to use it. First of all, its content is essentially a picture, and users cannot copy or edit the text in the file. Secondly, due to the camera angle or uneven lighting of the mobile phone, the generated PDF file often has large shadows, which affects the user's reading experience. Finally, the captured paper documents often have distortions such as folds and wrinkles, which increases the difficulty of content extraction.

At present, the latest image processing technologies are almost all aimed at natural scenes, and there is little research on document images, and the existing PDF file processing software on the market does not have the function of shadow removal. Therefore, starting from user needs and social practical value, this topic implements a PDF content extraction system based on deep learning and other technologies. The main work includes the following aspects:

1. Combine the research background of the subject and the current research status at home and abroad, and clarify the research direction and goals of the system. The technical selection of the system is carried out,

and the division of functional modules is completed by analyzing the system requirements, and then the detailed design of the system is given.

2. The BEDSR-Net model and related technologies are used to realize the shadow removal function of PDF file content. This model is the first and only deep learning model specially designed for document image shadow removal, which solves the problem of traditional heuristic manual design. Features remove the drawbacks of image shadows.

3. A method of making document text image dataset is proposed for text detection and text recognition model training. Crawl the PDF files published on the Internet and randomly select the pages in them. Since these PDF files are digitally generated, we need to use the relevant libraries to convert the pages in the files into images, and then crop the images according to the text line coordinates to get high-quality images. Text line image. Then image enhancement is performed on these data to improve the generalization ability of the model.

4. Using the tool library to operate the document object realizes the function of extracting text or tables in digital PDF files. Using OCR technology based on deep learning, the function of extracting text or tables in image PDF files is realized. According to the different needs of users, the system can process only part of the page range and generate files in different formats. The detailed processing flow chart is given and the extraction results are shown.

5. The front-end page of the system is realized, which lowers the threshold for using the system, allowing users to use the system easily. From the perspective of system practicability, some common additional functions have been added, such as PDF merging, PDF splitting, PDF rotation, and PDF watermarking. Finally, the system's interface, function and performance are tested, and the test results are given.

KEY WORDS: pdf file image shadow removal ocr technology synthetic dataset

目 录

第一章 绪论.....	1
1.1 研究背景和意义.....	1
1.2 国内外研究现状.....	2
1.2.1 文档图像阴影去除研究现状.....	2
1.2.2 OCR 技术研究现状	3
1.3 各章节内容及组织结构.....	4
1.4 本章小节.....	5
第二章 理论知识与关键技术介绍	6
2.1 深度学习理论知识介绍.....	6
2.1.1 卷积神经网络.....	6
2.1.2 循环神经网络.....	7
2.1.3 生成对抗网络.....	7
2.2 关键技术介绍.....	8
2.2.1 数据库技术介绍.....	8
2.2.2 Web 开发技术介绍.....	9
2.2.3 OCR 技术介绍	10
2.2.4 其他技术介绍.....	13
2.3 本章小节.....	13
第三章 系统需求分析与详细设计	15
3.1 系统需求分析.....	15
3.1.1 功能需求分析.....	15
3.1.2 性能需求分析.....	17
3.1.3 界面需求分析.....	18
3.2 系统总体设计.....	18
3.2.1 架构设计.....	18
3.2.2 功能模块设计.....	20
3.2.3 前端界面设计.....	22
3.2.4 数据库设计.....	23
3.2.5 算法模型设计.....	26
3.3 本章小节.....	34
第四章 系统具体实现	35
4.1 文档文字图像数据集制作	35
4.1.1 问题描述.....	35
4.1.2 数据集合成.....	35

4.1.3 图像增强.....	36
4.2 PDF 阴影去除模块实现	37
4.2.1 界面实现与展示.....	37
4.2.2 数据处理.....	39
4.2.3 去除结果展示.....	41
4.3 PDF 文字提取模块实现	42
4.3.1 界面实现与展示.....	42
4.3.2 数据处理.....	43
4.3.3 提取结果展示.....	46
4.4 PDF 表格提取模块实现	48
4.4.1 界面实现与展示.....	48
4.4.2 数据处理.....	49
4.4.3 提取结果展示.....	52
4.5 PDF 其他操作模块实现	54
4.5.1 PDF 合并	54
4.5.2 PDF 拆分	55
4.5.3 PDF 旋转	57
4.5.4 PDF 加水印	58
4.6 图片内容提取模块实现.....	61
4.7 用户管理模块实现.....	63
4.7.1 用户注册及登录.....	63
4.7.2 用户使用次数统计.....	65
4.7.3 用户评论及点赞.....	67
4.8 本章小节.....	68
第五章 系统测试	69
5.1 界面测试.....	69
5.2 功能模块测试.....	69
5.2.1 PDF 阴影去除模块测试	69
5.2.2 PDF 文字提取模块测试	70
5.2.3 PDF 表格提取模块测试	70
5.2.4 PDF 其他操作模块测试	70
5.2.5 图片内容提取模块测试.....	71
5.2.6 用户管理模块测试.....	71
5.3 性能测试.....	72
5.4 本章小节.....	74
第六章 总结和展望	75
6.1 论文总结.....	75
6.2 工作展望.....	76

参考文献.....77

第一章 绪论

1.1 研究背景和意义

PDF 全称 Portable Document Format, 翻译为可移植文档格式, 是电子文件格式的一种。由于这种文件格式与各种操作系统和硬件无关^[1], 也就是说 PDF 文件格式在任何平台都不会像其他文件格式容易出现排版错乱、变样的情况, 所以当前 Internet 中各领域出版物(如学位论文、教学书籍、期刊杂志等)的电子文献普遍采用 PDF 文档作为存储方案, 使其成为当前最热门的存储载体。在日常办公和生活中, 我们可以将 Word、Excel、PPT、图片等内容转换成 PDF 文件格式, 一方面可以避免排版变样, 另一方面则因为这种文件格式不可编辑, 可以有效保证原文件的安全。

PDF 文件主要分为数字 PDF 文件和图片 PDF 文件, 数字 PDF 文件内容可以直接复制, 一般由 Word 等文件转换而来。如果是图片 PDF^[2](也叫纯图像 PDF, 是由手机拍摄纸质文档或者扫描仪扫描纸质文档生成的 PDF 文件), 由于其内容本质就是一张张图片, 所以不能对 PDF 文件中的文字内容进行复制。在日常生活中, 我们也会接触很多图片 PDF 文件, 例如大量的专业资料或者学术论文都是图片 PDF 格式的。学生在纸质试卷上考试后, 学校会用扫描仪进行扫描存储为 PDF 格式让老师进行网上批阅。公司的各种通知、报表等也经常会存储为图片 PDF。

我们工作中经常需要把 PDF 文件转换为 Word 文件进行一些修改, 或者需要把 PDF 文件里的表格导出到 Excel 文件进行填写、修改数据。如果我们采取人工提取、操作和处理 PDF 文件里的内容, 例如手动打印文字、使用 Excel 文件制表然后填写表中内容等方式会存在很多的问题。首先, 由于文字或者表格数量大, 手工提取文件中的信息往往是一个繁琐的过程, 消耗了人们大量的精力去做一些和工作学习无关的事。其次, 如果工作时间有限, 慌乱中这种人工处理方法经常会出现内容处理错误、数据不一致等问题。

因此, 高效地从 PDF 文件中提取文字或者表格信息, 成为了一个亟待解决的问题。对于传统的数字 PDF 文件, 由于其内容可以复制, 并且有大量的开源工具可以用来提取其内容^[3], 所以处理起来稍微简单。目前主要困难是图片 PDF 文件的内容提取工作, 但是 OCR 技术的兴起解决了我们的难题^[4]。OCR 是英文 Optical Character Recognition 的简称, 它中文全称为光学字符识别。OCR 能够对文本文档、手写文字、自然场景等各种图片上的文字和表格信息进行分析、识别

和处理，最后可以获取文字及版面信息（对数字 PDF 文件进行图片版转换，或者直接截图，也依旧可以使用 OCR 技术来处理）。

文档文本图片与自然场景中的文本图片（例如广告牌、商标上的文字）相比，具有图片背景干净、字体相对简单且文字排布整齐的特点^[5]。自然场景图像文字识别因为受光线、拍摄角度、字体样式较多的影响而识别难度较大，并且商用价值也较高，所以现在的各大 OCR 服务提供商和最新论文几乎都是针对自然场景的，很少有专门针对文档图像识别方向的研究，导致文档图像识别技术发展停滞，市场上的文档图像识别软件的性能提升相对较慢。而且对于拍照或者扫描纸质文档生成 PDF 文档时，因为物体遮挡或者不均匀光照很容易在图片上生成阴影，影响后续的可阅读性和 OCR 处理，但是市场上现有的文档处理软件并没有专门去除文档阴影的功能。因此，借助常规技术和深度学习技术去消除文档图片上的阴影、检测和识别文档图片上的文字和表格，去设计和实现一套高性能和高准确率的 PDF 文件内容提取系统是有意义的。

1.2 国内外研究现状

1.2.1 文档图像阴影去除研究现状

文档图像阴影一般指因物体遮挡以及光照不均匀，导致移动设备（如手机）内置摄像头拍摄得到的文档图像出现的一个或多个暗区。为了方便文件传输，人们通常会使用手机扫描软件（例如我们生活中最常见的扫描全能王软件）将拍摄得到的多张文档类照片生成 PDF 文件格式，但是 PDF 文件里的图片阴影会严重影响文件的可视化质量和内容的可阅读性，同时也会影响后续的 OCR 处理。如果有打印 PDF 文件内容的需求时，打印机通常会把有阴影的部分内容打印的漆黑一片。

对于文档阴影消除技术，最早是由 Finlayson 及其团队提出的利用光度彩色不变量方法来进行阴影检测和阴影消除^[6]。如基于一维光照无关图和投影垂直于光照变化方向的二维彩色度来产生阴影不变性的图像阴影检测方法，但此方法对输入的图像质量要求很高。后来他们采用的基于二维积分的图像阴影去除方法，通过求解二维泊松方程来得到去除阴影后的图像，缺点在于算法的复杂度不但过高，而且很容易造成图像的失真。Arbel 等人利用光照条件作为依据建立图像阴影模型来检测图像的阴影区域，然后计算图像表面的光照变化来去除图像中的阴影区域^[7]，但是经过这种方法处理后，图像纹理会变得不连续。目前最新的关于阴影检测与去除的研究大都是关注于自然场景，而关注于文档图像阴影检测与去

除的研究又大部分是基于一些启发式手工设计的特征,这使得这些方法的鲁棒性和泛化性都不太理想。

2020年,CVPR 收录了一篇专门针对文档图像阴影检测和去除的学术论文^[8]。该论文提出了名为 BEDSR-Net 的网络结构,主要由 BE-net 和 SR-net 两个子网络构成,这是第一个为文档图像阴影去除而设计的深度学习模型,并且达到了 SOTA 的效果。作者 Yun-Hsuan Lin 等人为了利用和文档图像相关的特定属性,设计了一个背景估计网络 (Background Estimation Network, BE-net) 来提取文档的背景颜色,网络学习了图像背景像素和非背景像素的分布信息,然后把这些信息编码成一个注意力地图。利用估计出的全局背景颜色和注意力图,阴影去除网络 (Shadow Removal Network, SR-net) 可以较好地恢复无阴影文档图像。作者在几个基准上进行了大量定量和定性的实验,结果表明 BEDSR-Net 优于现有的文档阴影去除方法。本系统中 PDF 文件阴影去除模块所采用的深度学习算法正是来自于此。

1.2.2 OCR 技术研究现状

OCR 技术可以对图像中的文字、表格等信息进行识别。二十世纪 20 年代德国科学家 Tauscheck 最先提出来了 OCR 的概念,并且向人们表达了该技术可以极大缓解人工操作,后来他也获得了 OCR 技术专利^[9]。同时期一位美国研究员 Handel 也提出了对文字进行检测和识别的思路,但是由于当初硬件设备的限制导致无法实现,直到后来计算机的出现才变为现实。欧美国家为了将越来越多的报刊杂志、工作报表、文献资料等文字材料输入计算机硬盘中进行信息存储和处理,在二十世纪 50 年代开始了西方文字的 OCR 技术研究,以便代替人工从键盘输入到计算机。在上世纪 60 年代初期,美国的一些著名科技公司分别研制出了他们的 OCR 软件。最早商用的是一款名为 IBM1418 的 OCR 软件,它由 IBM 公司研发,但是它只能识别印刷字体的大小写字母、数字、及部分符号。

汉字由于种类繁多,字体结构相对复杂,不同书写样式的变化和字符之间的相似性导致计算机对汉字的识别是一个非常困难的事情。上世纪 60 年代,印刷体汉字识别技术才开始正式被研究。1966 年,来自美国 IBM 公司的 OCR 技术研究员发表了第一篇针对印刷体汉字识别的研究论文^[10],论文中讲述了由于汉字结构复杂、字数众多等原因,所以识别起来比英文更加困难,实验中他们利用简单的模板匹配技术(一种非常原始的模式识别技术)识别了一千个印刷体汉字。我国研究 OCR 技术相比较于欧美国家来说起步较晚,在 70 年代初期才开始对英文字母、数字、符号的识别进行研究,之后又对汉字的识别进行研究。1986 年,

国家 863 计划计算机领域研究员组织了清华大学等高校进行中文 OCR 软件的开发工作。1989 年，我国研发出了第一款名为 TH-OCR 的中文 OCR 软件，至此中文 OCR 软件正式从实验室走向了国内市场。

如今，深度学习的兴起也让 OCR 技术得到了快速的发展，深度学习的一些重要网络结构，例如卷积神经网络（CNN）、循环神经网络（RNN）等为 OCR 问题的解决提供了新的思路。2021 年 9 月，美国科技公司微软（Microsoft）的文字识别研究员们提出了一个名为 TrOCR 的模型^[11]，它是第一个利用预训练模型的端到端基于 Transformer 的文本识别模型。研究员们实验对比了其他的文字识别模型，结果表明 TrOCR 模型在文档文字识别和手写文本识别任务上都优于当前最先进的模型。本系统中 PDF 文档文字识别模块所采用的深度学习算法正是来自于此。

1.3 各章节内容及组织结构

本论文分为六个章节，对 PDF 内容提取系统的研究背景、技术选型、需求分析、总体设计、具体实现以及系统测试等内容和工作进行了详细的阐述。六个章节的内容介绍如下：

第一章，绪论。本章主要介绍了对 PDF 文件内容进行提取的研究背景及研究意义，并介绍了文档图像阴影去除和 OCR（光学字符识别）技术的国内外研究现状及成果，最后给出论文组织结构。

第二章，理论知识与关键技术介绍。本章主要介绍课题系统研究过程中所涉及到的关键知识和技术，理论知识部分主要介绍了深度学习的基础模型，技术知识主要包括服务端技术、数据库技术、前端技术以及 OCR 技术，通过对这些理论和技术的研究，选择合适的搭配方案应用于 PDF 内容提取系统中。

第三章，系统需求分析与详细设计。首先从系统的实用性角度出发，对功能模块进行需求分析，列举出系统应该具有的功能。其次对系统性能和界面展示进行分析，优化用户使用体验。最后根据系统分析，对系统的架构、功能模块、前端界面、数据库、算法模型进行详细的设计。

第四章，系统具体实现。首先制作了训练深度学习模型所使用的数据集。之后根据第三章的系统需求分析和详细设计，使用前后端框架、功能库、算法模型、数据库等技术对系统的界面和各个功能模块进行具体实现。

第五章，系统测试。从界面、功能、性能方面对系统进行测试，论述了测试步骤和测试结果。

第六章，总结和展望。对论文所做的工作内容进行分析和总结，结合现在系统存在的不足，对未来需要改进的内容进行展望。

1.4 本章小节

本章主要介绍了论文的研究背景、研究意义、相关技术的国内外研究现状和论文的各章节内容及组织结构。

第二章 理论知识与关键技术介绍

2.1 深度学习理论知识介绍

2.1.1 卷积神经网络

卷积神经网络（Convolutional Neural Networks, CNN）是当前最为流行的一种神经网络，它特别擅长处理图像相关的机器学习问题，给图像识别、目标检测、图像分割等领域带来了重大突破。微软研发的 ResNet^[12]、谷歌研发的 GoogleNet^[13] 一经推出便被广泛使用，世界围棋大赛上打败韩国围棋九段棋手李世石的 AlphaGo 也用到了这种神经网络。

在卷积神经网络之前，图片的特征提取工作主要由全连接网络完成，全连接网络将图片上的所有像素点整合成一维向量输入到网络中，然后去计算图片的特征。但是这种方式存在两个问题，一是由于图片上相邻的像素点一般具有相似的 RGB 颜色值，而整合成一维向量时这些空间信息会被丢失。二是每个像素点都要跟所有的输入神经元相直接连接，这样会增加大量的模型参数，经常会出现过拟合的现象。为了解决这两个问题，人们引入了卷积神经网络来进行特征提取，如图 2-1 所示。卷积神经网络的核心就是卷积运算，原始图片经过输入层后，会变为 RGB 数值填充的矩阵。之后与一个个填充着数字的正方形小格子（卷积核）做一系列运算，得到携带原始图片部分特征的新矩阵，这个新矩阵被称为特征图。设定不同的卷积核，我们就能找到各种各样的特征。卷积神经网络能够利用池化层降低每一层的样本数，这样做的好处是不仅降低了参数量，还能够增加模型的鲁棒性。因此卷积神经网络既能够获取到相邻像素点间的图像特征信息，又能够保持参数量的个数不随图片尺度而改变，成为了当前最为流行的神经网络。

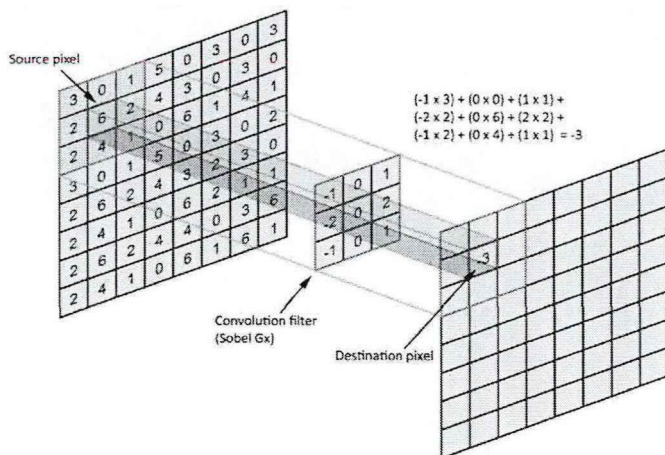


图 2-1 卷积运算示意图

2.1.2 循环神经网络

对于全连接神经网络和卷积神经网络来说，他们的输出都是只考虑前一个输入的影响而不考虑其它时刻输入的影响，即样本的处理在各个时刻是互相独立的，这对于某些需要处理序列信息（序列就是数据的前后关系）的任务来说，这些算法就无能为力了。为了解决和序列有关的任务，人们找到了循环神经网络(Recurrent Neural Network, RNN)，一个高度重视序列信息的网络。

循环神经网络的基础结构仍然是神经网络，它的隐藏层用来记录数据输入时网络的状态，在下次输入数据时，网络必须要考虑隐藏层中保存的信息，随着数据的一次次输入，隐藏层中存储的信息也在不断的更新。图 2-2 为循环神经网络示意图，其中 W 和 U 为权重矩阵，当前隐藏层的值取决于本次的输入值和上一次隐藏层计算之后存储的值，箭头所指为循环神经网络的展开图。不过基础的 RNN 有个最大的缺陷就是存在短期记忆问题，无法处理很长的输入序列，容易发生信息丢失。所以后来人们发明了一系列的改进的算法，比较经典的是 LSTM（长短期记忆网络）和 GRU（循环门单元），他们都属于 RNN 的一种^[14]。RNN 最常见的应用领域就是自然语言处理，在机器翻译、诗歌生成、语音识别等方向上大放异彩。

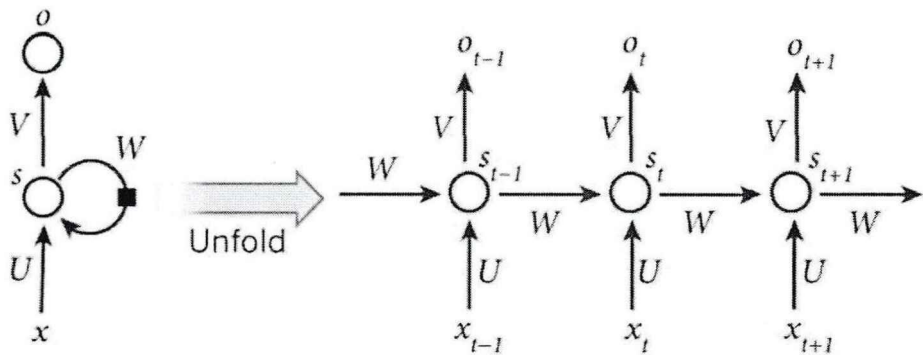


图 2-2 循环神经网络示意图

2.1.3 生成对抗网络

生成对抗网络 (Generative Adversarial Networks, GAN) 包含三个部分，生成、判别、对抗^[15]。生成和判别指的是两个独立的模块，其中生成器 (Generator) 负责依据随机向量产生内容，这些内容可以是图片、文字等信息，取决于你想要创作什么数据。判别器 (Discriminator) 负责判别接收的内容是否是真实的，通常它会生成一个概率，代表这个内容的真实程度。

对抗并不是一个模块，它指的是生成对抗网络的交替训练过程。以图片生成器为例，先让生成器生成一些“假”图片，和收集到的“真”图片一起交给判别

器，让它学习区分两者，给“真”图片高分，给“假”图片低分。当判别器能够熟练判断现有数据后，再让生成器以从判别器处获得的高分为目标不断生成更好的“假”图片，直到能骗过判别器。重复执行这一个过程，直到判别器对任何图片的预测概率都接近 0.5，也就是无法分辨图片的真假，就停止训练。训练生成对抗网络的最终目标，是获得一个足够好用的生成器，可以生成逼真的图像等数据内容。生成对抗网络如下图 2-3 所示。

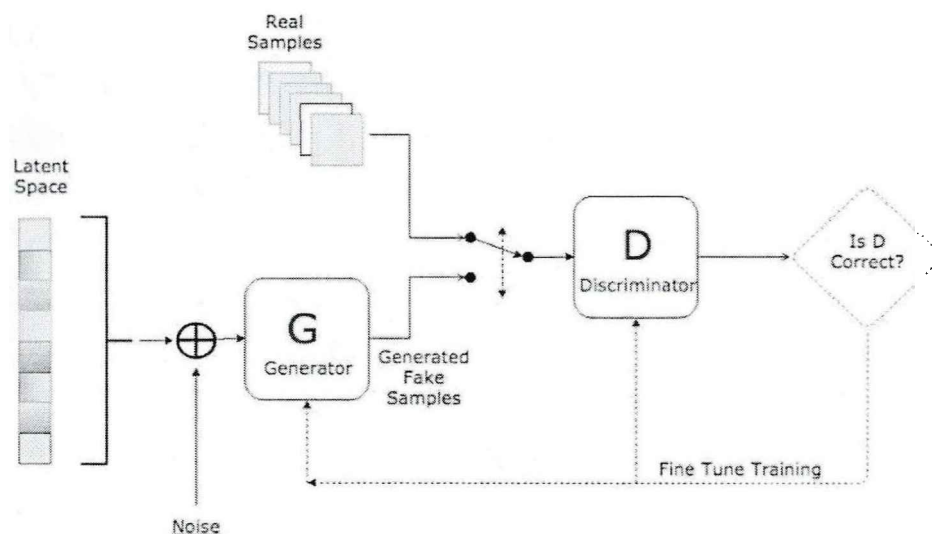


图 2-3 生成对抗网络示意图

2.2 关键技术介绍

2.2.1 数据库技术介绍

2.2.1.1 关系型数据库MySQL

MySQL 数据库是当前互联网公司使用最多的关系型数据库，它可以存储海量的数据，方便用户检索和访问，它最常用的存储引擎是 innoDB，实现了 MVCC 机制和事务机制。由于 MySQL 包含了 binlog、redo log、undo log 等各种各样的日志并且拥有锁机制，所以 MySQL 保证了数据的一致性、隔离性、原子性和持久性^[16]。MySQL 作为当前最为流行的关系型数据库，拥有以下优点：

- (1) 底层使用 C 和 C++ 编写，软件占用空间较小，服务稳定且易于安装。
- (2) 社区版本开源免费，并且拥有多种可视化工具，用户使用成本较低。
- (3) 官方社区活跃、用户群体庞大，遇到问题可以很快得到解决。
- (4) 它会自动优化用户编写的 SQL 语句，极大的提高了数据查询速度。
- (5) 支持多线程操作，提高了并发程度，可以应对大流量的访问。

(6) 拥有良好的跨平台性，可以在多个操作系统上使用。

2.2.1.2 非关系型数据库Redis

Redis 是当前互联网公司使用最多的非关系型数据库，也被称为缓存数据库。它以简单的 key-value 模式存储数据而不用依赖业务逻辑，极大的提高了数据存储的灵活性^[17]。Redis 是完全基于内存的，所以对数据的读写操作是非常快的，因此通常用它作为缓存数据库。不过 Redis 除了用作缓存外，还有很多其它用处，例如在高并发的场景下用来做分布式锁，在系统架构较为复杂时用作消息队列来为业务提供异步、解耦服务。

Redis 提供了 String、List、Set、Hash、Sorted Set 五种数据类型，支持多种复杂的业务场景，例如可以用 String 数据类型做计数器、用 List 数据类型做消息队列、用 Set 数据类型做数据的交集和并集、用 Hash 数据类型存储业务对象、用 Sorted Set 数据类型做排行榜等。Redis 还提供了数据持久化机制，可以灵活设置内存数据持久化的时间，保障数据安全。同时也支持 Lua 脚本、事务等功能，极大的提高了数据的安全性。因为 Redis 执行读写操作时，网络 IO 占用了大量的 CPU 时间，所以 Redis6.0 版本引入了多线程机制，极大的提高了网络 IO 的性能。

2.2.2 Web 开发技术介绍

2.2.2.1 Django框架

Django 是采用 Python 语言开发的一套开源 web 框架，被称为完美主义者的框架^[18]。Django 提供了大量的功能组件，开发人员无需编写复杂的底层架构代码，只需要专注于实际场景中的业务逻辑，极大提高了开发效率。Django 采用了 MTV 设计模式，即 Model（模型）、Template（模板）和 View（视图）。Model 负责处理与数据相关的事务。Template 负责处理和表现相关的决定，将数据库中的内容嵌入到 HTML 中。View 负责业务逻辑，接收请求后进行业务处理，最后返回应答。Django 集成了 ORM 组件，让开发者可以使用相同的方式操作不同类型的数据库。

Django 不但功能强大，而且简单易用。官方提供了完善的教学文档，让初学者只需要掌握基本的 Python 知识就可以入手。此外，Django 还提供了完善的错误提示机制，让开发者可以方便的定位问题。目前，Django 已经成为最流行的 Python Web 框架。

2.2.2.2 Vue框架

Vue 是一套用于构建用户界面，自底向上增量设计的 JavaScript 框架，目前各大互联网公司都有使用 Vue 来进行前端页面的开发^[19]。Vue 采用组件化模式开发，可以使一个应用拆分成一个个功能不同的模块，每个模块本质就是一个 Vue 文件，然后每个模块又可以通过一定的规则相互联系，共同组成一个页面。这种拆分方式，极大的提高了代码的复用率，维护代码也变得更加便捷。Vue 抛弃了传统的命令式编码，采用了声明式编码，让开发人员无需操作 DOM，开发过程变得简洁，提高了编写代码的效率。

Vue 之所以被广泛使用，除了性能较好外，还因为它教程众多。Vue 的创作者是中国人尤雨溪，因此 Vue 框架的官方中文文档对它的功能、使用方式介绍的比较详细，非常适合初学者入门。Vue 也是一个非常轻量级的框架，便于安装，和 Angular、React 共称为 Web 前端三大主流框架。

2.2.2.3 ElementUI组件

当我们用 Vue 自己编写组件时，经常为组件的样式设计、事件触发等问题而烦恼，不但消耗了我们大量的精力，而且设计出来的组件也不见得美观，ElementUI 的出现解决了我们这个问题。ElementUI 是饿了么的前端团队基于 Vue 开发并且开源的一套桌面端组件库，一经面世，就得到很多前端开发人员的喜爱。ElementUI 提供了各种样式的表格、导航菜单、标签、进度条等网页界面中常用的组件，还提供了主题定制功能，为大小型项目的组件开发工作提供了一套高效的解决方案，前端工程师不再需要逐个编写相应的页面组件，极大地降低了开发成本。

2.2.3 OCR 技术介绍

2.2.3.1 传统OCR技术

传统的 OCR 技术处理文字图像时，从输入文字图像到最后输出文字结果，主要经过文字图像预处理；版面处理；图像切分；文字的特征提取及模型训练；识别后对文字进行校正五个关键步骤，识别流程及技术如图 2-4 所示。

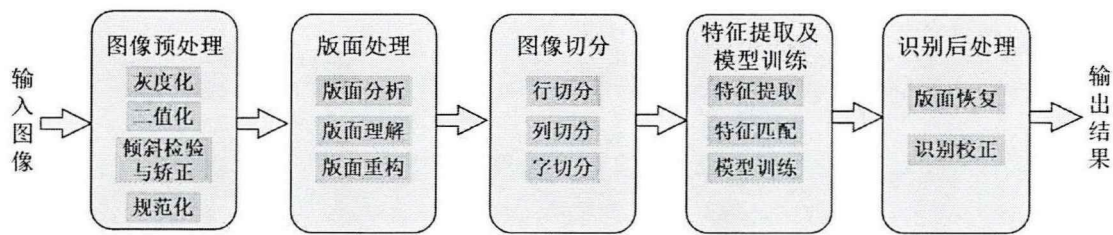


图 2-4 传统 OCR 识别文字流程图

(1) 图像预处理

输入的文字图像通常受到拍照或者扫描设备、纸质印刷质量、光照等因素的影响存在一些问题，例如文字变形、文字迷糊、文字粘连、图像角度倾斜等。所以为了保证后续文字识别的正确性，需要在文字识别之前，对带有噪声的文字图像进行预先处理。图像预处理用到的主要技术包括灰度化（滤除噪音信息）、二值化（将文字与背景分离）、倾斜检测与校正、图像平滑处理、规范化处理等操作。

(2) 版面处理

文字图像的版面处理主要包括版面分析、版面理解、版面重构三部分。版面分析就是把图像分割为横排文本、竖排文本、表格、图片等不同的部分，版面理解指获取文本的层次关系、阅读顺序等逻辑结构，版面重构指根据版面分析和版面理解的结果进行文字版面的还原。

(3) 图像切分

图像切分可以分为行切分、列切分、字切分。根据语言的不同（如汉字和英文），切分的方式也会有所不同。经过这些切分处理后，对得到的单个文字或单词才能进行后续的处理。

(4) 特征提取及模型训练

特征提取是指人们利用经验知识，利用文字的边缘特征、变换特征、结构特征等进行特征库的设计，特征匹配是指从人们设计的文字特征库中找到与待识别文字相似度最高的文字。模型训练也叫分类器的训练，对输入的文字和特征库中的文字进行“距离”判定，从特征库中寻找出“距离”最短的文字输出为匹配结果，常用的计算方法有余弦相识度、欧式距离等。

(5) 识别后处理

因为形近字等原因，文字识别的结果可能并不准确，需要进行识别校正等操作。识别校正主要通过语言模型来判断输出的语句是否“语义化”，进而进行文字纠正。

2.2.3.2 深度学习OCR技术

随着手机的普及,越来越多人倾向于使用手机相机来获得文字图片,但是手机拍摄可能会导致图像中文字的形状扭曲、粘连,并且文字背景也可能出现阴影。而传统的 OCR 技术严重依赖人工设计的字符特征来训练文字识别模型,在字体扭曲、模糊或存在阴影背景干扰时,文字识别模型的泛化能力会迅速下降。在字符切分时,如果字体存在粘连,也会严重影响后续的文字识别。人工智能时代的到来使得基于深度学习的 OCR 技术代替了传统的 OCR 技术,深度学习的 OCR 方法不但可以面对更加复杂的文字图像场景,而且可以借助上下文来判断形似字的问题,准确率和识别效率都得到了大幅度的提升,成为目前主流的 OCR 识别方法。下面介绍基于深度学习的文字检测、文字识别、表格识别方法。

(1) 文字检测方法

文字检测就是将图像中的文本区域定位出来,和目标检测有相似之处。下面列出几种常用的文字检测模型:

- CTPN^[20]: 网络结构和 Faster-RCNN^[21]类似,主要包括卷积层、LSTM 层、全连接层三部分,只能检测水平或者略微倾斜的文本行。
- SegLink^[22]: 提出了 Segment 和 Link 的思想,首先检测一个个带有方向的 Segment,它可能是一个单词或者若干个字符。然后通过 Link 将属于同一个单词或者文本行的 Segment 关联起来。
- TextBoxes++^[23]: 在 SSD 网络的思想上进行改进,使用了密集的 default box 检测文本,长卷积核提取文本特征。与 TextBoxes^[24]相比,可以检测任意方向的文本行。
- CRAFT^[25]: 一种基于字符的文字检测模型,首先检测单个字符,然后根据字符间的位置关系把它们连接成文本目标。该模型优势在于只需要很小的感受野即可检测较长文本。

(2) 文字识别方法

文字识别就是根据文字检测得到的文本行坐标,识别出文本框中的文字内容。下面列出几种常用的文字识别模型:

- CNN+RNN+CTC^[26]: 循环层采用了双向的 LSTM 网络提取图像特征,引入 CTC 解决字符对齐问题。该模型可以从变长序列标签中直接学习,但是对角度较大的文字难以识别。
- CNN+RNN+Attention: 基于注意力机制的解码架构,解决了文字识别任务中输入字符和输出字符的长度不对齐问题。

- TrOCR^[27]: 基于图像 Transformer 结构和文本 Transformer 结构, 用于提取视觉特征和语言建模, 目前在印刷文本识别和手写文本识别上具有最先进的性能。

(3) 表格识别方法

表格识别就是找到图像中的表格, 然后提取表格中的结构和数据信息。下面列出几种常用的表格识别模型:

- TableNet^[28]: 一种端到端的图像语义分割模型, 使用了编码器-解码器架构, 利用 VGG-19 作为基础网络。该模型可以同时处理表格检测和结构识别。
- CascadeTabNet^[29]: 使用两阶段学习策略, 以 HRNet 为基础网络, 利用语义分割的方法检测表格区域, 之后从检测到的表格区域进行结构识别。
- DB+CRNN+RARE^[30]: 百度飞桨最新的表格识别架构, DB 模型检测文本坐标, CRNN 模型识别文本内容, RARE 模型预测表格结构和单元格位置。

2.2.4 其他技术介绍

2.2.4.1 PyTorch框架

PyTorch 是由美国 Facebook 公司开发的一款深度学习框架, 它本质上是加入了 GPU 支持的 Numpy, 而且可以用来设计、构建、训练深度神经网络。如果初学者对 Python、Numpy 以及深度学习知识有一定的了解, 会非常轻松上手 PyTorch 框架。

在工作中, 经常需要调试程序代码, 如果使用 TensorFlow 这样的静态图框架, 需要从会话中请求要检查的变量, 或者去学习额外的 TensorFlow 调试器工具。而 PyTorch 作为一款动态图框架, 在程序中调试 PyTorch 代码的方式和调试 Python 代码一样, 可以使用 pdb 并且能在任何一行代码前打断点。PyTorch

属于轻量级的框架, 集成了英伟达的 CuDNN、英特尔的 MKL 来优化计算速度, 无论运行多大尺寸的神经网络, 它总是非常高效的。PyTorch 提供了一个简单的 API, 可以方便的保存模型所有的权重或序列化整个类, 因此使用 PyTorch 来保存以及加载模型都很简单。迄今为止, PyTorch 和 Tensorflow 是最受欢迎、应用最为广泛的两个深度学习框架。

2.3 本章小节

本章为系统的具体实现提供了技术基础。首先介绍了深度学习理论知识，之后对开发本系统所需的数据库技术、前端技术、服务端技术、OCR 技术等进行了介绍。

第三章 系统需求分析与详细设计

本章首先从系统需求开始分析,明确系统应该具有的核心功能以及模块结构,并且从用户的角度出发,明确系统应该具有的附加功能。之后根据系统的需求分析完成系统的设计,制定出一套功能完善、安全实用的 PDF 内容提取系统框架。

3.1 系统需求分析

3.1.1 功能需求分析

3.1.1.1 PDF阴影去除功能

在日常生活或者工作中,为了方便信息的传输,人们经常用手机相机拍摄一些纸质文件,然后转换为电子版 PDF 发给他人。但是拍摄得到的文档图像很容易出现大片的阴影,原因是光源经常被拍摄的手机或者其他物体挡住。即使光源没有被遮挡,当在现实世界拍照时,在纸质文件上的光照通常是不均匀的,从而拍摄得到的文档图像上出现不均匀的阴影。

当我们把这些带阴影的文档图像转换为一个 PDF 格式文件时,这些阴影依然会存在于 PDF 文件中。文件上的阴影通常会带来两个问题。首先是阴影会影响文件的美观度和清晰度,给人们的阅读造成了困惑,留下了文件质量较差的印象。其次,我们可能需要利用 OCR 技术,对图片版 PDF 里的内容进行提取,方便我们后续的编辑和使用,但是这些阴影使文件背景复杂度增加,进而影响了 OCR 的性能。现在市面上的 PDF 处理软件都没有阴影去除这个功能,因此,给本系统添加一个 PDF 阴影去除功能是必要的。系统首先接收用户上传的带有阴影的 PDF 文件,经过相关处理后生成无阴影的 PDF 文件供用户下载。

3.1.1.2 PDF文字提取功能

由于 PDF 格式文件兼容性好,不会引起排版的混乱,因此越来越多的文档采用 PDF 格式。但是它有一个缺点,用户无法便捷地对文件内容进行编辑,如果是图片型 PDF,用户甚至不能对 PDF 里的文字内容进行复制。要想解决这些问题,只能通过专业的 PDF 工具对数字 PDF 进行编辑,对图片 PDF 转为 Word 格式后再进行编辑。但是正版专业的 PDF 编辑器使用价格相对较高,而且部分

编辑器对图片 PDF 文字内容进行提取时，因为所使用的文字检测和识别算法模型相对较老，所以识别速度和准确度都不高。

因此，为了方便用户对 PDF 文件内容进行编辑，需要实现一个 PDF 文件转 Word、TXT 文件的功能。为了文件转换更加高效，针对 PDF 文件类型（数字 PDF 和图片 PDF）的不同，可以采用不同的方法去处理 PDF 文件里的文字。有时候用户并不需要提取 PDF 文件里的全部文字，例如，用户只想提取第 3-5 页的全部文字。因此，系统应该提供一个让用户选择转换页码范围的提示框，这样不但符合了用户的需求，也节省了系统的响应时间，提高了吞吐量。

3.1.1.3 PDF 表格提取功能

PDF 文件内容通常会包含大量的表格信息，使用人工的方式提取表格信息经常会出现表格处理错误、不一致等问题。而上述的 PDF 文字提取功能所使用的算法模型主要是针对 PDF 文件里的文本行文字的，对表格里文字提取效果会变差。因此，高效地从 PDF 文件中找到表格，同时准确的提取表格中的文字和结构信息成为了一个亟待解决的问题。

PDF 内容提取系统应该增加针对表格信息的提取功能，使用专门的表格检测和结构识别算法模型，读取用户上传的 PDF 文件，自动提取其中的表格信息并导出为 Excel 文件（人们通常用 Excel 格式文件存储和编辑表格信息）供用户下载。为了只提取用户需要的表格信息，提高系统的运行效率，表格提取功能也应该提供一个提示框，让用户选择合适的页码范围。

3.1.1.4 PDF 其他操作功能

系统除了具有去除阴影、提取文字、提取表格这些关键功能外，也应该包含一些其他的 PDF 操作功能。例如人们扫描纸质文档转为 PDF 文件时，经常颠倒了横向模式和纵向模式，我们系统可以开发出一个旋转页面功能，旋转有问题的页面。有时候我们希望将两个或多个 PDF 合并到一个 PDF 中，或者需要将 PDF 拆分为多个 PDF，我们系统就应该具有合并 PDF 和拆分 PDF 的功能。

水印是一种电子文档或者图片上的图案，它可以保护人们知识产权，当我们制作了 PDF 文件时，也可以在文件中加上我们自己的水印。如果使用正版专业的 PDF 软件，即使我们只需要使用一次这种功能，通常也需要花钱购买软件会员，因此 PDF 内容提取系统开发出这些常用的功能也是必要的。

3.1.1.5 图片内容提取功能

当用户对纸质文档拍照后，可能不需要转换为 PDF 文件，而只想对单张图片里的内容进行提取，或者去除图片里的阴影。因此，系统应该提供文档类图片内容提取功能，可以去除图片阴影，也可以提取图片里的文字、表格内容。

图片内容提取功能流程应为用户上传所需要的文档类图片，根据用户选择，可以把图片里的文字转为 Word 文件，表格转为 Excel 文件，或者得到无阴影图片。之后用户可以把文件下载到合适的位置。

3.1.1.6 用户管理功能

系统应该具有账号注册、个人信息修改、系统登录等基本功能，并且一个手机号只能申请一个账号。对于后台管理人员，应该具有权限管理功能，不同角色可以赋予不同的权限。

为了更深入的了解用户需求，需要统计所有用户使用各个模块的总次数，以便于针对某些模块开发出更加实用的功能。并且由于服务器数量的限制，为了节约系统维护成本，后期考虑限制单个用户的使用次数，避免单个用户大量使用系统而影响其他用户的正常使用。所以系统应该具有用户使用次数统计功能，结合系统压测结果，计算出每个用户每天最大使用次数。

系统的各项功能是否合理，文字识别准确率和速度是否达到预期标准，除了靠系统开发者自测外，还需要根据用户的反馈不断优化。因此系统应该提供用户评论功能，并且用户可以给其他用户的评论点赞，评论的点赞数越多，显示也就越靠前。热评功能有利于开发者发现系统存在的一些问题，方便进行系统维护。

3.1.2 性能需求分析

系统的性能决定了用户的体验，也决定了系统本身存在的意义，结合本系统具有的功能，提出了以下的性能需求：

(1) 准确性

对于 PDF 内容提取系统来说，最重要的就是准确的提取出 PDF 里的文字或者表格数据内容，这样系统的设计才有意义。识别结果的准确率也是判断深度学习算法模型优劣的关键指标，根据准确率去不断优化算法模型。

(2) 响应时间

响应时间即用户在操作相应功能后等待系统返回结果的时间,等待的时间越短,就代表性能越好,用户的体验也就越好。所以为了减少响应时间,系统通常采用并发设计,提高系统的吞吐量,降低系统的响应时间。

(3) 稳定性

稳定性即用户在操作本系统时,不会出现页面崩溃或者卡死现象,用户的数据也不会发生丢失。

(4) 可扩展性

系统各模块耦合程度较低,独立性较强,方便后续的系统功能扩展、新增功能模块。

3.1.3 界面需求分析

界面是用户和系统直接交互的媒介,是系统的重要组成部分。布局美观、操作友好的界面可以降低用户使用成本、提升用户使用体验。因此,提出了以下界面设计需求:

(1) 布局合理

系统界面整体应该简洁清晰,导航栏、菜单栏等各种组件应该出现在合适的位置,符合多数用户的操作习惯。

(2) 简单易用

尽量简化用户的操作步骤,让用户可以更加简单的使用系统的各种功能,降低用户的使用难度。

(3) 美观性

系统的界面应该配色合理,字体大小适中,各个页面风格基本保持一致,给用户带来美的感受。

3.2 系统总体设计

3.2.1 架构设计

系统采用分层式架构设计,各层职责清晰,代码容易被复用,开发者只需要关注每一层的具体实现。架构图如 3-1 所示,具体可以分为前端 UI、展示层、业务层、算法层、数据库、运行环境六个部分。

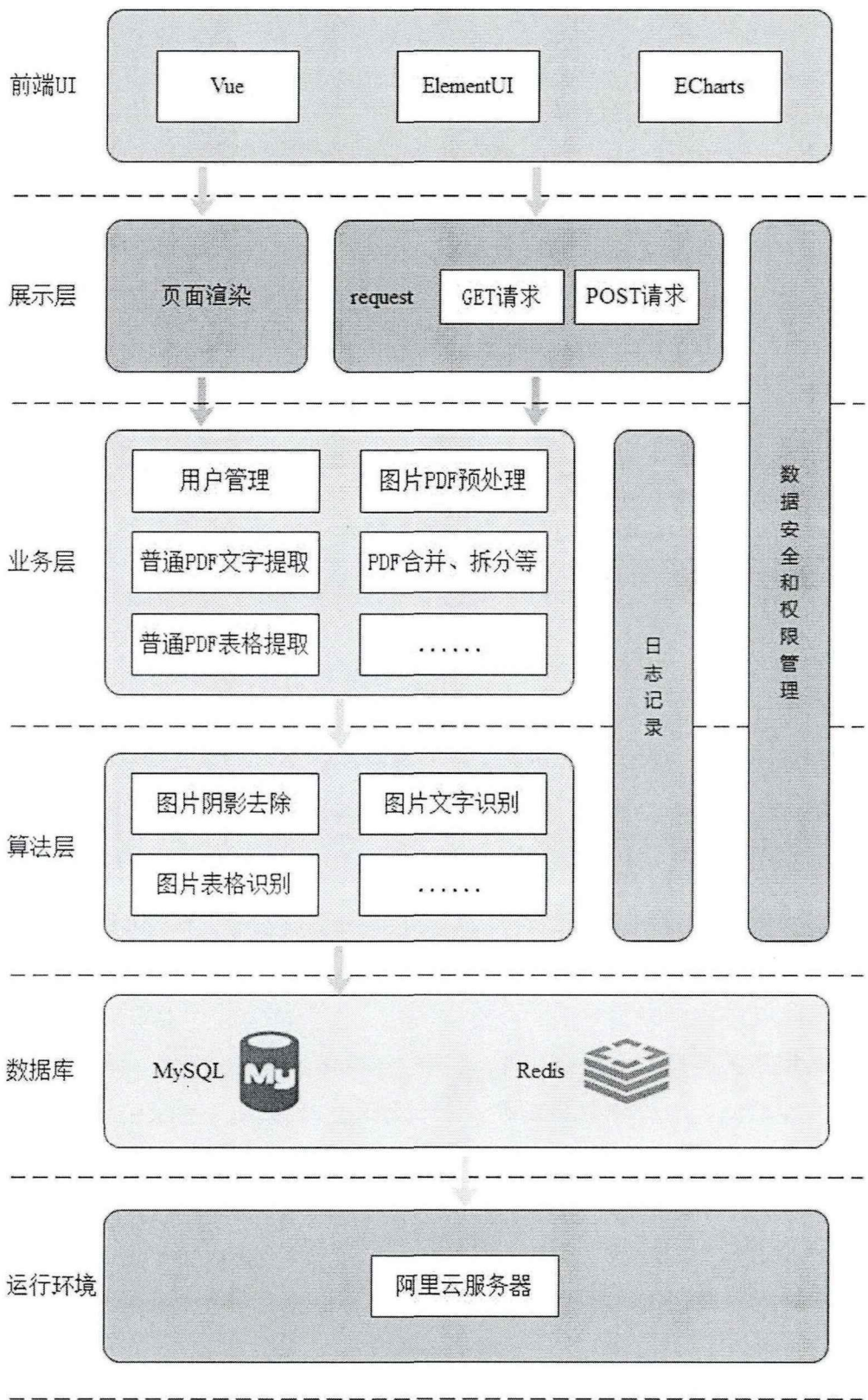


图 3-1 系统架构图

前端 UI（界面设计）以渐进式框架 Vue 为基础，使用 ElementUI 组件库提供导航栏、进度条、提示框等组件，帮助系统页面快速成型。用户使用次数通过 ECharts 中的图表功能进行展示。

展示层负责了页面渲染，也是系统前端和后端的桥梁。HTML 的渲染由该层完成，HTTP 请求分为 GET 请求和 POST 请求，访问查询接口使用 GET 请求，访问添加、更新、删除接口使用 POST 请求。

对于数字 PDF，在后端业务层直接解析 PDF 对象的详细信息，然后提取出所需要的文本内容，比使用 OCR 技术提取效率更高，识别结果也更加准确，所以后端业务层直接实现对数字 PDF 的文字和表格提取。对于图片 PDF，需要进行预处理，根据图片 PDF 的页码数转为多张图片，使算法层能够得到正确格式的数据，之后根据算法层返回的结果再生成用户所需的文件。该层还完成了 PDF 合并拆分、用户管理、日志记录、权限校验等多个任务。

算法层根据业务层传送的数据，调用不同的算法模型去执行不同的任务，之后把得到的结果返回给业务层。例如，业务层可以调用文字检测模型得到文本行坐标、调用文字识别模型得到文本行字符串、调用表格识别模型得到表格结构和单元格坐标。

数据库使用了关系型数据库 MySQL 和非关系型数据库 Redis。MySQL 存储用户的相关数据，Redis 配合 MySQL 做高速缓存，并且其特定的数据结构可以帮助我们完成系统的注册、统计、评论等功能。

为了便于系统的访问和维护，我们把系统部署在阿里云服务器，该服务器配置简单、扩容方便，性能也较为可靠。我们还可以给系统网站申请域名，使用户容易记住网站地址。

3.2.2 功能模块设计

系统基于深度学习算法、PDF 工具库、图像处理等技术实现了 PDF 文件阴影去除和相关内容的检测和识别，并通过主流前后端框架和数据库技术完善了系统的其他部分。系统分为 PDF 阴影去除模块、PDF 文字提取模块、PDF 表格提取模块、PDF 其他操作模块、图片内容提取模块、用户管理模块。系统的功能模块设计如图 3-2 所示。

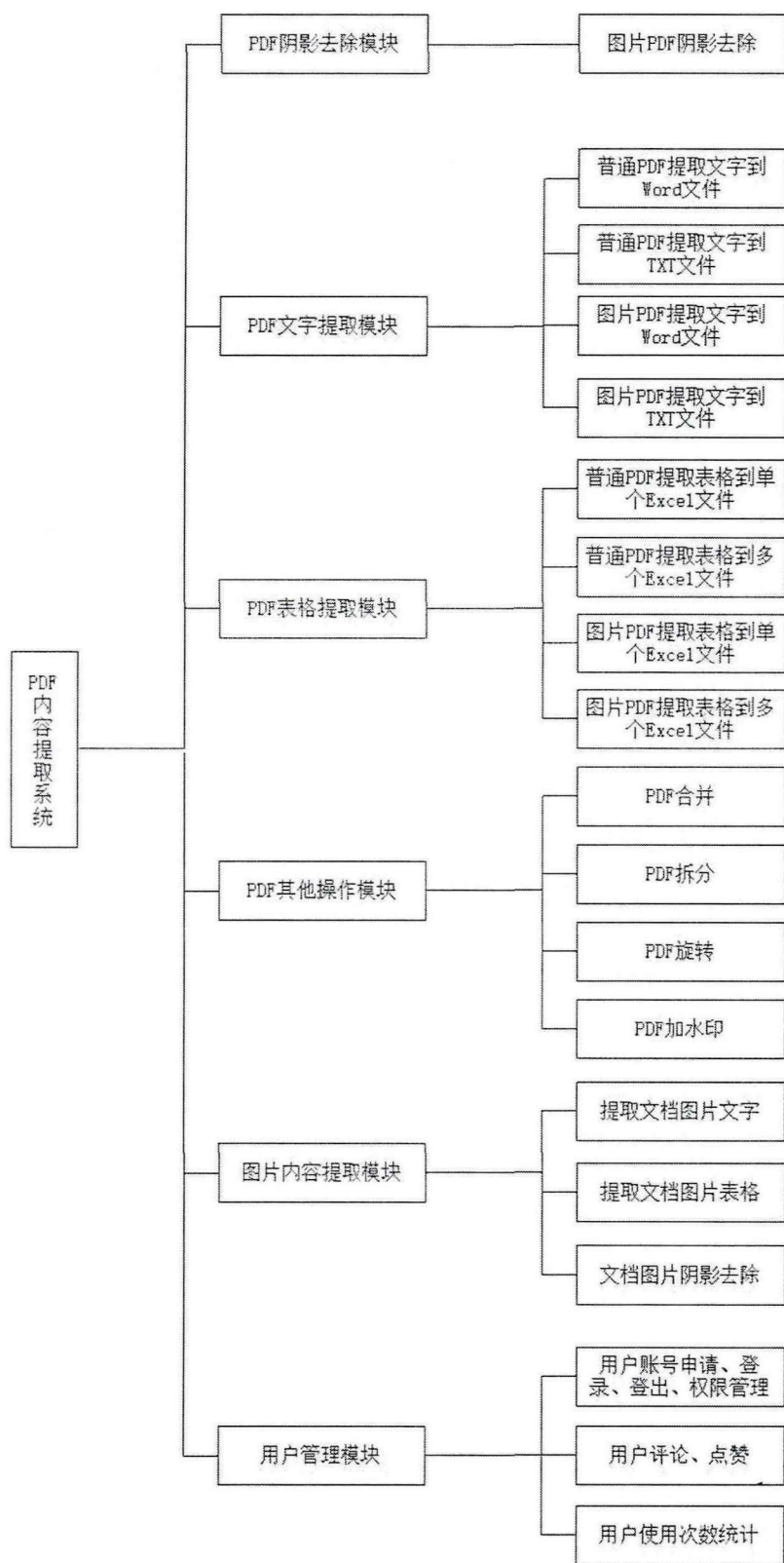


图 3-2 系统功能模块图

PDF 阴影去除模块可以检测到用户上传的图片 PDF 里的阴影背景(数字 PDF 不会出现阴影背景)并对其进行去除,提升了文档的质量和清晰度,也对 OCR 性能有所提升。

PDF 文字提取模块针对数字 PDF 文件和图片 PDF 文件的内容形式不同,使用了不同的处理方式。为了方便用户排版文字或者复制文字,系统可以生成 Word 或 TXT 两种不同格式的文件。

PDF 表格提取模块同样针对数字 PDF 文件和图片 PDF 文件的内容形式不同,使用了不同的方式进行表格提取。考虑到 PDF 文件里通常会有多个表格,生成的提取结果可以放在一个 Excel 文件里的单个工作表,也可以放在一个 Excel 文件里的多个工作表。

PDF 其他操作模块可以实现一个 PDF 文件拆分为多个 PDF 文件,也可以让多个 PDF 文件合并为一个 PDF 文件。该模块提供了 PDF 旋转功能,对横向的页面进行旋转 90 度回到正确的纵向方向。模块还提供了 PDF 加水印功能,用户可以加文字水印,也可以上传图片添加图片水印来防止文件被他人盗用。

图片内容提取模块可以实现文档类图片的文字提取、表格提取、阴影背景去除。当用户只需要对 PDF 文件里的某一些区域进行处理时,可以对该区域截图,然后使用该模块的功能进行相关操作。

用户管理模块实现了用户的注册、登录、登出等基本操作,为了便于维护系统网站,添加了管理员的角色进行权限处理。用户可以在评论区评论、点赞,为系统提出建议。为了更好的运营系统和了解用户需求,添加了用户使用次数统计功能,系统管理员可以方便的查看用户更喜欢使用哪个功能模块。

3.2.3 前端界面设计

PDF 内容提取系统前端页面使用 Vue 框架和 ElementUI 组件库进行开发,其需要包含上节所设计的所有功能模块。前端界面设计图如图 3-3 所示,前端界面由顶部栏、导航栏、文件读取区、流程控制区四部分组成。顶部栏负责展示系统名称、登录和退出按钮。用户点击登录按钮会跳转到登录页面进行登录,未注册账号的用户可使用个人手机号免费注册系统账号。用户使用完毕后,点击退出按钮即可退出系统。导航栏包含了该系统所有的功能模块按钮,点击某个具体的模块按钮后,会在右侧文件读取区和流程控制区显示该模块的具体内容,所以文件读取区和流程控制区要由功能模块的实际需求进行设计。

文件读取区负责读取用户上传的文件,支持文件拖拽上传和选择上传两种方式。某些功能模块的文件读取区会有下拉框,例如 PDF 文字提取功能,用户可以通过下拉框选择是生成 Word 文件还是 TXT 文件。不同功能模块的文件读取区也会有不同的文字说明,指引用户完成操作。流程控制区负责展示用户上传的文件编号、名称、总页数,用户可以自由选择需要提取的 PDF 文件页码范围,点击开始按钮即可开始提取,当提取状态变为已完成后即可下载相应的文件。

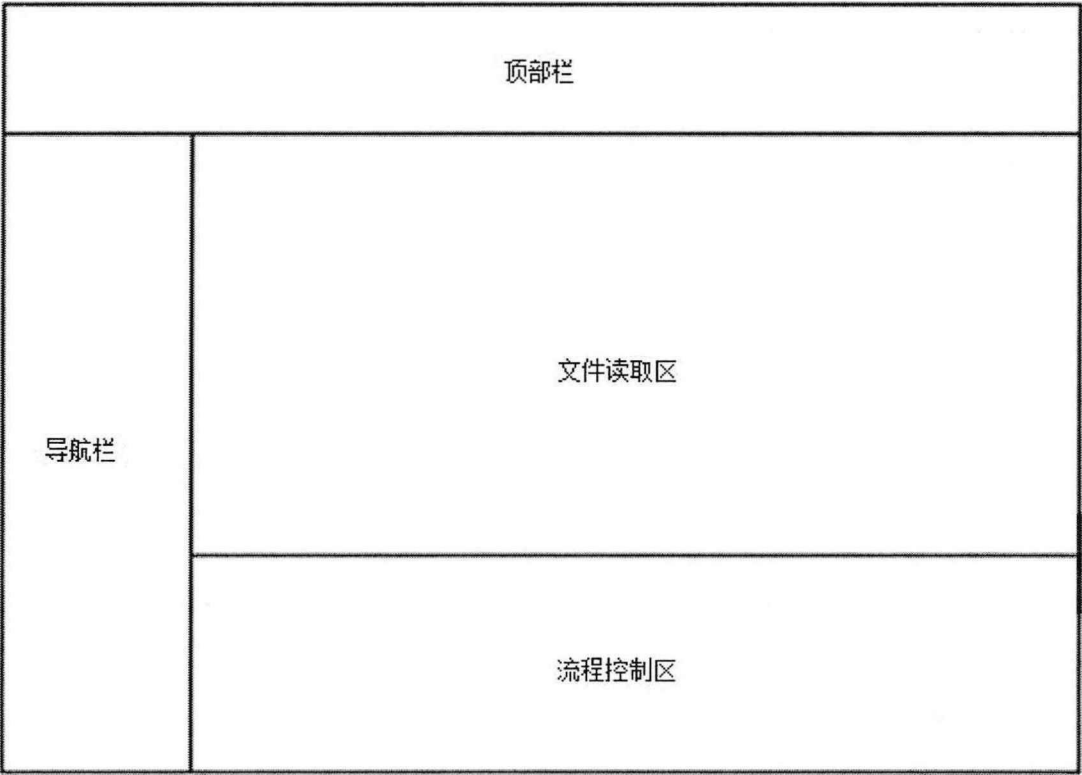


图 3-3 前端界面设计图

3.2.4 数据库设计

本系统使用 MySQL 数据库存储用户信息、权限信息、使用信息、评论信息。用户权限管理采用了经典的 5 张表设计，把用户、角色、权限进行了关联，一个用户可以拥有多个角色，一个角色可以拥有多个权限。这种设计构成了“用户-角色-权限”授权模型，通过这种多对多的关系模型，实现了系统的功能使用、数据访问的安全管理。用户使用信息和评论内容各分一张表设计。系统数据库表结构的设计参考了三大范式，表格信息如下所示。

(1) 用户表 `acl_user`

用户表如下表 3-1 所示，包含了账号 `id`、用户名、密码、手机号、昵称、年龄、性别、头像地址、用户签名、逻辑删除、创建时间、修改时间。其中逻辑删除字段有两个值，当值为 1 表示已删除，当值为 0 表示未删除。

表 3-1 用户表 `acl_user`

字段名	类型	备注描述	长度
id	char	账号 id	19
username	varchar	用户名	20
password	varchar	密码	32
mobile	varchar	手机号	11
nick_name	varchar	昵称	50
gender	tinyint	性别	1
age	varchar	年龄	3
head_address	varchar	头像地址	255
token	varchar	用户签名	100
is_deleted	tinyint	逻辑删除	1
gmt_create	datetime	创建时间	0
gmt_modified	datetime	更新时间	0

(2) 角色表 `acl_role`

角色表如下表 3-2 所示，包含了角色 id，角色名字、角色编码、备注、逻辑删除、创建时间、修改时间。

表 3-2 角色表 `acl_role`

字段名	类型	备注描述	长度
id	char	角色 id	19
role_name	varchar	角色名称	20
role_code	varchar	角色编码	20
remark	varchar	备注	255
is_deleted	tinyint	逻辑删除	1
gmt_create	datetime	创建时间	0
gmt_modified	datetime	更新时间	0

(3) 权限表 `acl_permission`

权限表如下表 3-3 所示，包含了编号 id、编号父 id、权限名称、类型、权限值、访问路径、组件路径、状态、逻辑删除、创建时间、修改时间。其中类型有两个值，当值为 0 表示菜单，当值为 1 表示按钮。状态也有两个值，当值为 0 表示禁止，当值为 1 表示正常。

表 3-3 权限表 `acl_permission`

字段名	类型	备注描述	长度
id	char	编号 id	19
pid	char	编号父 id	19
name	varchar	名称	20
type	tinyint	类型	1
permission_value	varchar	权限值	50
path	varchar	访问路径	100
component	varchar	组件路径	100
status	tinyint	状态	1
is_deleted	tinyint	逻辑删除	1
gmt_create	datetime	创建时间	0
gmt_modified	datetime	更新时间	0

(4) 用户角色表 `acl_user_role`

用户角色表如下表 3-4 所示，主要对用户表和角色表进行关联，包含了主键 id、角色 id、用户 id、创建时间、修改时间。

表 3-4 用户角色表 `acl_user_role`

字段名	类型	备注描述	长度
id	char	主键 id	19
role_id	char	角色 id	19
user_id	char	用户 id	19
gmt_create	datetime	创建时间	0
gmt_modified	datetime	更新时间	0

(5) 角色权限表 `acl_role_permission`

角色权限表如下表 3-5 所示，主要对角色表和权限表进行关联，包含了主键 id、角色 id、权限 id、创建时间、修改时间。

表 3-5 角色权限表 `acl_role_permission`

字段名	类型	备注描述	长度
id	char	主键 id	19
role_id	char	角色 id	19
permission_id	char	权限 id	19
gmt_create	datetime	创建时间	0
gmt_modified	datetime	更新时间	0

(6) 使用记录表 u_use_record

使用记录表如下表 3-6 所示, 包含了主键 id、用户 id、功能类型、逻辑删除、创建时间、修改时间。其中功能类型指用户使用了系统的哪个具体的功能。

表 3-6 使用记录表 u_use_record

字段名	类型	备注描述	长度
id	char	主键 id	19
user_id	char	用户 id	19
type	varchar	功能类型	20
is_deleted	tinyint	逻辑删除	1
gmt_create	datetime	创建时间	0
gmt_modified	datetime	更新时间	0

(7) 评论表 u_comment

评论表如下表 3-7 所示, 包含了评论 id、评论父 id、用户 id、昵称、评论内容、逻辑删除、创建时间、修改时间。如果评论父 id 不为空, 代表该评论为某条评论的子评论。

表 3-7 评论表 u_comment

字段名	类型	备注描述	长度
id	char	评论 id	19
fatherId	char	评论父 id	19
user_id	char	用户 id	19
nickname	varchar	昵称	50
content	varchar	评论内容	500
is_deleted	tinyint	逻辑删除	1
gmt_create	datetime	创建时间	0
gmt_modified	datetime	更新时间	0

3.2.5 算法模型设计

3.2.5.1 阴影去除模型设计

PDF 阴影去除模块所利用算法模型名为 BEDSR-Net, 它是第一个专门为文档图像阴影去除而设计的模型, 由国立台湾大学的学者提出, 并被 CVPR 2020 收录。

现有的文档图像阴影去除算法都采用一些手工制作的启发式方法去寻找文档图像的特定特征。但是, 这些算法通常只对部分文档图像工作有效, 对其他文

档图像效果却很差。近些年来,深度学习被广泛应用于许多计算机视觉问题中,但是基于深度学习的图像阴影去除算法大都是针对自然场景的。虽然自然场景图像的阴影去除算法也可以应用于文档图像,但是会存在一些问题。例如这些算法需要大量的文档阴影图像和无阴影图像进行训练,使它们能够处理文档图像。其次,这些算法没有利用文档图像的特有属性,因此性能并不是很好。

针对以上问题, Yun-Hsuan Lin 等人推出了 BEDSR-Net, 它主要由 BE-net 和 SR-net 两个子网络构成。BE-net 的输入为带有阴影的文档图像, 根据文档图像的特定属性, 计算出了反应纸张颜色的背景颜色图和反应阴影分布的注意力热图, 这些信息在改善图像质量和减少模型参数方面是极其有用的。之后带有阴影的文档图像、背景颜色图、注意力热图这三个图像经过 SR-net 之后得到了去掉阴影的文档图像。BEDSR-Net 模型结构如下图 3-4 所示。

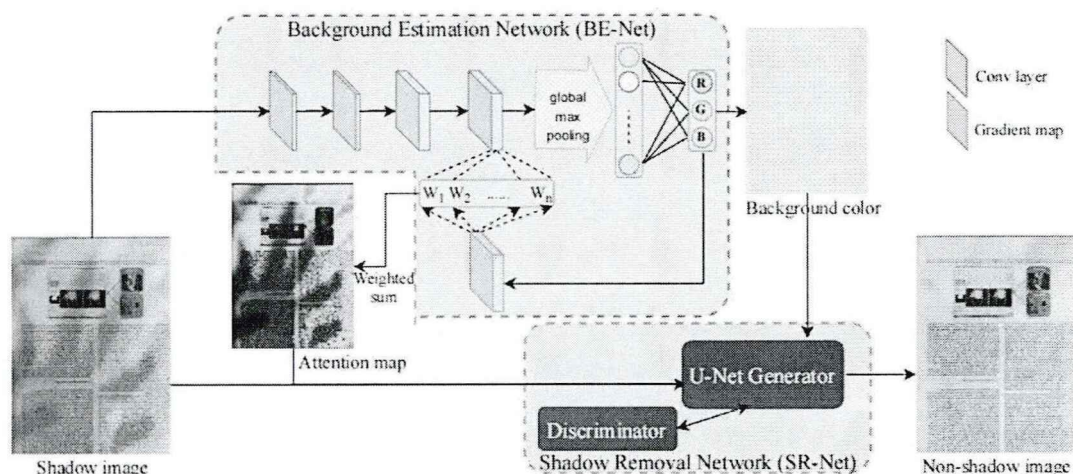


图 3-4 BEDSR-Net 模型结构图

BEDSR-Net 训练集可以表示为:

$$D = \{S_i, N_i\}_{i=1}^N \quad (3-1)$$

由 N 对 (S_i, N_i) 图像组成, 其中 S_i 为带阴影的图像, N_i 为对应的无阴影图像。训练后, BEDSR-Net 形成一个函数:

$$\Psi_{BEDSR}(S) = \tilde{N} \quad (3-2)$$

该函数接受阴影图像 S , 并返回一个与真实的非阴影图像 N 近似的预测的非阴影图像。

BE-Net 由四个卷积层、一个全局最大池化层、一个全连接层组成。卷积层负责提取输入图像的特征信息, 全局最大池化层将特征进行压缩、减小计算量, 全连接层预测输出 RGB 值。背景颜色图可以让用户手动选择一个背景非阴影的区域, 但是为了减轻人工操作, 作者编写了一个脚本自动获取真实图像的背景颜色。首先将无阴影图像的像素根据其强度值聚类成两组, 两组通常分别对应于内

容和背景。在聚类方面，采用了期望最大化的高斯混合模型(GMM)。无阴影图像不含阴影，仅由背景和内容构成，并且文档的背景颜色通常比较亮。因此，作者将更亮的组定为背景，更暗的组定为文件内容。有了背景颜色和含阴影图像，就能以监督学习的方式最小化损失函数：

$$\mathcal{L}_{color} = \sum_{i=1}^N \|b_i \Psi_{BE}(S_i)\|_1 \quad (3-3)$$

SR-Net 是一个条件生成对抗网络，主要由生成器和判别器构成。生成器采用 U-Net 模型，判别器采用 PatchGAN。U-Net 是一个全卷积神经网络，由编码器和解码器组成。在每个空间分辨率上，解码器的特征将通过跳跃连接与编码器的特征结合起来。SR-Net 的损失函数为：

$$\mathcal{L} = \lambda_1 \mathcal{L}_{data} + \lambda_2 \mathcal{L}_{GAN} \quad (3-4)$$

SR-Net 损失函数中的 $\mathcal{L}_{data}$ 是预测的无阴影图像和真实的无阴影图像的偏差：

$$\mathcal{L}_{data} = \mathbb{E}_{S_i, N_i \sim P_{data}(S_i, N_i)} \|N_i - \tilde{N}_i\| \quad (3-5)$$

SR-Net 损失函数中的 $\mathcal{L}_{GAN}$ 是 GAN 网络的损失函数：

$$\begin{aligned} \mathcal{L}_{GAN} = & \mathbb{E}_{S_i, N_i \sim P_{data}(S_i, N_i)} [\log D(S_i, N_i)] + \\ & \mathbb{E}_{S_i \sim P_{data}(S_i)} [\log (1 - D(S_i, \tilde{N}_i))] \end{aligned} \quad (3-6)$$

作者将 BEDSR-Net 与四种最先进的图像阴影去除方法进行了比较，其中包括 Bako 等人^[31]、Kligler 等人^[32]、Jung 等人^[33]提出的三种传统的文档阴影去除方法，以及一种最先进的基于深度学习的自然图像阴影去除方法 ST-CGAN。为了显示背景估计模块的重要性，作者将 BE-Net 合并到 ST-CGAN 中，并将其命名为 ST-CGAN-BE。

实验结果如下图 3-5 所示，Bako 等人的方法和 Kligler 等人的方法在有强阴影时都显示出剩余的阴影边缘，Jung 的方法经常导致结果发生严重的颜色变化，通常比真实无阴影的图像要明亮得多。当文档图像中有很大的阴影时，ST-CGAN 会出现明显的残留阴影。ST-CGAN-BE 相比于 ST-CGAN，几乎没有了残留阴影，在可视化质量上有了很大的提升。最后是作者的 BEDSR-Net，有更少的伪影，非常接近真实无阴影图像。

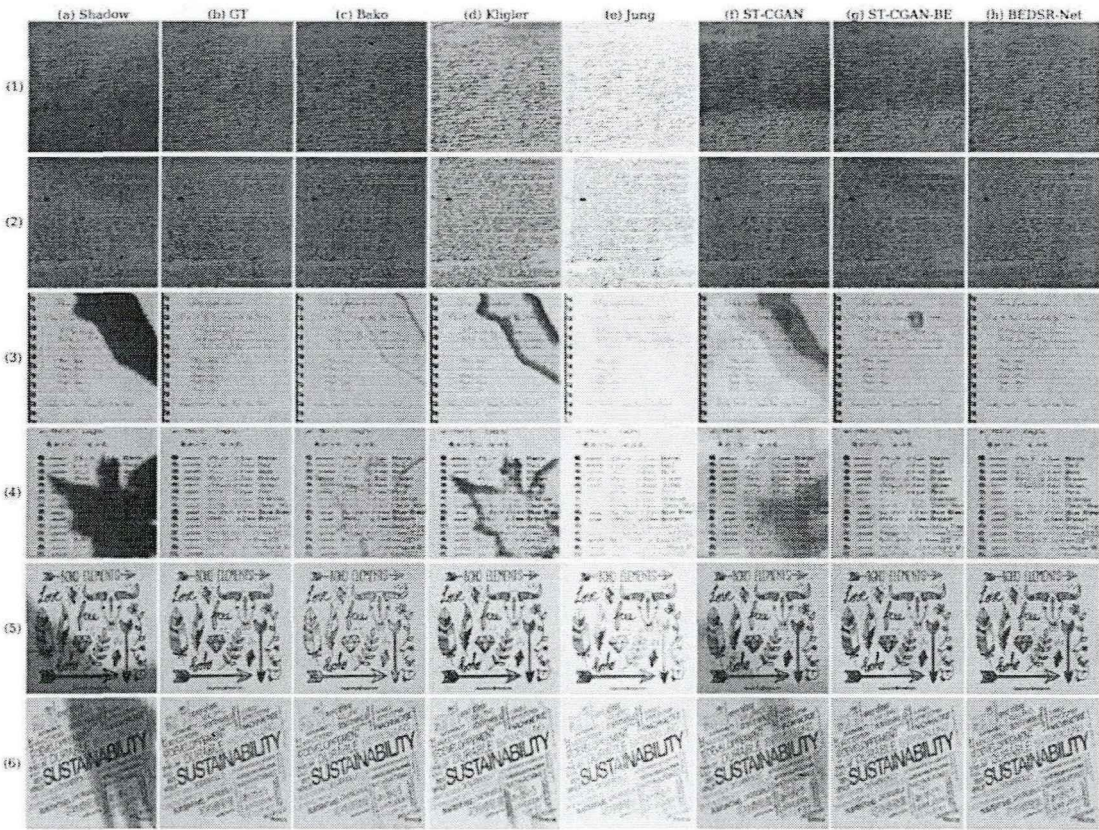


图 3-5 不同方法的视觉比较。

在使用不同方法去除阴影后，作者还比较了在恢复的图像上的 OCR 性能。首先，使用开源的 OCR 工具对真实无阴影图像和其他方法生成的图像进行文本识别得出结果。然后，通过使用 Levenshtein 距离^[34](也称为编辑距离)比较文本字符串来度量 OCR 性能。结果如下表 3-8 所示，距离越短代表性能越好，越接近对真实无阴影图像的识别结果。BEDSR-Net 的性能优于其他方法，表明它不仅提高了文档的视觉质量，而且提高了 OCR 的性能。

表 3-8 使用不同方法去除阴影后 OCR 的结果

method	input	Bako	Kligler	Jung	ST-CGAN	BEDSR-Net
distance	551.9	50.2	93.2	92.5	133.1	38.5

3.2.5.2 文字检测模型设计

基于深度学习的 OCR 技术主要分为两种，一种是图像先经过文字检测模型，检测出文字区域后，文字识别模型再对区域内的文字进行识别。还有一种是文字检测和识别放在一起的端到端模型。对于有固定格式的文本，如火车票等票据比较适合端到端模型。对于复杂场景下的文本图像（如文本行长短不一、文本行倾斜、文字语言不同），拍照得到的文档图像就属于这种，比较适合文字检测和文

本识别两阶段去做。因此，本节介绍的文字检测模型和下一节介绍的文字识别模型一起完成系统中图像文字识别功能。

PDF 文字提取模块中用到的文字检测算法自于 CVPR 2019 收录的学术论文 Character Region Awareness for Text Detection，现在对该算法思想进行简单介绍。

随着人工智能的发展，基于深度学习的文本检测技术表现出了很好的性能。这些方法主要是训练他们的模型来定位单词级别的边界框，但是在文本较长、文本变形等情况下，检测效果会受到影响。针对这种情况，NAVER 公司的学者提出了名为 CRAFT 的文本检测模型。该模型先根据区域得分定位图像中单个字符位置，然后根据关联得分对检测到的字符进行分组，把它们连接成一个个文本行达到文字检测的目的。该模型主要关注字符间的距离，因此对于文档类图像（字符间距离较为规律）效果较好。

CRAFT 采用了基于 VGG16 的全卷积网络体系结构，在模型解码部分 skip-connection，就像 U-net（一个语义分割网络）聚合网络底层特征一样。网络最终会输出两个分支得分：区域得分和关联得分，根据这两个得分，我们可以得到字符具体位置和字符间连接关系，最后将结果整合成边界框。该网络结构体系如图 3-6 所示。

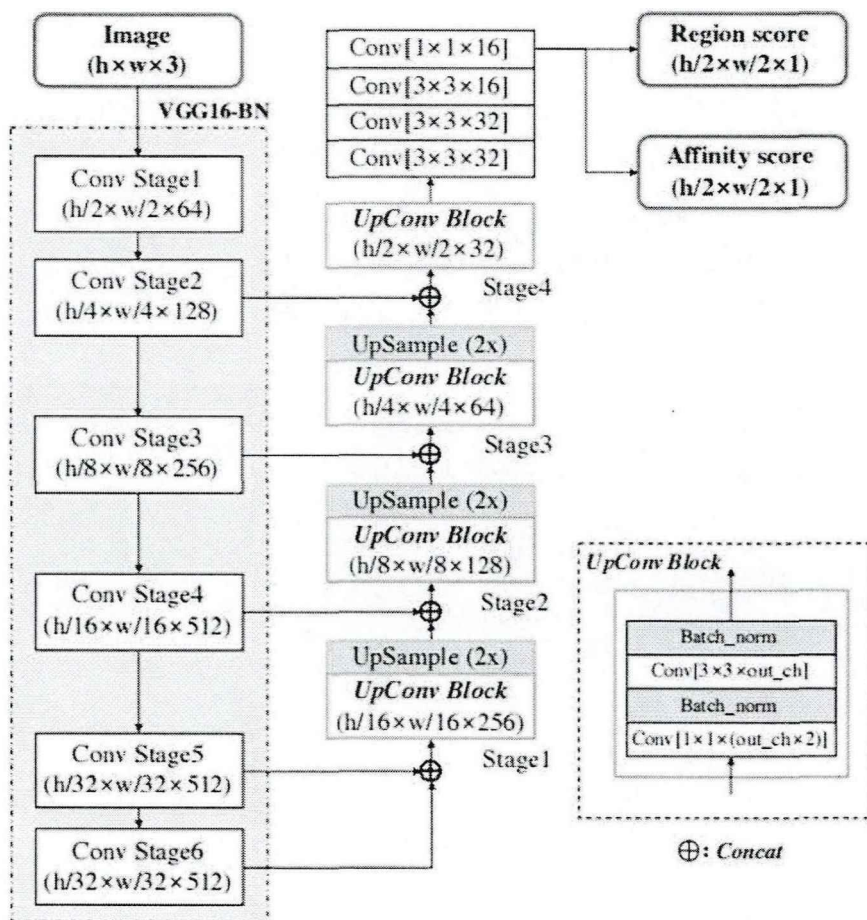


图 3-6 CRAFT 网络架构图

数据集中的真实图像通常是单词框标注，CRAFT 采用了弱监督学习方式解决了从单词框标注获得字符框标注的难题。下图 3-7 展示了字符分隔流程。首先，从输入图像中裁剪出单词级别的图像块。第二，从训练模型中得出区域分数。第三，使用 watershed 算法分割字符区域，获得字符框。最后，使用裁剪步骤中的反变换将字符框标注转换回原图上。

作者把训练数据中的单词级标注样本设为 ω ， $R(\omega)$ 表示样本 ω 的边界框区域， $l(\omega)$ 表示样本 ω 的单词长度。通过字符分割，模型可以得到字符框和字符长度 $l^c(\omega)$ 。并且随着训练的进行，CRAFT 模型可以更准确地预测字符位置，置信度得分 $S_{conf}(\omega)$ 也会逐渐的提高，置信度 $S_{conf}(\omega)$ 的计算公式为：

$$S_{conf}(\omega) = \frac{l(\omega) - \min(l(\omega), |l(\omega) - l^c(\omega)|)}{l(\omega)} \quad (3-7)$$

图像像素置信图计算公式为：

$$S_c(p) = \begin{cases} S_{conf}(\omega) & p \in R(\omega), \\ 1 & otherwise, \end{cases} \quad (3-8)$$

其中 p 表示 $R(\omega)$ 中的像素， $S_r(p)$ 表示预测的区域分数， $S_a(p)$ 表示预测的关联分数，当用合成数据集进行训练时，可以得到真实的标注，所以 $S_c(p)$ 设置为 1，目标 L 定义为：

$$L = \sum_p S_c(p) \cdot (\|S_r(p) - S_r^*(p)\|_2^2 + \|S_a(p) - S_a^*(p)\|_2^2) \quad (3-9)$$

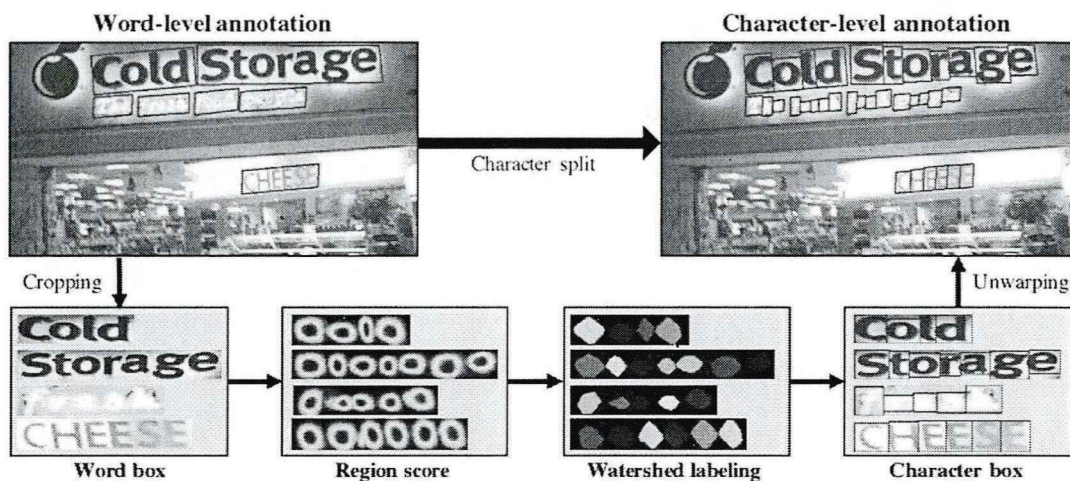


图 3-7 字符分割流程图

CRAFT 使用了字符感知的方法，利用很小的感受野就可以处理弯曲的、超长的文本，对于中英文等字符界限分明的语种效果很好，但是对于粘连字符（例如阿拉伯语和孟加拉语）的检测效果表现较差。CRAFT 模型无需微调就有很强的泛化能力，在大多数公开数据集上进行文本检测时达到了最先进的性能。

3.2.5.3 文字识别模型设计

文字识别就是将图像文本框（文字检测到的文本块）里的内容进行识别并提取的过程。现在主流方式是用 CNN 提取图像的卷积特征，之后使用 RNN 提取文字序列特征，此外，还需要加入语言模型来提高识别的准确性。这种模型虽然取得了良好的效果，但是采用了复杂的卷积网络作为骨干，所以不容易维护，而且需要一些后处理操作去保证识别的准确性。因此，这种模型仍然有很大的提升空间。

针对这种情况，微软亚洲研究员的科研人员进行了深入研究，在 2021 年提出了 TrOCR，它是第一个端到端的基于 Transformer 的 OCR 模型，利用预先训练过的 CV 和 NLP 模型进行文本识别。TrOCR 没有使用复杂的卷积神经网络作为模型的主干网络，只需一个简单的编码器-解码器模型，也不依赖任何复杂的前后处理操作，因此实现和维护都较为简单。

TrOCR 的模型结构如图 3-8 所示，它首先将输入文本图像的大小调整为 384×384 ，然后将图像分割成一个 16×16 块的序列，作为图像 transformer 的输入。在编码器和解码器部分都使用了标准 Transformer 架构和自注意力机制，编码器用于提取图像切片的特征，解码器生成的 wordpiece 序列作为输入图像的可识别文本。科研人员为了高效的训练 TrOCR 模型，用预训练 ViT 模式的模型初始化编码器，用预训练 BERT 模式的模型初始化解码器。

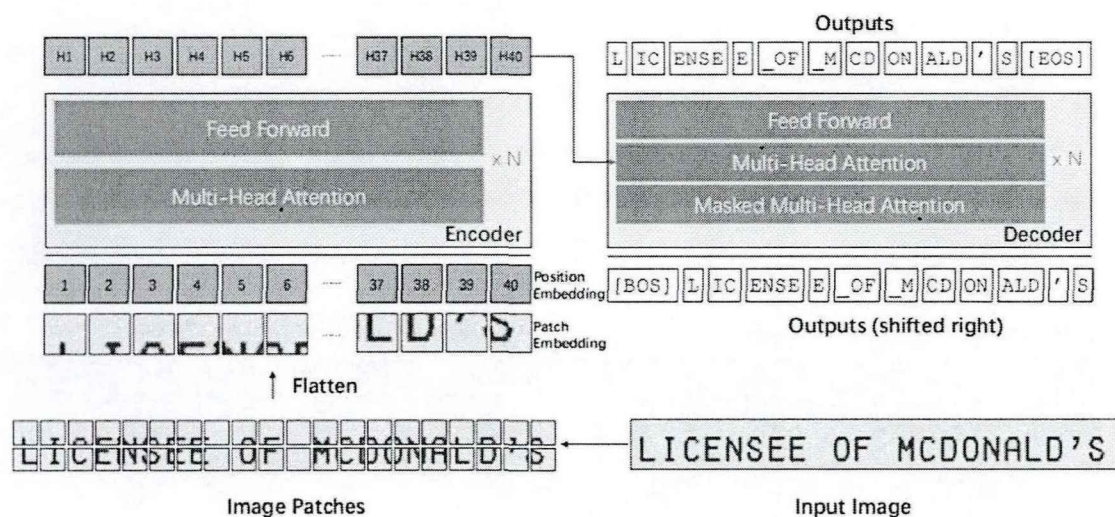


图 3-8 TrOCR 模型结构图

编码器不能处理原始图像，除非它们是一系列输入 tokens，所以编码器接受输入图像后，会将其调整为固定大小的正方形图像块。模型保留了通常用于图像分类任务的特殊标记“[CLS]”，它代表了整张图片的特征信息，同时，在使用 DeiT 预训练模型对编码器进行初始化时，也保留了输入序列中提取的 token，使模型

能够向教师模型学习。每个 Transformer 层都有一个多头自注意模块和一个全连接的前馈网络。这两部分之后是剩余连接和层归一化。

注意机制是在值上分配不同的注意力，并输出它们的加权和，其中值的权重由相应的键和查询计算。对于自注意模块，所有的查询、键和值都来自相同的序列。计算注意力输出矩阵公式为：

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad (3-10)$$

使用 $\frac{1}{\sqrt{d_k}}$ 的缩放因子是为了避免 softmax 函数的极小梯度，其中 d_k 是查询和键的维度。多头注意力是将 query、key、value 以不同的投影可学习权值进行 h 次投影，使得模型可以联合收集不同子空间的信息，公式为：

$$MultiHead(Q, K, V) = Concat(head_1, ..., head_h)W^O$$

$$where head_i = Attention(QW_i^Q, KW_i^K, VW_i^V) \quad (3-11)$$

TrOCR 模型使用了原始的 Transformer 解码器结构。标准的 Transformer 解码器也有一组相同的层，它们的结构与编码器中的层相似，只是解码器在多头自注意和前馈网络之间插入了“编码器-解码器注意”，以将不同的注意分配到编码器的输出上。在编码器-解码器注意模块中，键和值来自编码器输出，而查询来自解码器输入。此外，该解码器利用自注意中的注意掩蔽，防止在训练过程中获得比预测更多的信息。

研究员们制作数据集使用了数字 PDF 转图像 PDF，然后提取文本行和裁切图像的方法，和本系统图片 PDF 提取文本内容所需数据集高度类似。并且研究员在多个数据集上对比了 TrOCR 模型和其他文字识别模型的性能，实验结果表明，TrOCR 在打印文本（对应图片 PDF 文本）和手写文本识别上都达到了最先进的精度。因此，本系统使用了 TrOCR 模型作为文字识别的预训练模型。

3.2.5.4 表格识别模型设计

表格识别就是提取出图像里的表格内容，在该系统中，就是把图片 PDF 里的表格检测出来并转为 Excel 文档。因为表格大小、背景填充、框线、种类等形式复杂多样，所以表格识别技术一直是文档识别领域的重点和难点，包含了表结构检测、单元格坐标检测、单行文本检测和识别。

百度飞桨是国内领先的开源开放的深度学习平台，本系统使用了其开源的表格识别模型实现了表格提取功能。百度飞桨表格识别主要包括三个深度学习模型。首先，用于文本检测的 DBNet 模型^[35]，它是一种基于分割的文本检测方法，并

且简化了分割后的处理步骤，提升了检测的速度。第二，用于文字识别的 CRNN 模型，它也是目前主流的文字识别模型，采用了 CNN 提取图像的卷积特征，之后使用 LSTM 进一步提取文本序列特征。最后，用于表格结构和单元格坐标检测的 RARE 模型^[36]，它最初被用作对不规则文本区域先矫正再进行文本识别的算法，也可以作为 seq2seq 的表格结构预测。表格识别流程如下图 3-9 所示：

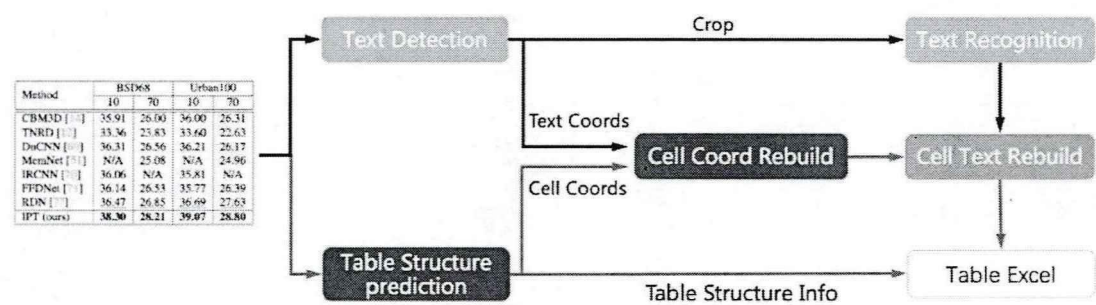


图 3-9 表格识别流程图

首先， DBNet 模型检测图像单行文本的坐标，之后把检测结果发给文字识别模型 CRNN 得到识别结果。第二， RARE 模型预测出表格结构和单元格坐标。第三，文本坐标、文本识别结果、单元格坐标共同组成单元格的识别结果。第四，表格结构和单元格识别结果共同组成了表格识别结果。最后，根据用户需求，系统把表格识别结果转化为 Excel 文档的单个工作表或者多个工作表。

3.3 本章小节

本章从社会应用价值和用户角度出发，对系统的功能需求、性能需求、界面需求都做了详细的分析和阐述。然后根据这些需求，对系统架构、功能模块、数据库表、前端界面、算法模型都进行了详细的设计。

第四章 系统具体实现

经过对系统的需求分析和详细设计后，本章结合前端框架 Vue、后端框架 Django 框架、关系型数据库 MySQL、非关系型数据库 Redis、相关算法模型对系统的各个功能模块进行具体实现。

4.1 文档文字图像数据集制作

4.1.1 问题描述

本课题中使用的 OCR 预训练模型大都是针对英文场景的，因此需要让这些模型在中文数据集上进行训练，提高文本检测和识别的准确率。ICDAR 等知名会议上都有大量的可用于训练文本检测、文本识别模型的公开数据集，但是这些数据集大部分都是应用于自然场景文本检测和识别、文档图像版面分析的，与本系统所需要的文档文本图像数据集有较大差异。

4.1.2 数据集合成

既然模型无法直接使用公开的数据集进行训练，我们就需要自己制作一个适用于本系统、大规模、高质量的文档文本图像数据集。生成数据集的操作步骤如下：

第一，获得文档数据。我们使用了爬虫技术，从互联网中下载了大约两万份公开的 PDF 文档，放在固定的文件夹下。文档中的字体包括了正常、倾斜、加粗以及加粗倾斜风格，也包含了宋体、仿宋、黑体、楷体等样式，部分样式如图 4-1 所示：



图 4-1 字体样式图，从左到右依次为宋体、仿宋、黑体、楷体

第二，获得文本检测数据集。由于这些 PDF 文档都是数字生成的，因此，我们可以使用 pdfplumber 解析库按页处理 PDF 文档，把每页的文本行坐标解析到文本文件作为标签（一张页面对应一个文本文件）。然后使用 fitz 和 PyMuPDF

解析库把 PDF 文档转为一张张页面图片放在固定文件夹下（图片名和文本文件名需要对应），作为文本检测模型的输入图像。提取坐标使用了 `extract_words()` 函数，它可以生成本文行的坐标属性，字段属性含义如下表 4-1 所示：

表 4-1 文本坐标属性表

属性	含义
x0	文本行左侧到文档页面左侧的距离
x1	文本行右侧到文档页面左侧的距离
top	文本行顶部到文档页面顶部的距离
doctop	文本行顶部到文档顶部的距离
bottom	文本行底部到文档页面顶部的距离

第三，获得文本识别数据集。我们使用 OpenCV 软件库的 `cv2.imread()` 函数批量读取第二步得到的页面图片，之后根据文本行坐标对读取的页面图片进行裁剪。裁剪坐标为 `[y0:y1, x0:x1]`，`y0` 和 `x0` 为起始坐标，`y1` 和 `x1` 为终点坐标。最后，我们使用 `pdfplumber` 解析库的 `extract_text()` 函数读取数字 PDF 的文件内容，根据换行符按行输出到文本文件，文本行内容需和裁剪后的相关图片内容对应。文本识别数据集的标签如下图 4-2 所示：

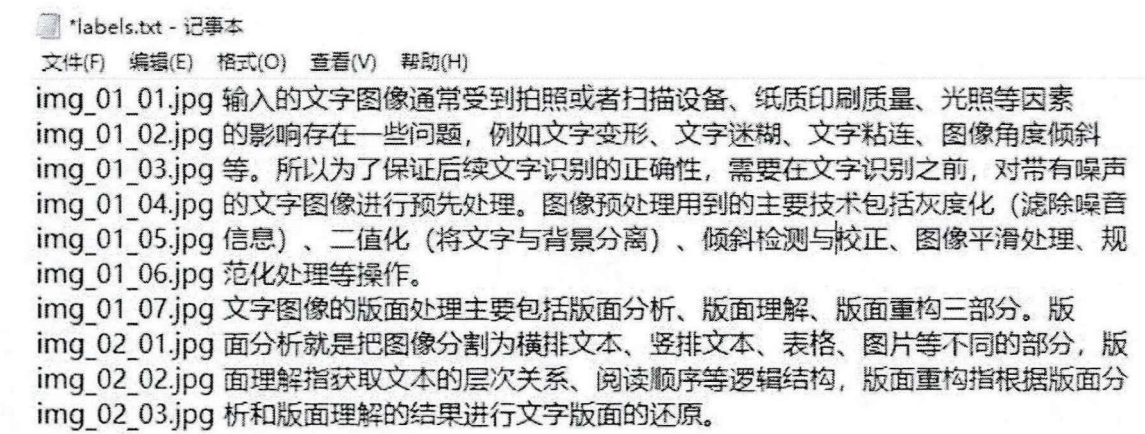


图 4-2 文本识别数据集标签

第四，添加中英文词典。由于 TrOCR 是一个针对英文文字识别的预训练模型，所以我们需要加入中英文词典，使文本转换为数字序列，让我们的系统可以同时识别中文和英文。

4.1.3 图像增强

印刷文字经常会出现笔画粘连、笔画断裂等情况。当我们用手机等设备对纸质文档拍照生成电子文档时，由于拍摄角度、光照强弱的影响，又会出现图片文字倾斜、图片文字模糊、图片背景亮度不均等情况。而且学习样本太少时，模型

在训练时很容易过拟合。为了应对这些问题,我们需要做一些图像增强的工作来提高我们模型的泛化能力。

图像增强是从现有的数据集中生成更多的训练数据,通常是利用一些能够生成可信图像的随机变换方式来增加数据样本^[37],让模型能够学习到数据的更多内容。本系统使用了 OpenCV 软件库来完成文本图像的增强任务。例如使用 `cv2.erode(x, kernel, iteration)` 函数对图像进行腐蚀来模拟笔画断裂,其中 `src` 表示输入图像, `kernel` 表示结构元素, `iteration` 表示腐蚀程度。使用 `cv2.dilate(src, kernel, iteration)` 函数对图像进行膨胀来模拟笔画粘连,其中 `src` 表示输入图像, `kernel` 表示结构元素, `iteration` 表示膨胀程度。使用 `cv2.warpAffine(src, M, dsize)` 函数对图像进行旋转来模拟文字倾斜,其中 `src` 表示输入图像, `M` 表示变换矩阵, `dsize` 表示输出图像的大小。此外,还对数据集进行了下采样、高斯模糊、高斯造点、随机雾化等操作。生成的部分图像效果如图 4-3 所示,图像增强不仅使我们的数据量扩大了将近两倍,图像的丰富度也得到了极大的提高。

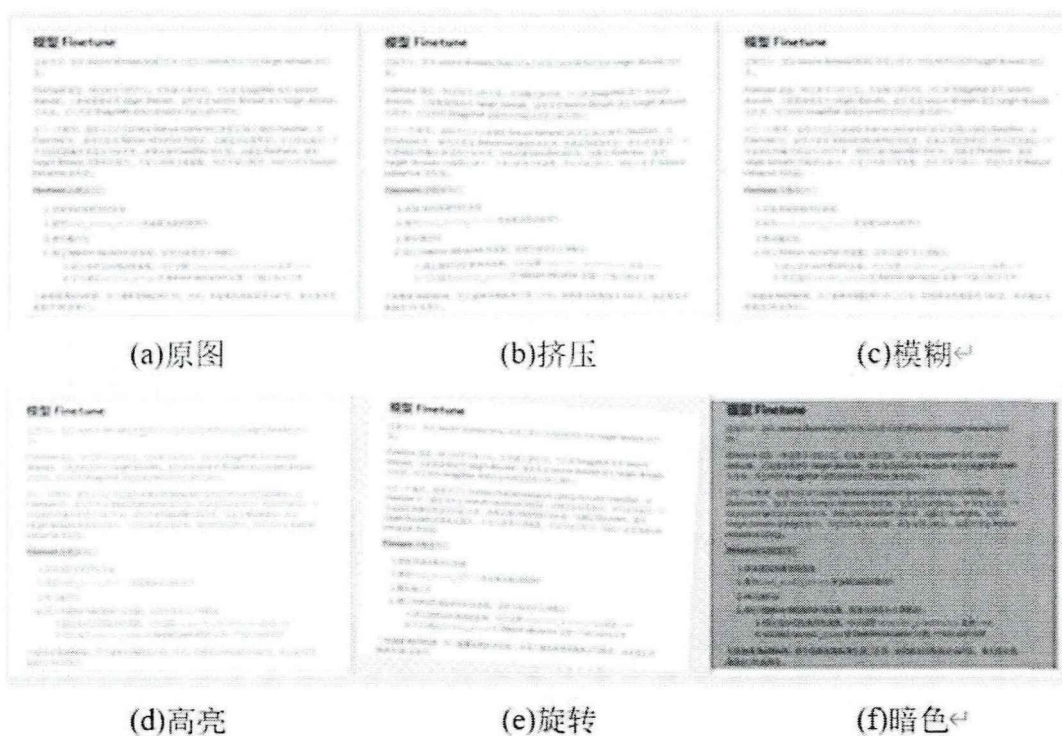


图 4-3 图像增强后的部分文档图

4.2 PDF 阴影去除模块实现

4.2.1 界面实现与展示

Vue 的核心思想强调的是从状态到界面的映射实例中,即 $UI=VM(State)$ 。当数据状态发生变化,页面 UI 也会相应发生变化,但是数据和页面视图并非直接

联系，而是通过 ViewModel 实现数据双向绑定，将数据更新映射到页面视图的动态变化。因此，本系统的前端界面组件皆采用 Vue.js 的 v-model 来绑定组件数据，使得前端界面中的表单/表格的数据和后台返回的更新数据实现双向的绑定。

用户登录系统后，前端通过 route 实现页面路由，跳转至系统首页（默认为 PDF 阴影去除界面）。界面展示如图 4-4 所示，界面左边为导航栏，采用了 ElementUI 的导航布局，当用户在导航栏选择不同的系统功能后，前端通过 route 实现路由，进入不同的功能模块界面，并且导航栏里相对应的功能框就会高亮。

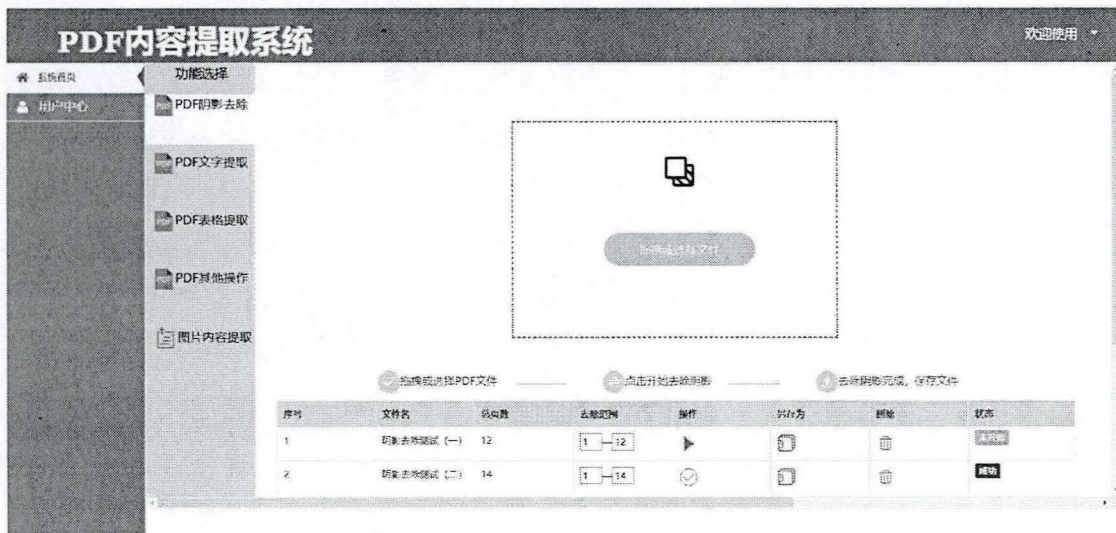


图 4-4 PDF 阴影去除模块界面图

界面中间为上传文件的区域，使用了 ElementUI 的 Upload 组件，用户可以拖拽文件到该区域或者点击区域内的选择文件框选择文件上传。

界面下方为上传的文件信息，在 created()渲染周期阶段，前端通过 API 请求后端获取 PDF 文件中的相关数据（编号、文件名、总页数等）。后端处理完成后将数据返回前端，前端 Vue 通过 v-model 将接收到的数据绑定到页面中的表格中。在去除范围表格框中，前端默认填写第一页到最后一页。如果用户只想去除某些页面的阴影，可以自行选择页面范围。例如，用户在去除范围框里输入 1,3,5 后，系统只会去除 PDF 文件里的第 1、3、5 页阴影。如果用户输入 1-5，则系统会处理 1、2、3、4、5 页阴影。无需处理的页面原样输出，生成的 PDF 文件页面顺序和输入的 PDF 页面顺序保持一致。

用户点击操作栏的三角形图标后，前端将文件流和属性参数通过 API 请求发送到后端。当后端处理完成时，将操作结果状态返回至前端，基于数据双向绑定，前端可以即时更新当前 PDF 文件的处理结果状态。状态栏由黄色的未开始图标变为绿色的成功图标时，用户点击另存为栏的保存按钮即可把处理后的 PDF 文件下载到指定的位置。

4.2.2 数据处理

阴影去除网络模型 BEDSR-Net 采用了 PyTorch 深度学习框架实现, 使用 albumentations 库对携带阴影的文档图像进行增强(上节是对文本检测和文本识别的训练数据进行增强), 最后用 matplotlib 绘图工具查看效果。

由于阴影去除网络模型要求输入数据为单张文档图像, 所以系统需要把用户上传的 PDF 文件拆分成多张图片放在固定的文件夹中供模型读取。程序中定义了一个 pdf_picture(pdfPath, picturePath)函数, pdfPath 为 PDF 文件的路径, picturePath 为生成图片的路径。该函数内调用了 fitz 解析库的 Matrix(zoom_x, zoom_y)函数, 其中 zoom_x 表示 x 轴方向的缩放系数, zoom_y 表示 y 轴方向的缩放系数。两个变量值一般取相同值, 取值越大, 我们的图像分辨率就会越高。PDF 文件和拆分后的图片如图 4-5 所示, 图片按照页码顺序从小到大生成到固定的文件夹。



图 4-5 PDF 文件拆分为多张图片

系统把 PDF 文件拆分成多张图片后, 模型根据用户的需求读取相应图片进行处理。值得注意的是, 生成的图片从 0 开始编号, 而用户输入的页码数是从 1 开始编号, 所以读取图片时输入数据应该做减 1 操作。

网络模型 BEDSR-Net 的输出为一张张图片, 所以我们需要把多张图片合并为一个 PDF 文件。合并操作我们使用了 Python 的 PIL 模块, 使用 Image.open() 函数打开第一张图片, 并以这张图片尺寸来创建单页的 PDF 文件, 之后根据 add_page()函数把剩下的图片按顺序插入 PDF 文件中。阴影去除模块的流程图如图 4-6 所示。

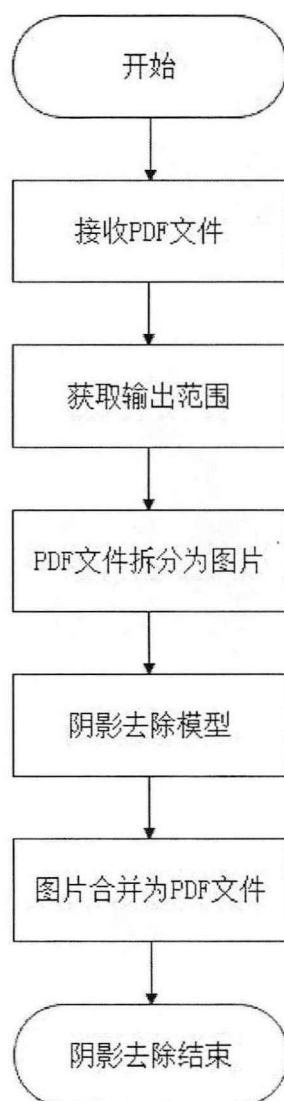


图 4-6 PDF 阴影去除模块流程图

从用户上传携带阴影的 PDF 文件开始,到用户下载无阴影的 PDF 文件结束,共涉及三个系统接口,分别为文件上传接口、阴影去除接口、文件下载接口。时序图如图 4-7 所示。

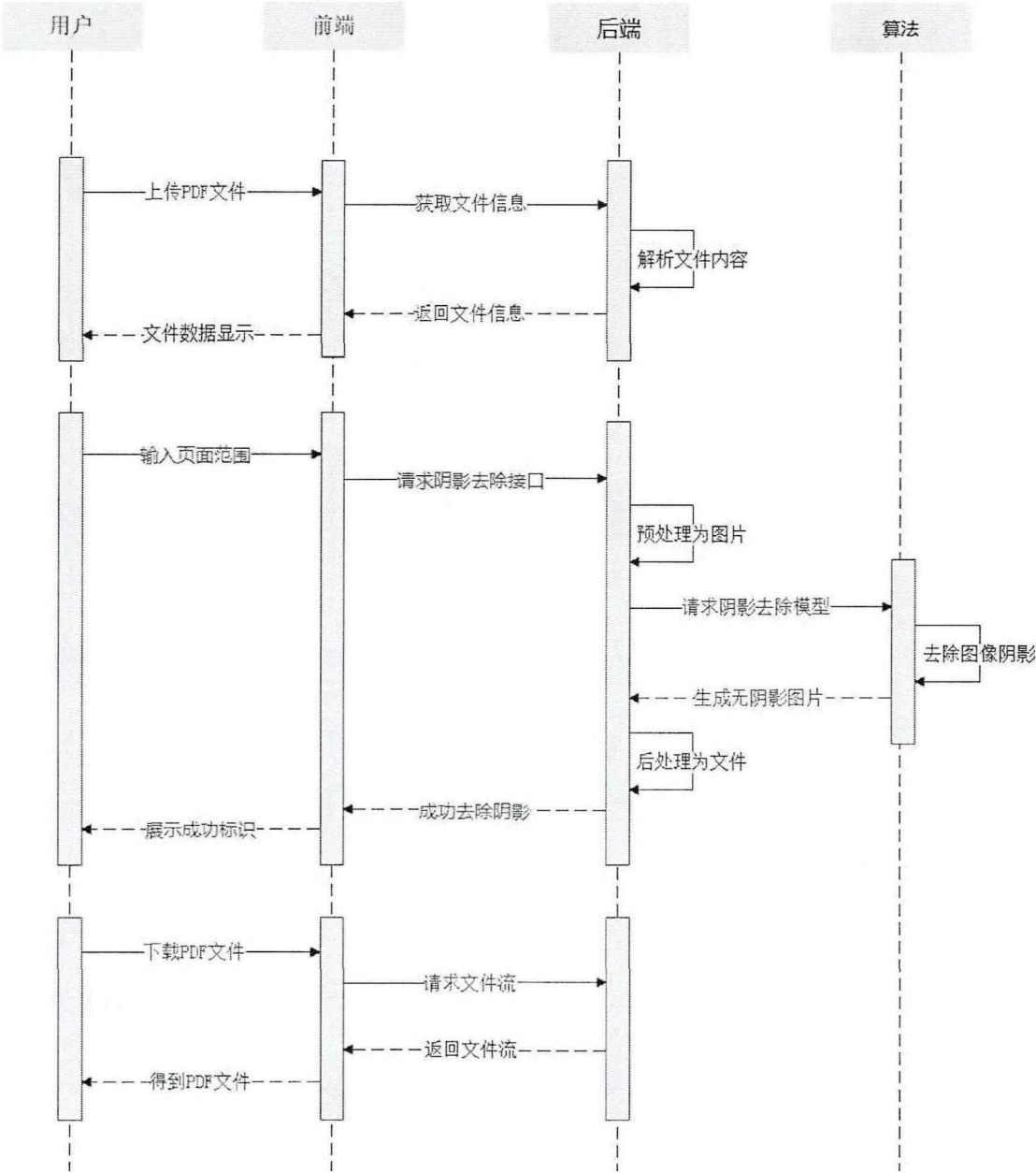


图 4-7 PDF 阴影去除时序图

4.2.3 去除结果展示

我们使用携带阴影的图片转换生成一个图片 PDF 文件，经过 PDF 阴影去除模块处理后，生成的 PDF 文件中某一页效果如图 4-8 所示。左边为拍摄后携带阴影的图像，右边是经过阴影去除后修复的清晰图。阴影去除后的图像没有出现以往方法中阴影残留、形状畸变、图像模糊的现象，较好地恢复了无阴影图像，提高了阅读质量。

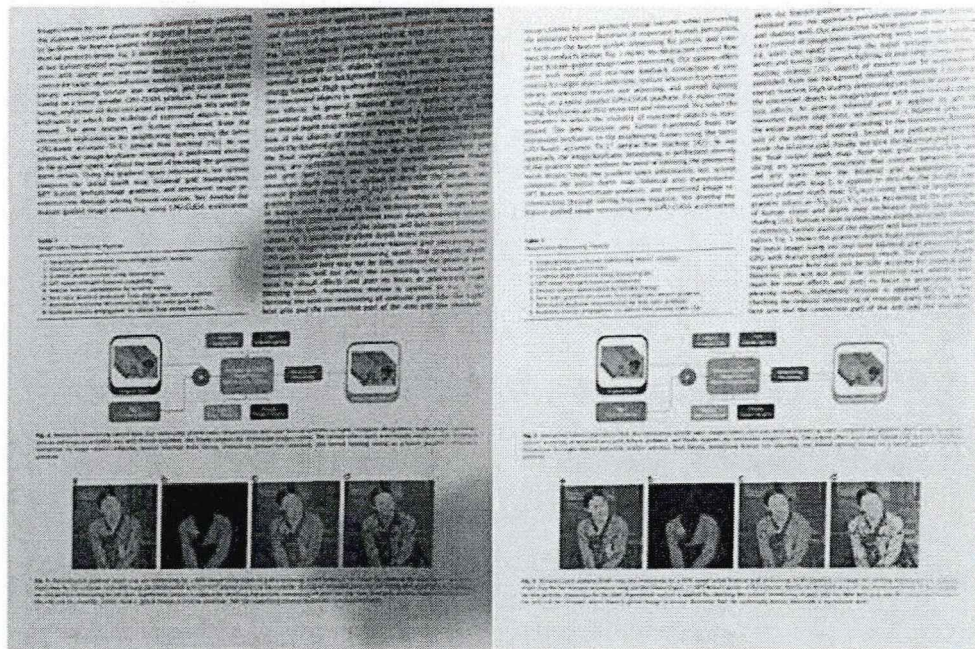


图 4-8 阴影去除结果

4.3 PDF 文字提取模块实现

4.3.1 界面实现与展示

PDF 文字提取界面在 PDF 阴影去除界面的基础上增加了一个下拉列表，由 ElementUI 中的 el-select 标签元素实现，如果想为下拉列表设置默认值，只需要把 v-model 绑定的值和想要选中 option 的 value 值设置一样即可。列表内容为 Word 文件、TXT 文件，在进行文字提取前需要选择生成哪种类型的文件。注重文字排版的用户可以选择 Word 文件，注重便捷性的用户可以选择 TXT 文件。界面如图 4-9 所示。

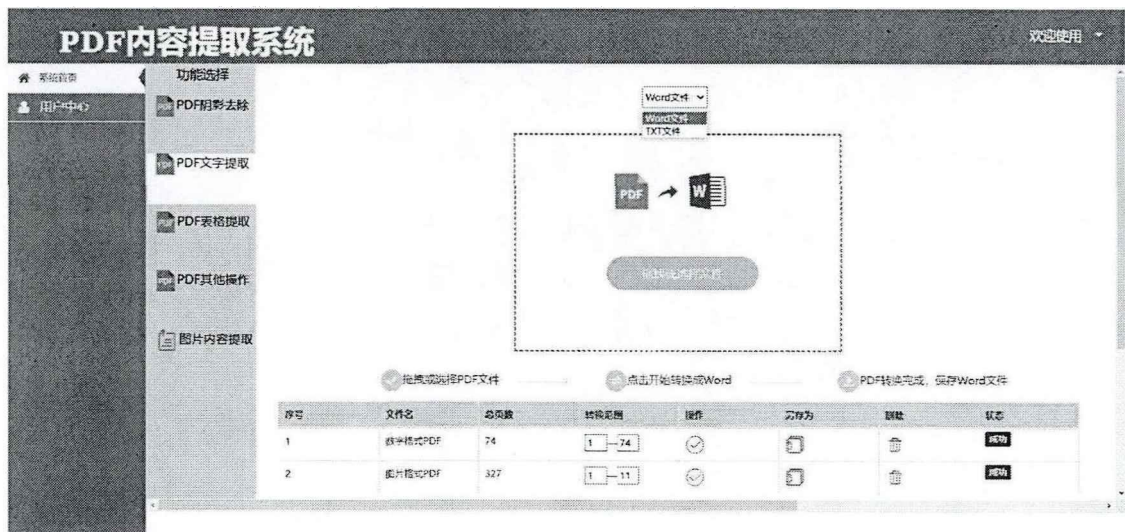


图 4-9 PDF 文字提取模块界面图

4.3.2 数据处理

从文字提取的准确性和效率出发，我们使用 pdfplumber 工具库对数字 PDF 进行文字提取，使用文字检测模型 CRAFT 加文字识别模型 TrOCR 对图片 PDF 进行文字提取。后端接口接收的参数信息如下表 4-2 所示。

表 4-2 请求参数表

参数名称	描述
file	用户上传的文件
page	用户需要提取的页码范围
type	用户需要生成的文件类型

我们的系统需要判断用户上传的 PDF 文件是数字 PDF 还是图片 PDF，由于 pdfplumber 工具库只对数字 PDF 有效，所以可以根据 extract_text()函数判断 PDF 的内容类型。

对于数字 PDF，pdfplumber 基于 PDF 中的各种对象信息进行文字提取。首先，使用 pdfplumber.open()函数打开文件并且返回 pdfplumber.PDF 类的实例。该类有一个 pages 属性，是一个内容为 pdfplumber.Page 实例的列表，每一个实例包含了数字 PDF 的每一页信息，我们对数字 PDF 的大部分操作都是围绕着 pdfplumber.Page 实例进行的。

我们这里使用了 extract_text()函数，对用户输入的页码内容进行文本行提取，函数主要通过 x_tolerance 和 y_tolerance 保留较好的提取格式。x_tolerance 含义为若其中一个字符右侧到页面左侧的距离与下一个字符右侧到页面左侧的距离之差大于 x_tolerance，则添加空格。若其中一个字符顶部到文档顶部的距离与下一个字符顶部到文档顶部的距离之差大于 y_tolerance，则添加换行符。pdfplumber 的 UML 类图如图 4-10 所示。

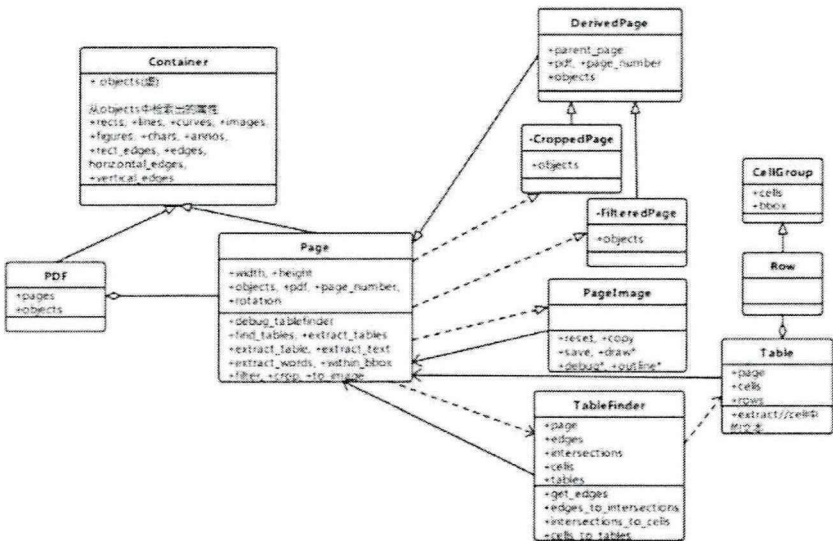


图 4-10 pdfplumber UML 类图

对于图片 PDF, pdfplumber 工具库是无法处理的。因此,我们需要使用 OCR 技术提取文本内容。首先,我们需要把 PDF 按页拆分为多张图片按序放在固定文件夹下。第二,调用文本检测模型 CRAFT,对输入图像进行预处理,使图像变为符合要求的格式。CRAFT 先基于置信度回归检测出字符级 bounding box 和字符之间的亲和力图 affinity score,之后亲和力图将相邻字符的 bounding box 连接起来,构成完整的文本行,最后通过二值化和联通分量分析得到每个文本行的 bounding box,后续文字识别,我们只需要文本行的 bounding box。第三,根据 bounding box 在原图上进行裁剪,作为文字识别模型的输入数据。第四,使用专注于文档文字识别的 TrOCR 模型改变输入图像的尺寸,把输入的图像切分为固定大小的图像切片。使用编码器提取图像切片的文字特性,使用解码器生成 wordpiece 序列,最后输出文字识别结果,输入图像和接口输出如图 4-11、4-12 所示。

你已经见过一个 Keras 模型的示例,就是 MNIST 的例子。典型的 Keras 工作流程就和那个例子类似。

- (1) 定义训练数据:输入张量和目标张量。
- (2) 定义层组成的网络(或模型),将输入映射到目标。
- (3) 配置学习过程:选择损失函数、优化器和需要监控的指标。
- (4) 调用模型的 fit 方法在训练数据上进行迭代。

图 4-11 输入的文档图片

```
predict_string: 你已经见过一个Keras模型的示例,就是MNIST的例子。典型的Keras工作流程就和那个
predict_string: 例子类似。
predict_string: (1)定义训练数据:输入张量和目标张量。
predict_string: (2)定义层组成的网络(或模型),将输入映射到目标。
predict_string: (3)配置学习过程:选择损失函数、优化器和需要监控的指标。
predict_string: (4)调用模型的fit方法在训练数据上进行迭代。
```

图 4-12 文字识别接口返回结果

无论是数字 PDF 还是图片 PDF,最终都需要根据文字识别结果生成一个 Word 文件或者 TXT 文件供用户下载。对于 TXT 文件,只需要把文字识别结果按行输入到文件中。对于 Word 文件,需要考虑文本排版问题,策略就是按照文本行坐标定位文字位置,并且从上到下,从左到右把文本行排序组合起来,PDF 文字提取模块流程图如图 4-13 所示。

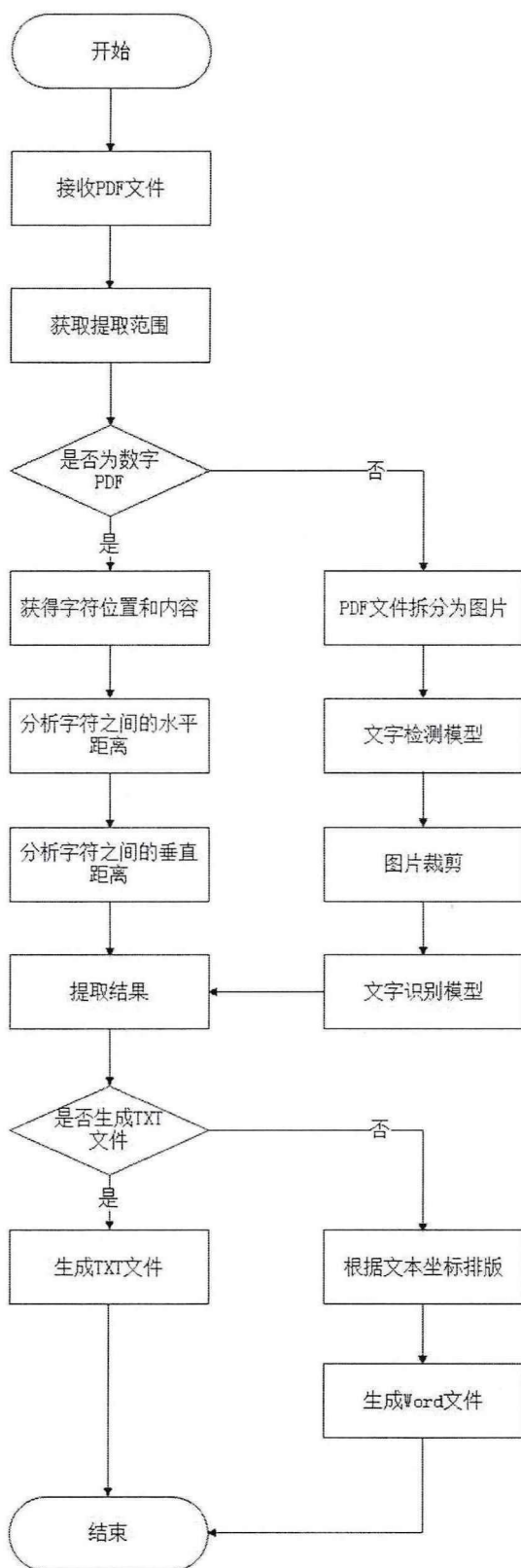


图 4-13 PDF 文字提取模块流程图

对图片 PDF 文件的文字提取时序图如图 4-14 所示，后端需要调用两次算法模型。而对数字 PDF 文件的文字提取不需要调用算法模型，只需要在后端层处理即可。

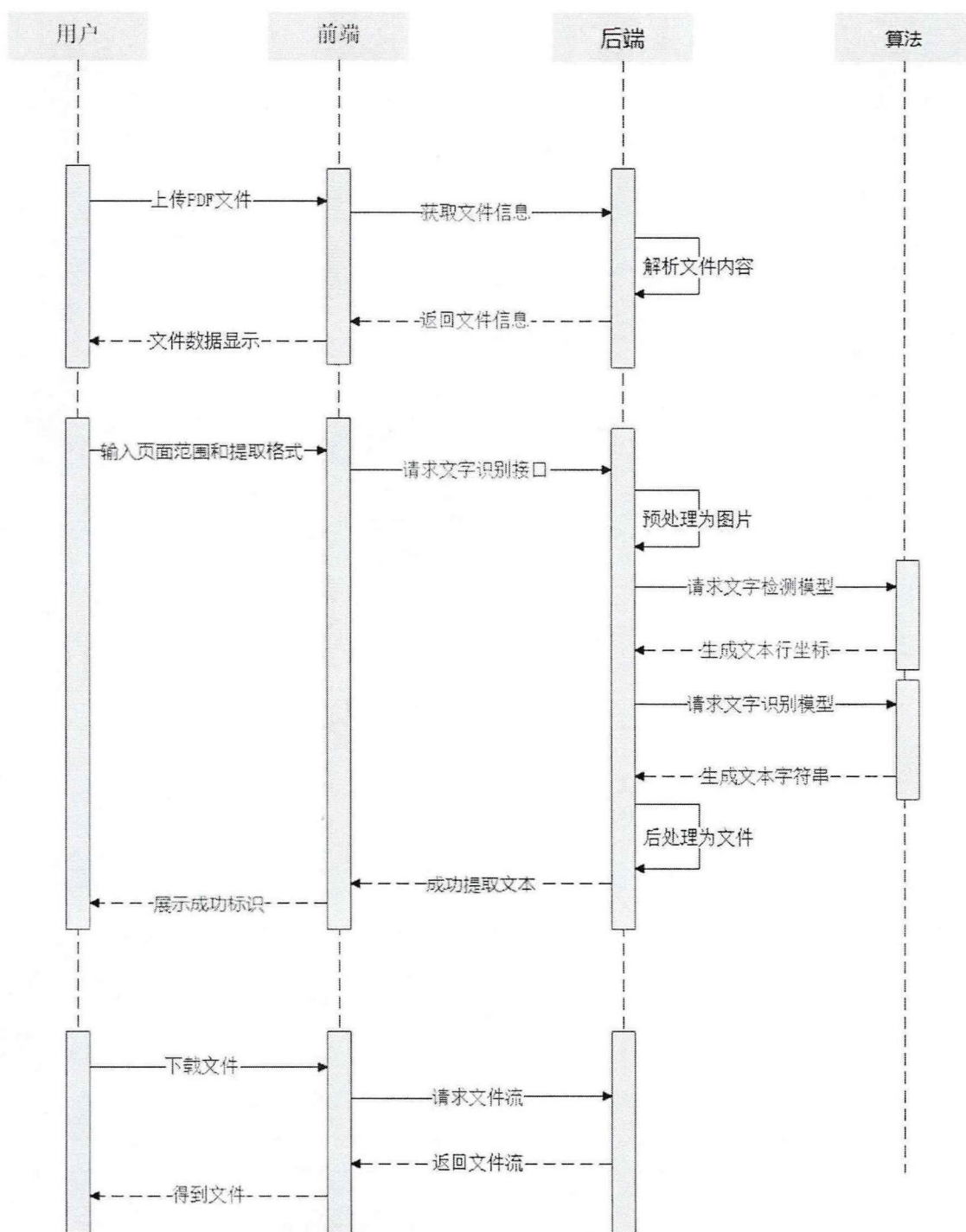
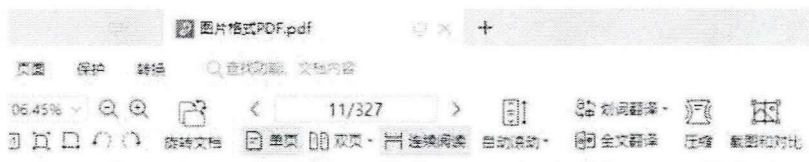


图 4-14 图片 PDF 文字提取时序图

4.3.3 提取结果展示

完成上面的数据处理后，就实现了对数字 PDF 文件或图片 PDF 文件文字提取的工作。对于数字 PDF 文件，处理过程中只会提取所需要的文本信息，图像表格等信息会被过滤掉。对于图片 PDF 文件，处理过程中会提取图中的所有信息。我们选取了一篇数字 PDF 文件，经过第三方软件转换生成图片 PDF 文件，

之后使用我们的系统对该文件中的文字内容进行提取，需要提取的页面内容如图 4-15 所示。



全球化智库 (CCG) 简介

全球化智库 (Center for China and Globalization), 简称 CCG, 成立于2008年, 总部位于北京, 在国内外有近十余个分支机构和海外代表处, 目前拥有全职智库研究和专业人员近百人。秉承“以全球视野, 为中国建言; 以中国智慧, 为全球献策”宗旨, CCG致力于全球化、全球治理、国际关系、人才国际化和企业国际化等领域的研究, 是中国最大的社会智库之一, 也是中国领先的国际化智库。CCG是中央人才工作协调小组全国人才理论研究基地, 中联部“一带一路”智库联盟理事单位, 财政部“美国研究智库联盟”创始理事单位, 并被国家授予博士后科研工作站资质。CCG也是唯一被联合国授予“特别咨商地位”的中国智库。

CCG成立十余年来, 已发展为中国推动全球化的重要智库。在世界最具权威性的全球智库评价报告美国宾夕法尼亚大学《全球智库报告2020》中, CCG位列全球顶级智库百强榜单第六十四位, 连续四年跻身世界百强榜单, 也是首个进入世界百强的中国社会智库, 并在国内外多个权威智库排行榜单评选中居中国社会智库首位。

图 4-15 图片 PDF 文件的页面内容
提取后生成的 TXT 文件内容如图 4-16 所示。

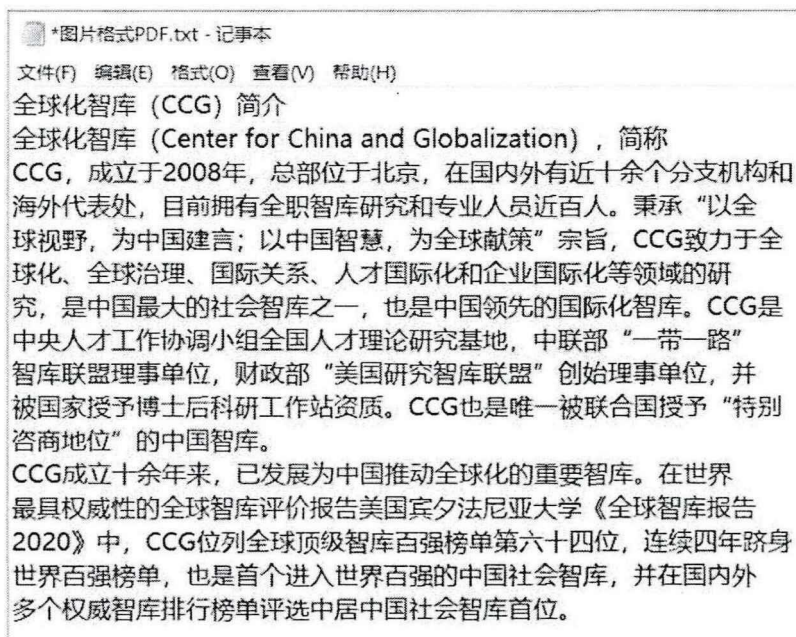


图 4-16 TXT 文件内容

提取后生成的 Word 文件内容如图 4-17 所示。

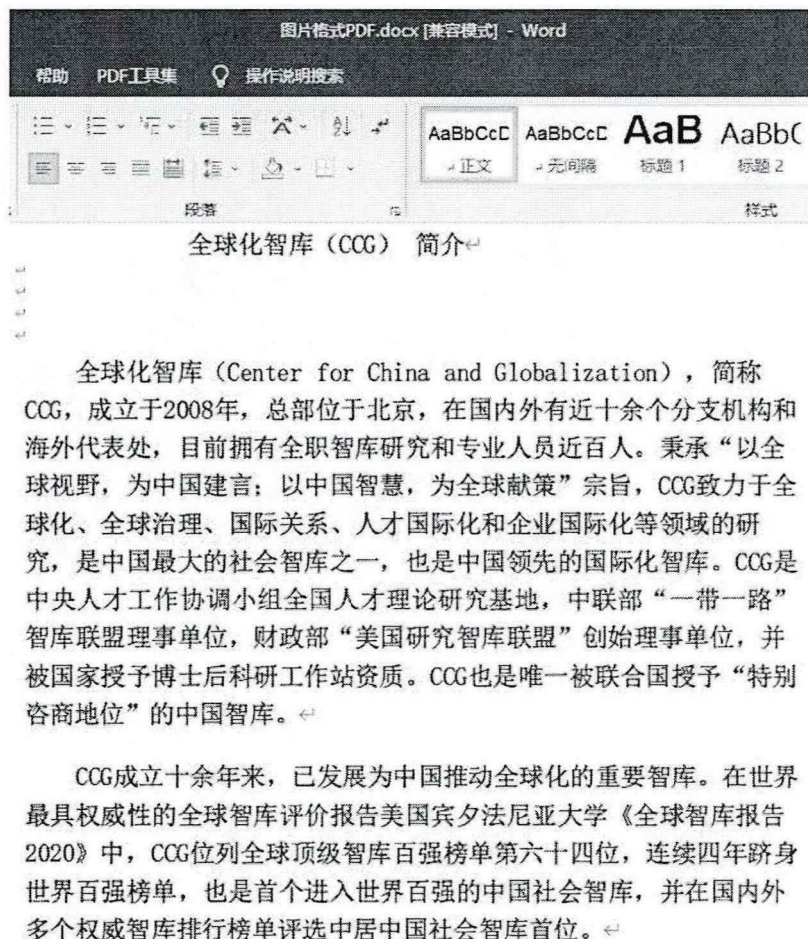


图 4-17 Word 文件内容

4.4 PDF 表格提取模块实现

4.4.1 界面实现与展示

PDF 表格提取模块界面实现与 PDF 文字提取模块基本一致，对 PDF 文件中的表格提取时，表格以外的文字内容会被程序忽略。当我们把 PDF 文件中的表格提取到 Excel 文件时，如果 PDF 文件中有多多个表格，可以把这些表格都提取到一个工作表，也可以提取到多个工作表。用户在点击开始按钮前，需要先在下拉列表中选择单个工作表或者多个工作表，然后系统才能处理用户上传的 PDF 文件。界面如图 4-18 所示。

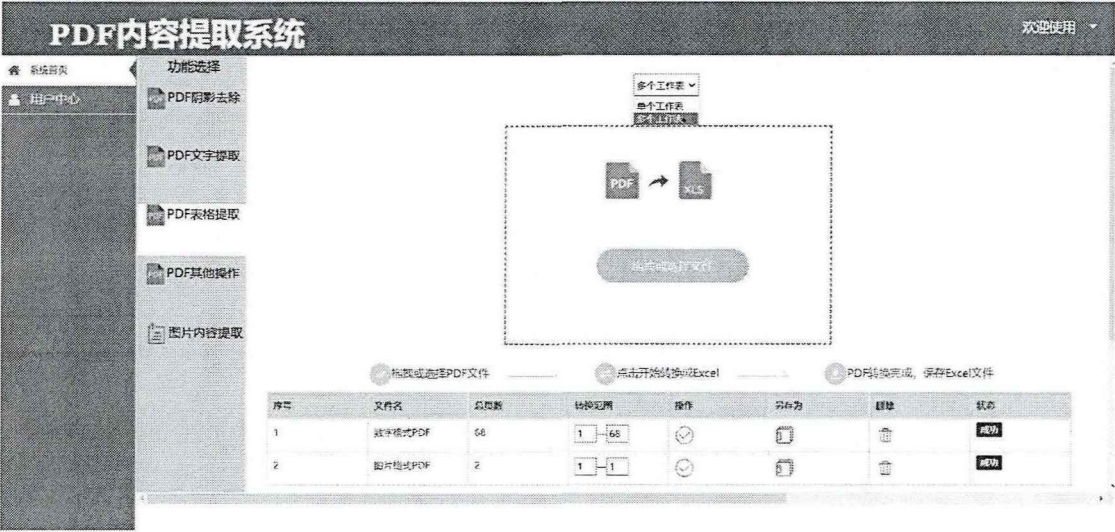


图 4-18 PDF 表格提取模块界面图

4.4.2 数据处理

我们依然使用 pdfplumber 工具库分析并操作数字 PDF，把目标页中的表格提取出来。对于图片 PDF，我们使用DB 模型进行单行文本检测、CRNN 模型进行单行文本识别、RARE 模型进行表结构和单元坐标预测。后端接口接收的参数信息如下表 4-2 所示。

表 4-2 请求参数表

参数名称	描述
file	用户上传的文件
page	用户需要提取的页码范围
sheetNum	用户需要生成工作表数量

pdfplumber 工具库对数字 PDF 里表格的提取，主要由 TableFinder 这个类来完成，在这个类的构造函数中详细的定义了表格识别和提取的流程。要想识别出表格，首先需要找出表格的线框。对于线框比较完整的表格，表格的各个单元格都是用矩形线框分割出来的，解析这种类型的表格比较简单。当我们使用 pdfplumber.open()函数打开数字 PDF 后，会把每一页的解析结果保存在 Page 类中，这个类包含了 rects、chars、edges 等基本对象的属性。TableFinder 类中的 get_edges()函数根据方向、长度等条件，对 Page 对象中的 edges 对象进行过滤，找到可能的表格线框。

对于那些线框不完整的表格，是目前表格识别领域的难题。pdfplumber 通过 words_to_edges_v()函数和 words_to_edges_h()函数，解析出字符串的对齐情况，根据字符串顶部位置进行聚类，筛选出水平方向和竖直方向存在的线，并进一步预测出单元格的边界框，实现表格提取的逻辑。如果表格提取的效果不理想，可

以在使用 `extract_tables()` 函数抽取表格信息时, 指定一些水平方向或者竖直方向的线, 以提升表格识别的效果。

在经过上面的步骤之后, 我们已经找到了表格线和交点, 接下来需要通过 `intersections_to_cells()` 函数找到可能存在的单元格。第一步就是根据交点坐标按照从上到下、从左到右的顺序进行排序。第二步, 以每个交点出发, 找到这个交点作为左上角顶点的最小单元格。最后, `cells_to_tables()` 函数把前面找到的单元格合并到一起生成表格, 把表格和表格里的文本以行的形式输出。图 4-19 为数字 PDF 某页内的表格, 图 4-20 为接口输出结果。

Kernel Methods	RBF		Linear	
Method	WSS-WR	WSS 3	WSS-WR	WSS 3
MPG	6.9927	6.4602	12.325	12.058
MG	0.01533	0.014618	0.02161	0.02138
Kernel Methods	Polynomial		Sigmoid	
Method	WSS-WR	WSS 3	WSS-WR	WSS 3
MPG	0.25	0.5	14.669	14.22
MG	0.019672	0.018778	0.023228	0.023548

图 4-19 数字 PDF 中的表格

```
[ 'Kernel Methods', 'RBF', None, 'Linear', None ]
[ 'Method', 'WSS-WR', 'WSS 3', 'WSS-WR', 'WSS 3' ]
[ 'MPG', '6.9927', '6.4602', '12.325', '12.058' ]
[ 'MG', '0.01533', '0.014618', '0.02161', '0.02138' ]
[ 'Kernel Methods', 'Polynomial', None, 'Sigmoid', None ]
[ 'Method', 'WSS-WR', 'WSS 3', 'WSS-WR', 'WSS 3' ]
[ 'MPG', '0.25', '0.5', '14.669', '14.22' ]
[ 'MG', '0.019672', '0.018778', '0.023228', '0.023548' ]
```

图 4-20 接口输出结果

对于图片 PDF, 我们依然需要将 PDF 拆分为多张图片作为模型的输入数据。首先, 我们使用 DB 模型读取原始图像, 预处理为符合要求的格式, 检测出单行文本的坐标。第二, 根据 DBNet 模型输出的 bounding box 对原图进行裁剪, 作为文字识别模型的输入数据。第三, 将裁剪后的图像发送给 CRNN 模型, 识别图像中的文字。第四, 表结构预测模型 RARE 读取原始图像, 根据单元格坐标和文本坐标对单元格坐标进行重建。最后, 根据重建后的单元格坐标、文本坐标、文本内容得到表格信息。

无论是数字 PDF 还是图片 PDF, 最终都需要根据表格提取结果生成一个 Excel 文件, 可以把所有表格都放在同一个工作表, 也可以一个表格放在一个工作表。表格提取模块流程图如图 4-21 所示。

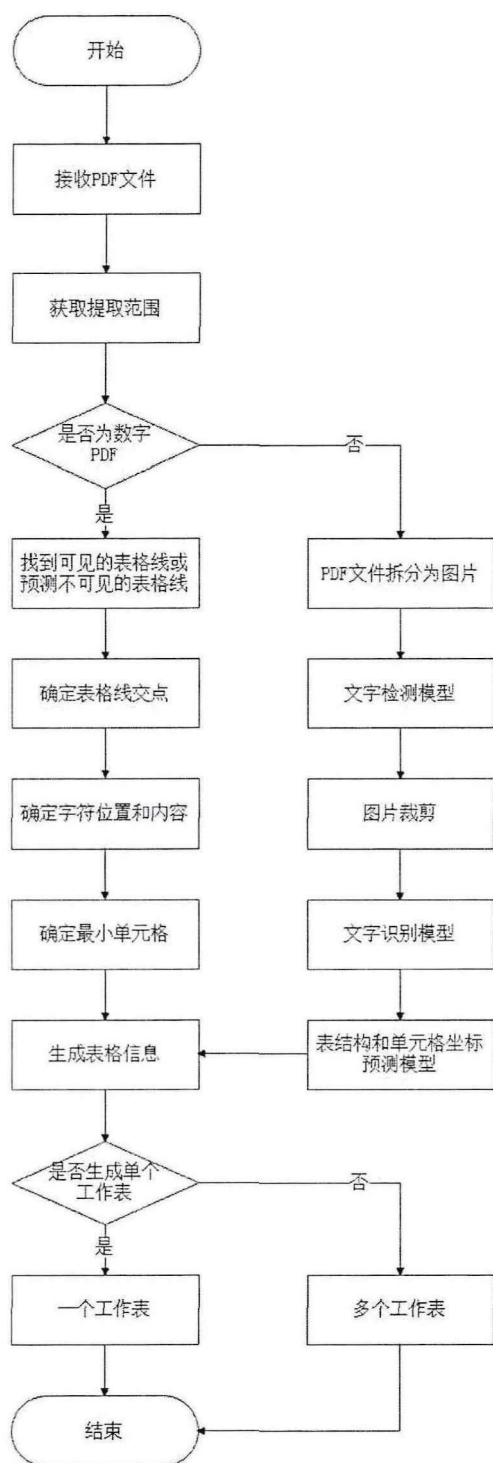


图 4-21 PDF 表格提取模块流程图

对图片 PDF 文件的表格提取时序图如图 4-22 所示，后端需要调用三次算法模型。而对数字 PDF 文件的表格提取不需要调用算法模型，只需要在后端层处理即可。

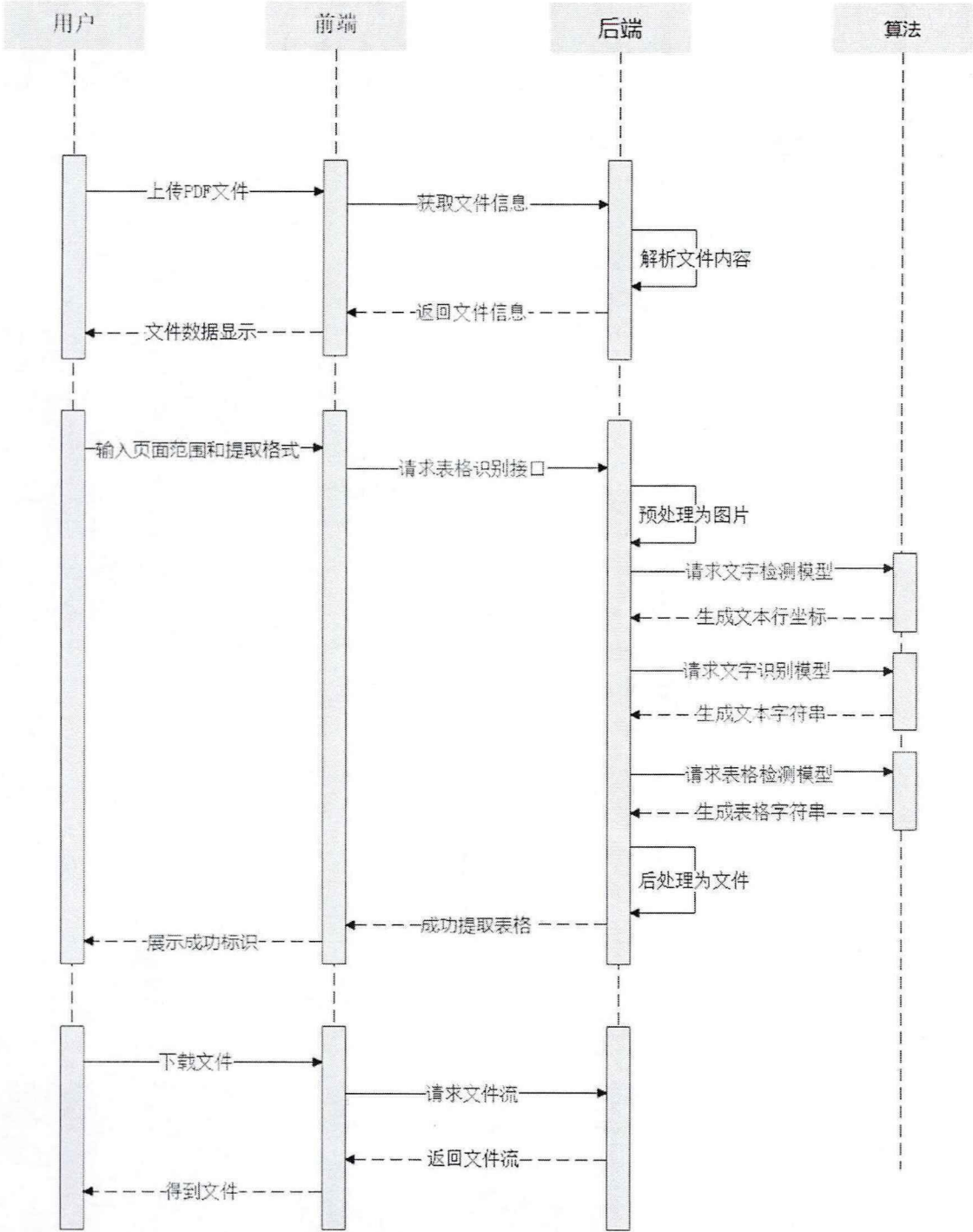


图 4-22 图片 PDF 表格提取时序图

4.4.3 提取结果展示

完成上面的数据处理后，就基本实现了将数字 PDF 或图片 PDF 里的表格提取到 Excel 文件的工作。提取过程中只会提取所需要的表格信息，表格以外的文字等信息会被过滤掉。我们选取了一篇含有多个表格的数字 PDF，然后经过第三方软件转换生成图片 PDF 进行实践，图片 PDF 内的表格信息如图 4-23 所示。

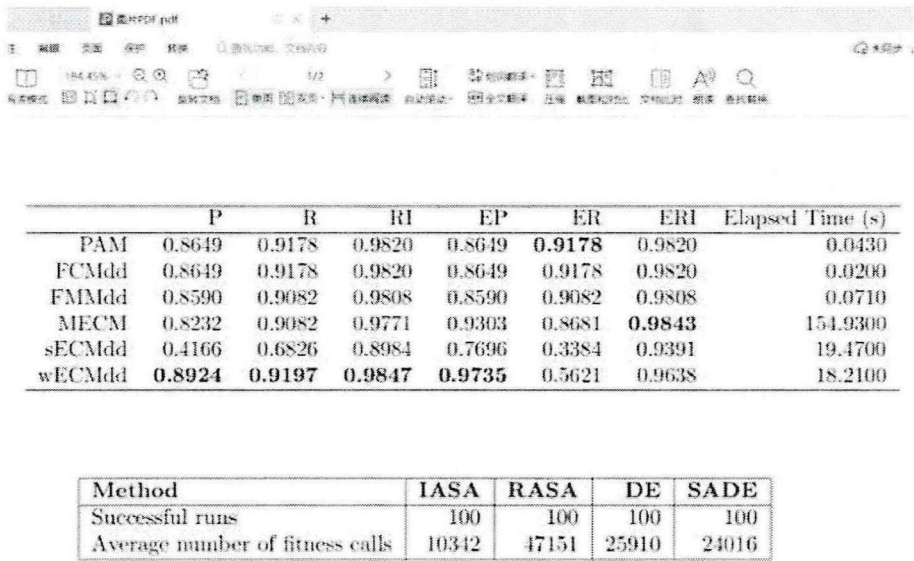


图 4-23 图片 PDF 文件的表格信息

我们只提取图片 PDF 文件第一页的两个表格，当提取到单个工作表时，结果如图 4-24 所示。

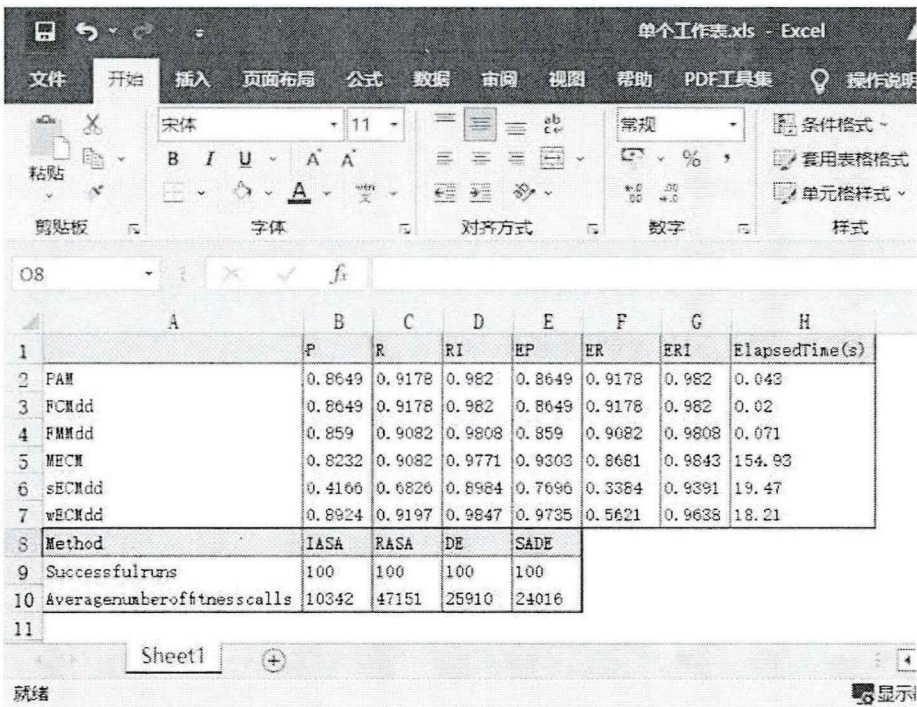


图 4-24 提取到单张工作表结果

当我们提取到多个工作表时，结果会生成两个工作表，我们只展示第一张工作表，如图 4-25 所示。

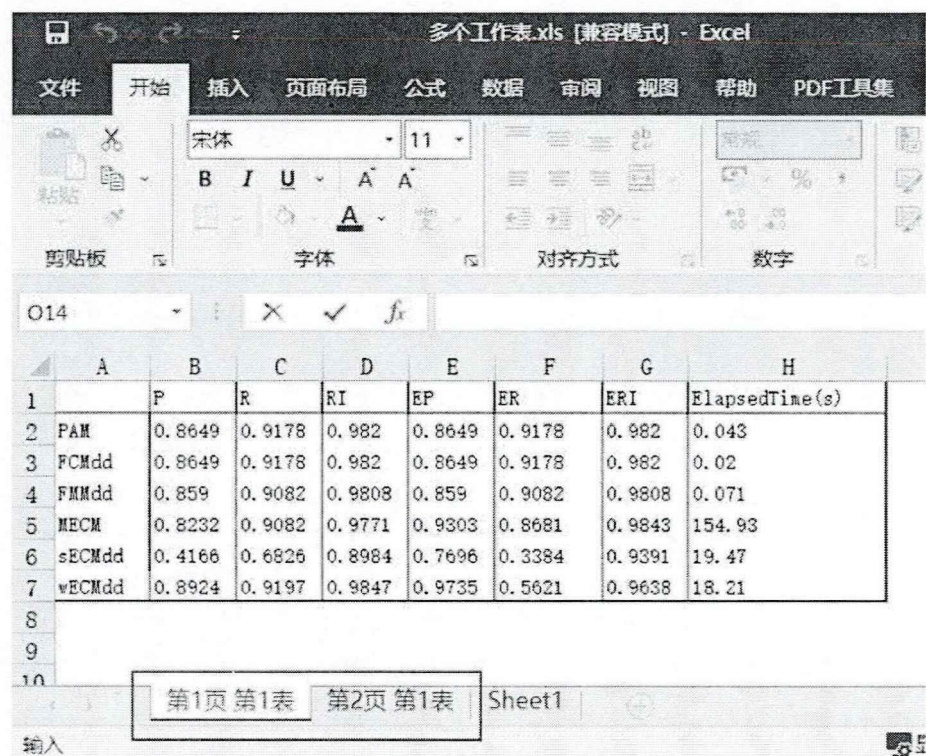


图 4-25 提取到多张工作表结果

4.5 PDF 其他操作模块实现

从用户需求的角度出发，我们的系统除了实现以上那些重要功能外，还增加了一个 PDF 其他操作模块，为用户提供 PDF 合并、PDF 拆分、PDF 旋转、PDF 加水印这些热门功能，结构如图 4-26 所示。

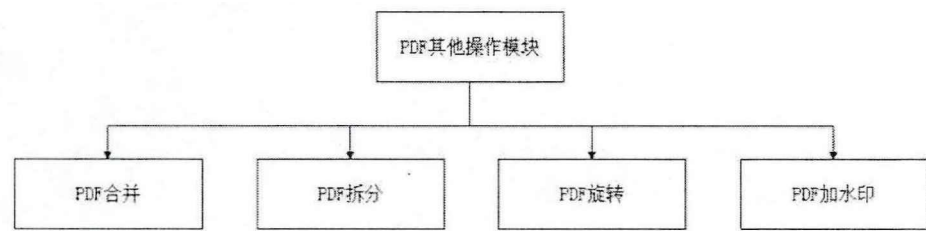


图 4-26 PDF 其他操作模块结构图

4.5.1 PDF 合并

在日常工作中，为了方便文档查看和管理，我们经常遇到将小组内每个人的 PDF 合并到一个 PDF 这种需求，这时可以使用我们系统的 PDF 合并功能，功能界面如图 4-27 所示。

用户点击添加文件按钮即可在本地选择需要合并的 PDF 文件，用户上传文件后，界面会显示已上传文件的名称和大小。为了帮助用户按某种顺序合并 PDF 文件，系统提供了文件上移和下移按钮，输入合并后的 PDF 文件名称后，点击

开始合并按钮（至少需要上传两个 PDF 文件才能点击该按钮），系统会按照用户选择的顺序从上到下合并 PDF 文件。



图 4-27 PDF 合并功能界面图

我们把用户上传的 PDF 文件进行编号，当用户点击开始合并按钮后，前端会把编号顺序发送到后端，由后端生成合并后的 PDF 文件。具体工作流程如下：

1. 后端根据功能、用户名、时间信息生成文件夹，并把用户需要合并的 PDF 文件存储在该文件夹下。
2. 后端按照编号顺序循环遍历这些 PDF 文件，根据文件路径调用 PdfFileReader()函数读取文件，生成 PdfFileReader 对象。
3. 根据 PdfFileReader 对象的 getNumPages()函数计算 PDF 文件的页数，getPage()函数可以得到指定的页面，我们从第一页开始遍历。
4. 调用 PdfFileWriter()函数得到 PdfFileWriter 对象，然后调用该对象的 addPage()函数添加 PDF 文件的每一页。
5. 最后根据用户输入的文件名，生成一个新的 PDF 文件，然后使用 PdfFileWriter 对象的 writer()函数把数据写入到新生成的 PDF 文件。

4.5.2 PDF 拆分

有时候某个 PDF 文件里的内容比较多，而我们只需要其中的一部分，就需要把这个 PDF 文件按页码拆分为多个 PDF 文件。这时可以使用我们系统的 PDF 拆分功能，功能界面如图 4-28 所示。

用户点击添加文件按钮即可在本地选择需要拆分的 PDF 文件，用户上传文件后，界面会显示已上传文件的名称和大小。PDF 拆分功能界面提供了两种拆分方式，每一页生成一个 PDF 文件和按照区间拆分（由用户输入需要拆分的范围，

中间用逗号隔开)。点击开始拆分按钮后,系统会按照用户选择的方式拆分 PDF 文件。

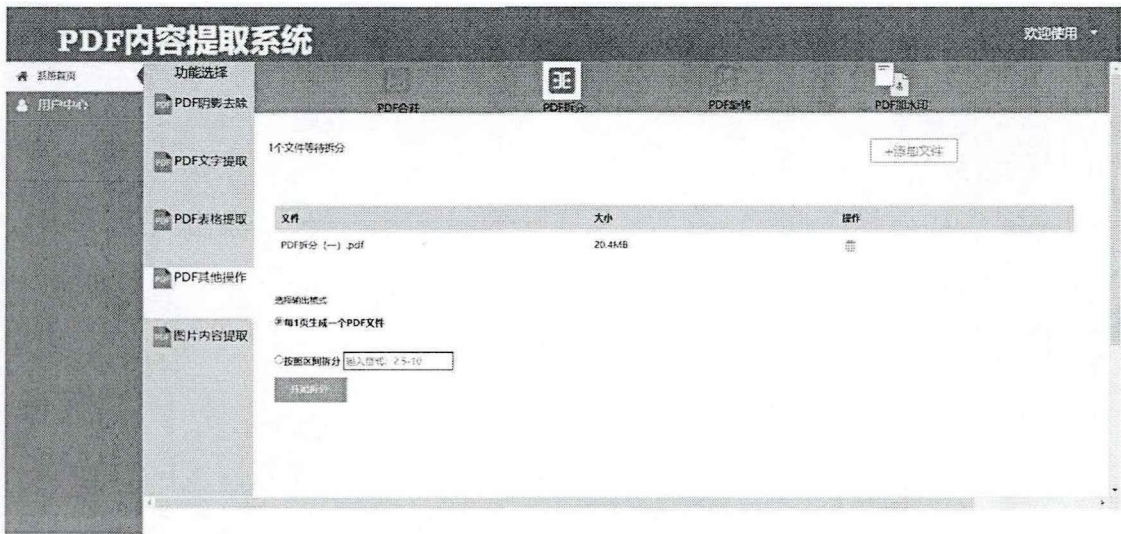


图 4-28 PDF 拆分功能界面图

当用户点击开始拆分按钮后,前端会把用户选择的拆分方式发送到后端,由后端生成拆分后的 PDF 文件,具体工作流程如下:

1. 后端根据功能、用户名、时间信息生成文件夹,并把用户需要拆分的 PDF 文件存储在该文件夹下。
2. 根据文件路径调用 PdfFileReader()函数读取文件,生成 PdfFileReader 对象。
3. 如果用户选择每一页生成一个 PDF 文件,则从第一页开始遍历 PDF 文件。如果用户选择按区间拆分 PDF 文件,则需要先根据逗号拆分前端传入的数据存入列表中,之后循环遍历列表得到每次的输出范围。
4. 根据用户所需的输出范围,每一页生成一个 PDF 文件,或者每几页生成一个 PDF 文件,系统按照原始 PDF 文件名加下划线加编号作为新生成的文件名。
5. 调用 PdfFileWriter()函数得到 PdfFileWriter 对象,然后调用该对象的 addPage()函数添加 PDF 文件的每一页。最后使用 PdfFileWriter 对象的 writer()函数把数据写入到新生成的 PDF 文件。

系统把名称为 PDF 拆分 (一)、页码数为 10 页的 PDF 文件,按照每一页生成一个 PDF 文件的方式拆分后,结果如图 4-29 所示。

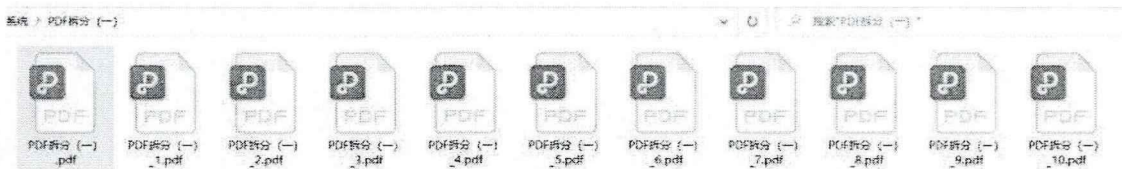


图 4-29 PDF 拆分结果图

4.5.3 PDF 旋转

当我们扫描纸质文档生成 PDF 文件时,由于人工操作失误,经常把 PDF 文件里的页面变为横向模式甚至颠倒模式。或者用计算机制作完 PDF 文件时发现里面的部分图片排版角度出错。如果我们想要正常阅读 PDF 文件内容,需要对 PDF 文件里的页面进行旋转,这时可以使用我们系统的 PDF 旋转功能,功能界面如图 4-30 所示。

用户点击添加文件按钮即可在本地选择需要旋转的 PDF 文件,用户上传文件后,界面会显示已上传文件的名字和大小。PDF 旋转功能界面提供了三种旋转方式,分别为顺时针 90 度旋转、逆时针 90 度旋转、180 度旋转。点击开始旋转按钮后,系统会按照用户选择的方式旋转 PDF 文件。



图 4-30 PDF 旋转功能界面图

当用户点击开始旋转按钮后,前端会把用户选择的旋转方式发送到后端,由后端生成旋转后的 PDF 文件,具体工作流程如下:

1. 后端根据功能、用户名、时间信息生成文件夹,并把用户需要旋转的 PDF 文件存储在该文件夹下。
2. 根据文件路径调用 PdfFileReader()函数读取文件,生成 PdfFileReader 对象,然后调用该对象的 getPage()函数获取所需的页面。
3. 调用 page 对象的 rotateClockwise()函数可以得到顺时针旋转的页面对象,在该系统中此函数的参数为 90 和 180。调用 page 对象的 rotateCounterClockwise()函数可以得到逆时针旋转的页面对象,在该系统中此函数的参数为 90。
4. 新建一个 PDF 文件,并调用 PdfFileWriter()函数得到 PdfFileWriter 对象,每次调用旋转方法后,都会调用该对象的 addPage()函数,这将向 PdfFileWriter 对象添加旋转后的页面。

图 4-31 展示了 PDF 文件中的某页顺时针旋转 90 度的效果，其中图中左边为正常页面，图中右边为旋转后的页面。



图 4-31 页面旋转 90 度图

4.5.4 PDF 加水印

为了防止他人盗用 PDF 文件内容，需要对文件声明版权，或者出于宣传品牌的目的，我们经常需要给 PDF 文件添加水印。这时可以使用我们系统的 PDF 加水印功能，添加文字水印的界面如图 4-32 所示。系统采用 ElementUI 组件库，使用 Table、Radio、Input、Select、ColorPicker、Button 和 Upload 等组件

用户点击添加文件按钮即可在本地选择需要添加文字水印的 PDF 文件，用户上传文件后，界面会显示已上传文件的名称和大小。类型选项里选择文字，之后可以选择字体样式、颜色、大小、透明度、旋转角度、放置层信息。点击立即开始按钮后，前端将组件值组合生成 json 通过 HTTP API 传参发送到后端，后端解析前端发送过来的数据信息，实现 PDF 文件添加文字水印的功能。

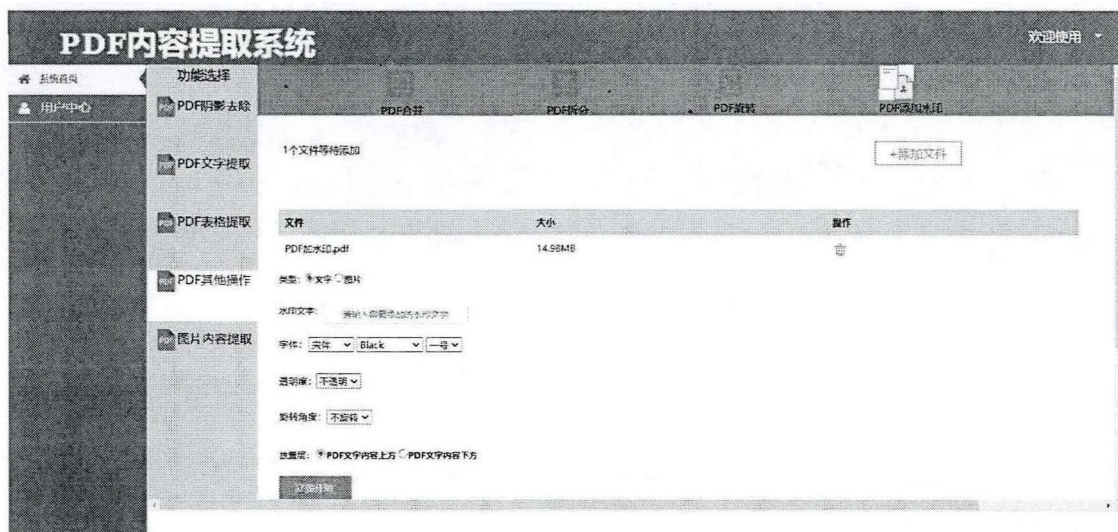


图 4-32 PDF 添加文字水印界面图

当用户点击立即开始按钮后，前端会把用户需求以参数形式发送到后端，由后端生成添加了文字水印的 PDF 文件，参数信息如下表 4-3 所示。

表 4-3 添加文字水印请求参数表

参数名称	描述
content	水印文字内容
style	文字样式
color	文字颜色
size	文字大小
diaphaneity	文字透明度
rotation	文字旋转角度
layer	文字放置层

为 PDF 文件添加水印，实际上就是提前制作一个水印文件，之后将生成的水印文件和用户上传的 PDF 文件进行合并。后端需要用到 PyPDF2、ReportLab 两个扩展模块，具体工作流程如下：

1. 后端根据功能、用户名、时间信息生成文件夹，并把用户需要添加图片水印的 PDF 文件存储在该文件夹下。
2. 调用 canvas 模块的 Canvas()函数生成水印文件并返回 canvas 对象。
3. 调用 canvas 对象的 setFont()函数设置字体样式和大小、setFillColorRGB()函数设置字体颜色、rotate()函数设置文字旋转角度、setFillAlpha()函数设置文字透明度、translate()函数移动坐标原点。
4. 根据文件路径调用 PdfFileReader()函数读取需要添加文字水印的 PDF 文件，遍历该文件，使用 getPage()函数顺序获取文件页面。调用 mergePage()函数对需要添加水印的页面和第三步生成的水印页面进行合并。
5. 新建一个 PDF 文件，并调用 PdfFileWriter()函数得到 PdfFileWriter 对象，每次合并页面后，都会调用该对象的 addPage()函数，这将向 PdfFileWriter 对象添加合并之后的页面。

图 4-33 展示了向 PDF 文件中添加文字水印的效果，文字内容为 PDF 内容提取系统。

strong image classifier to a distilled tokens in the initial embeddings, which leads to a competitive result comparing to the CNN-based models.

Referring to the Masked Language Model pre-training task, BEiT proposes the Masked Image Modeling task to pre-train the image Transformer. For each image, it will be converted to two views, image patches and visual tokens. They tokenize the original image into visual tokens by the latent codes of discrete VAE (Ramesh et al., 2021), randomly mask some image patches and make the model recover the original visual tokens. Essentially, the structure of BEiT is the same as the image Transformer and lacks the distilled token when comparing with DeiT.

2.2.2 Decoder Initialization

We use the RoBERTa models to initialize the decoder. Generally, RoBERTa is a replication study of (Devlin et al., 2019) that carefully measures the impact of many key hyperparameters and training data size. Based on BERT, they remove the next sentence prediction objective and dynamically change the masking pattern of the Masked Language Model. When loading the RoBERTa models to the decoders, the structures do not exactly match. For example, the encoder-decoder attention layers are absent in the RoBERTa models. To address this, we initialize the decoders with the RoBERTa models and the absent layers are randomly initialized.

2.3 Task Pipeline

The pipeline of the text recognition task in this work is described that given the textline images, the model extracts the visual features and predict the textline content.

the knowledge of both the visual feature extraction and the language model. The pre-training process is divided into two stages which differs by the used dataset. In the first stage, we synthesize a large dataset consisting of hundreds of millions of printed textline images with their corresponding text content and pre-train the TrOCR models on that. In the second stage, we build two relatively small datasets which correspond to printed and handwritten downstream tasks, containing about millions of textline images each. Subsequently, we pre-train two separate models on the printed data and the handwritten data of the second stage, both initialized by the first-stage model.

2.5 Fine-tuning

The pre-trained TrOCR models are fine-tuned on printed and handwritten text recognition tasks. The output of the TrOCR models are based on Byte Pair Encoding (BPE) (Sennrich et al., 2015) and does not rely on any task-related vocabularies.

2.6 Data Augmentation

To enhance the variety of the pre-training data and the fine-tuning data, we leverage the data augmentation. In total, seven kinds of image transformations (including keeping the original input image) are taken in this work. For each sample, we randomly decide which image transformation to take with equal possibilities. We augment the input images with random rotation (-10 to 10 degrees), Gaussian blurring, image dilation, image erosion, downscaling, underlining or keeping the original.

3 Experiments

图 4-33 PDF 添加文字水印效果图

当我们类型选项里选择图片时，可在 PDF 文件中添加图片水印，界面如图 4-34 所示。点击上传图片按钮即可选择本地图片进行上传，之后可以选择图片旋转角度、透明度、放置层信息。点击立即开始按钮后，系统会按照用户的要求给 PDF 文件添加图片水印。



图 4-34 PDF 添加图片水印界面图

为 PDF 文件添加图片水印，无论是前端发送的参数和后端对 PDF 文件的处理步骤，都和添加文字水印非常相似（除了一个是接收图片文件，一个是接收字符串），因此不再重复叙述。图 4-35 展示了向 PDF 文件中添加图片水印的效果，图片来源为本系统的名称截图。

strong image classifier to a distilled tokens in the initial embeddings, which leads to a competitive result comparing to the CNN-based models.

Referring to the Masked Language Model pre-training task, BEiT proposes the Masked Image Modeling task to pre-train the image Transformer. For each image, it will be converted to two views, image patches and visual tokens. They tokenize the original image into visual tokens by the latent codes of discrete VAE (Ramesh et al., 2021), randomly mask some image patches and make the model recover the original visual tokens. Essentially, the structure of BEiT is the same as the image Transformer and lacks the distilled token when comparing with DeiT.

2.2.2 Decoder Initialization

We use the RoBERTa models to initialize the decoder. Generally, RoBERTa is a replication of (Devlin et al., 2019) that carefully removes the impact of many key hyperparameters by using data size. Based on BERT, the model fine-tunes the next sentence prediction objective to gradually change the masking pattern. The Masked Language Model. When loading the RoBERTa models to the decoders, the structure does not exactly match. For example, the decoders' self-attention layers are absent in the RoBERTa models. To address this, we initialize the decoders with the RoBERTa models and the absent layers are randomly initialized.

2.3 Task Pipeline

The pipeline of the text recognition task in this work is described that given the textline images, the model extracts the visual features and predicts the knowledge of both the visual feature extraction and the language model. The pre-training process is divided into two stages which differs by the used dataset. In the first stage, we synthesize a large dataset consisting of hundreds of millions of printed textline images with their corresponding text content and pre-train the TrOCR models on that. In the second stage, we build two relatively small datasets which correspond to printed and handwritten datasets, respectively, containing about millions of textline images each. Subsequently, we pre-train the TrOCR models on the printed data and the handwritten data of the second stage, both using the first-stage model.

2.4 Fine-tuning

The trained TrOCR models are fine-tuned on the printed and handwritten text recognition tasks. The architecture of the TrOCR models are based on Byte Pair Encoding (BPE) (Sennrich et al., 2015) and does not rely on any task-related vocabularies.

2.6 Data Augmentation

To enhance the variety of the pre-training data and the fine-tuning data, we leverage the data augmentation. In total, seven kinds of image transformations (including keeping the original input image) are taken in this work. For each sample, we randomly decide which image transformation to take with equal possibilities. We augment the input images with random rotation (-10 to 10 degrees), Gaussian blurring, image dilation, image erosion, downscaling, underlining or keeping the original.

3 Experiments

图 4-35 PDF 添加图片水印效果图

4.6 图片内容提取模块实现

日常工作中，用户可能只需要对 PDF 文件里的部分文字或者表格进行提取，但是图片 PDF 里文字和表格内容是无法复制的。或者拍摄纸质文档时出现了阴影，但是并不需要把图片转为 PDF 文件。这时可以使用我们系统提供的图片内容提取功能，界面如图 4-36 所示。

用户在上传图片后，需要在下拉框中选择需要的功能（去除阴影、文字提取、表格提取），之后点击操作栏的开始图标，系统处理完成后即可在另存为栏中下载相应的文件。

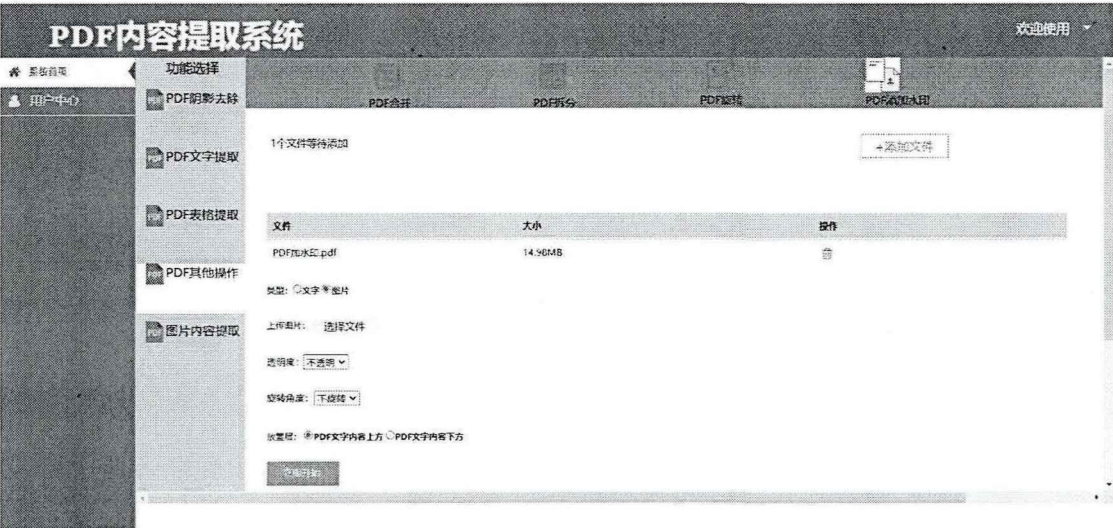


图 4-36 图片内容提取模块界面图

后端会对用户上传的文件进行过滤，如果不是 png、jpg 等图片格式，会返回给用户文件上传类型错误的提示信息。对图片进行处理的算法模型和对 PDF 文件进行处理的算法模型一致（区别就是需要先将 PDF 文件按页拆分为图片），因此对图片的处理不再重复叙述。对图片去除阴影是生成无阴影图片，对图片文字提取是生成 TXT 文件，对图片表格提取是生成单个工作表的 Excel 文件，图片内容提取模块流程图如图 4-37 所示。

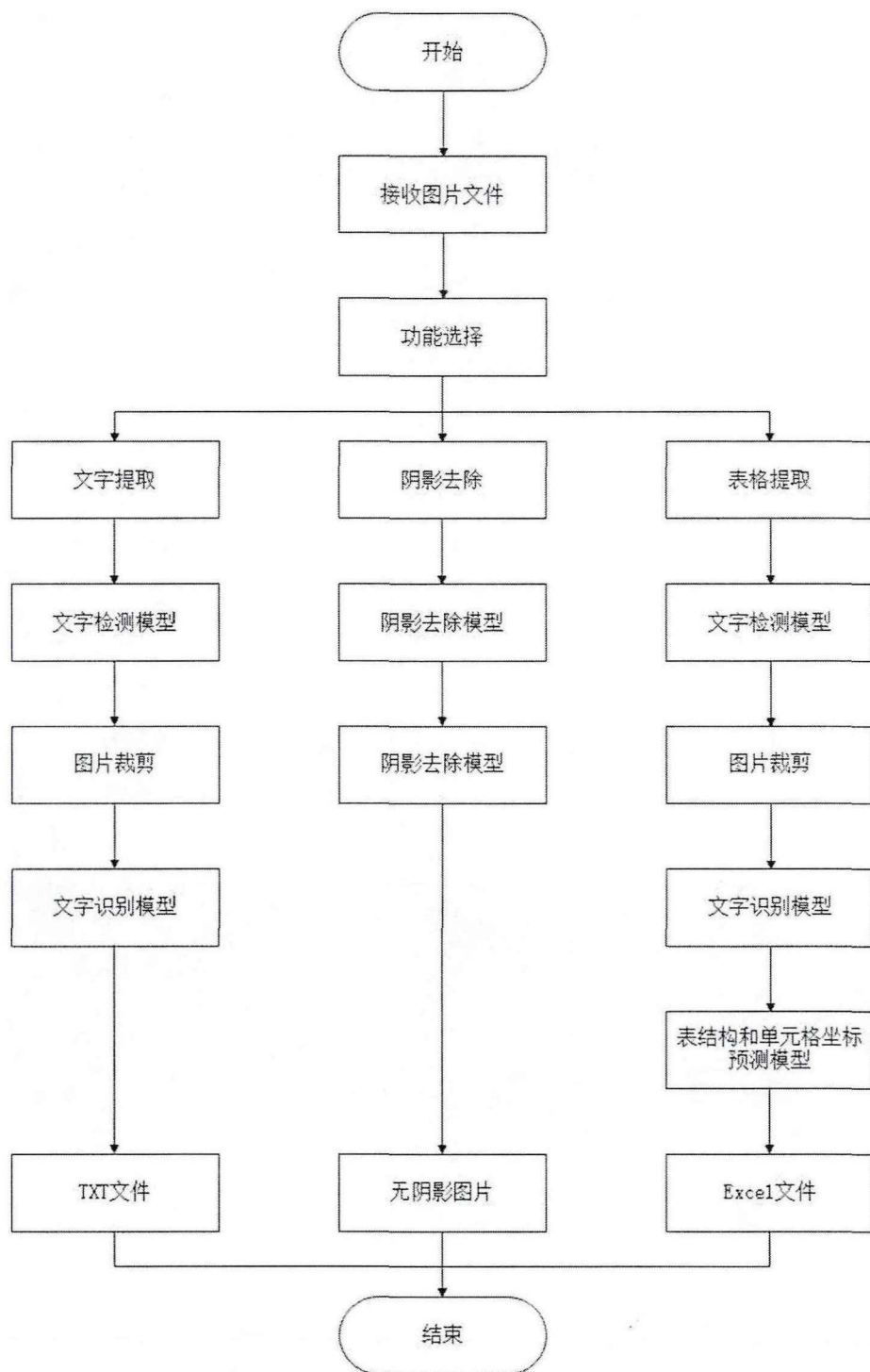


图 4-37 图片内容提取模块流程图

4.7 用户管理模块实现

4.7.1 用户注册及登录

用户要想使用本系统，首先需要注册用户名和密码，然后才能登录系统并使用所有功能。我们整合了阿里云的短信服务给用户发送验证码。首先，我们需要申请一个签名，表示短信发送者是谁，这里签名为 PDF 内容提取系统。第二，配置短信模板，表示具体发送的短信内容。第三，配置系统设置，可以使用 MNS 消息队列消费模式或者 HTTP 批量推送模式。最后，根据手机号，调用 API 进行短信发送。系统整合短信服务流程图如图 4-38 所示。

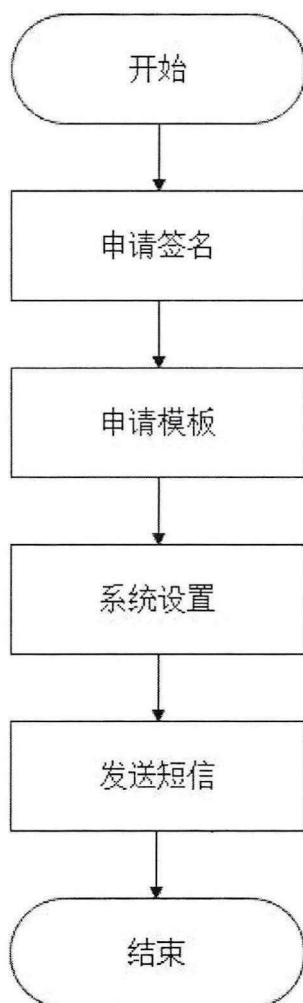


图 4-38 系统整合短信服务流程图

注册用户名时，后端会先去 MySQL 数据库中查询是否存在相同的手机号码（防止一个手机号注册多个用户名）。当数据库中不存在用户注册的手机号码时，短信服务会给用户发送验证码，并把该手机号码存入 Redis 中，以手机号码为 Key，用户 ID 为 Value，设置过期时间为 60 秒，防止用户在有限时间多次请求短信服务。当注册成功后，即可通过用户名（手机号）、密码登录 PDF 内容提

取系统。当后端返回验证成功信息后，前端通过 route 实现页面路由，跳转至系统主页面。登录界面如图 4-39 所示。

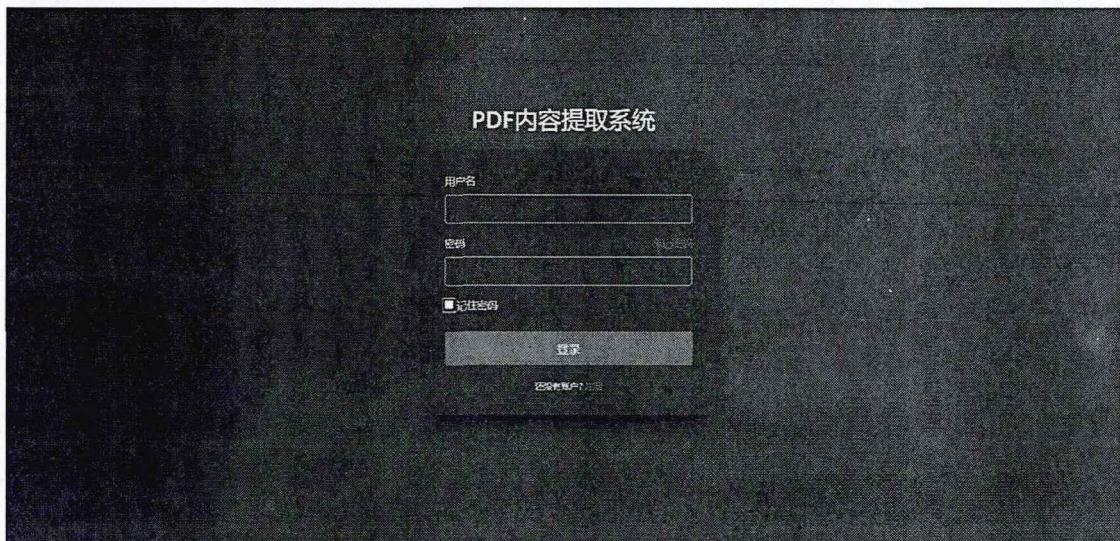


图 4-39 登录界面图

传统的用户身份验证使用 Cookie-Session 方式，Session 保存用户数据后生成一个 SessionID 返回给 Cookie，之后用户的每个请求都将携带 Cookie 中的 SessionID 传给服务器，服务器收到 SessionID 并对比之前保存的数据，确认用户的身份。这种方式虽然得到广泛应用，但是无法支持集群部署，没有分布式架构，无法支持横向扩展。而我们的系统要想保证高性能，至少需要在阿里云上部署两台及以上机器。

因此，我们使用目前最为流行的 JWT 技术来实现端到端的身份验证。JWT 本质就是加密后的 JSON 字符串，由以下三部分内容组成，各部分之间以「.»分隔。

1. Header 头部：用于指明 Token 类型和加密算法，最常用的加密算法为 HS256。

2. Payload 载荷：存储服务器需要的有效信息，例如用户名、权限信息、性别、年龄等，但是不建议存储敏感信息，例如密码。

3. Signature 签名：它包含三部分信息，base64 加密的头部、base64 加密的载荷、字符串密钥，密钥保存在服务端，用于解密验证用户信息。

服务端计算出 JWT 字符串后发送给客户端，客户端把 JWT 字符串存储在 cookie 或者浏览器的 LocalStorage 中。每次用户请求时都会携带这个 JWT 字符串，服务端收到后进行解密，判断是正常请求还是非法请求。请求流程如下图 4-40、4-41 所示。

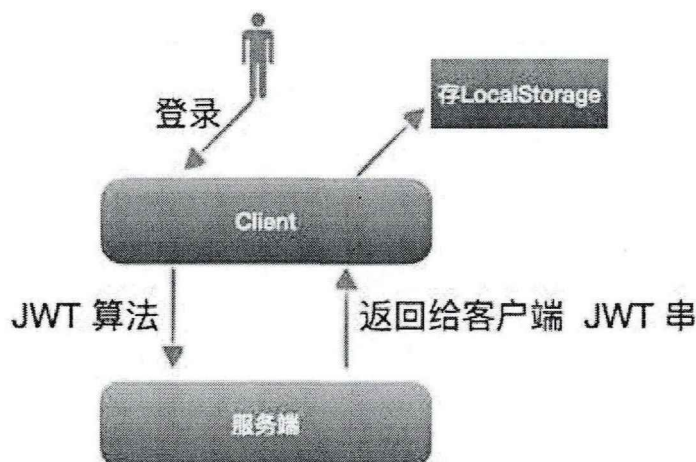


图 4-40 用户首次请求流程图

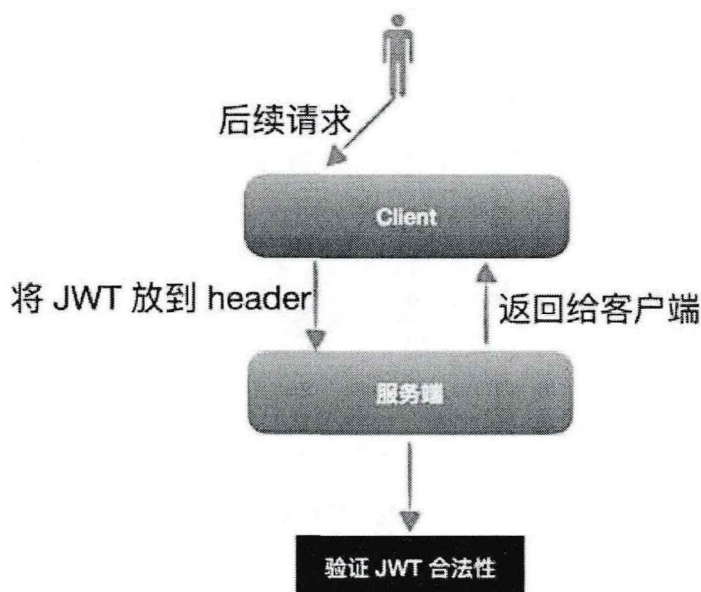


图 4-41 用户后续请求流程图

4.7.2 用户使用次数统计

为了更好的运营系统并且了解用户需求,例如我们需要知道实际场景下系统每天最多能接受多少使用量,也需要知道用户最喜欢使用哪个功能。我们开发了用户使用次数统计功能。

系统前端图表使用了 ECharts 可视化库。首先在 HTML 页面引入 echarts.min.js, 创建一个具备固定大小的 DOM 容器。然后使用 title 为图表配置标题, 使用 xAxis 配置需要在 X 轴上显示的数字, 使用 yAxis 配置需要在 Y 轴上显示的文字, 使用 type 决定图表类型 (bar 为柱状/条形图)。最后, 我们根据从后端数据库查询的数据生成我们的统计图表。

每当用户使用某个功能时，后端会把本次请求的用户 ID、使用功能类型等信息存入数据库。当需要展示使用次数时，只需要根据初始时间、末尾时间、功能类型，即可方便查询各个模块以及总模块使用次数。当需要展示某个用户使用记录时，加上用户 ID 即可查询。

系统管理员可以点击导航栏中的用户中心-使用统计，选择某月某周查看所有用户使用各个模块的情况，各模块分别使用次数如下图 4-42 所示，各模块总共使用次数如下图 4-43 所示。

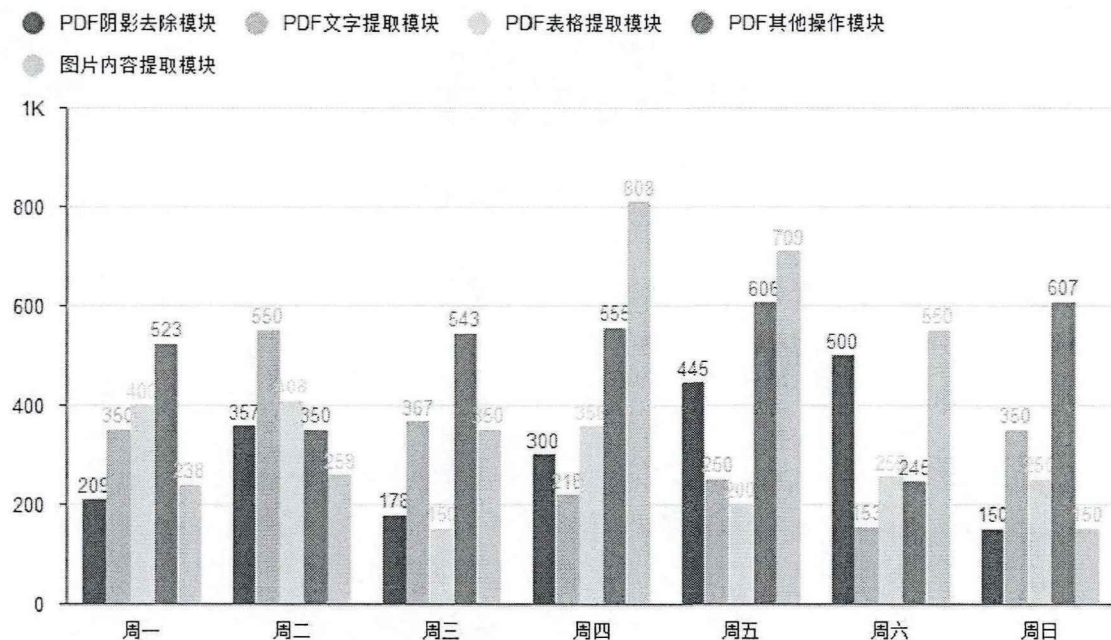


图 4-42 各模块使用次数统计图

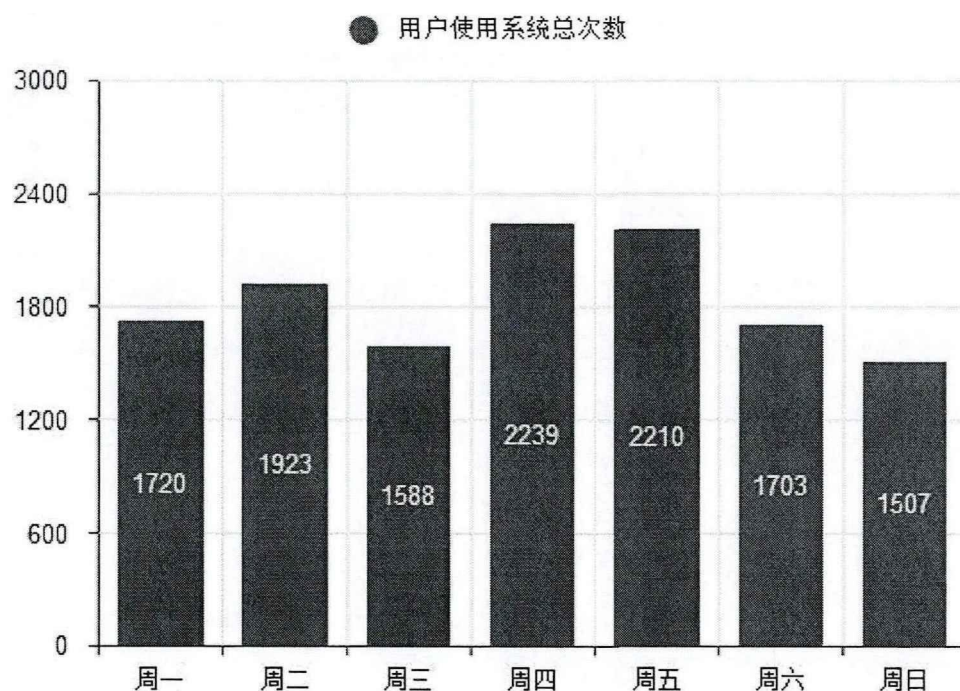


图 4-43 各模块总共使用次数统计图

4.7.3 用户评论及点赞

系统还提供了评论以及点赞功能，用户可以好评或者吐槽，让我们了解文字识别或者表格识别准确率、速率是否达到他们接受的标准。也帮助我们及时发现系统现有的问题，因为任何系统单靠自己测试是无法避免所有问题的。

前端页面用到了 ElementUI 的 Input、Avatar、Icon 和 Button 等组件，实现评论区域的 UI 界面。使用了 Vue 的 v-model 实现数据的双向绑定，用户点击“最热”或“最新”时，前端发起 API 请求，向后台调取对应页码的评论数据。

后端使用了 Redis 的 SET 和 ZSET 数据结构帮助我们实现评论排行榜功能。为了让评论按照点赞数和发布时间进行排行，我们建立了点赞数有序集合和发布时间有序集合，有序集合里以评论 ID 为 Key，排名分值为 Value。我们还需要为每个评论建立一个集合，用来记录哪个用户对这个评论点赞过，防止同一个用户对某个评论多次点赞。下图 4-44 为热门评论界面、4-45 为最新评论界面。



图 4-44 热门评论界面



图 4-45 最新评论界面

4.8 本章小节

本章首先介绍了制作文档数据集的原因和方法,之后根据第三章的相关设计,分别对 PDF 内容提取系统的阴影去除模块、文字提取模块、表格提取模块、其他操作模块、图片内容提取模块、用户管理模块的实现进行了详细阐述,并且都展示了处理结果。

第五章 系统测试

5.1 界面测试

界面测试主要是为了测试系统界面布局是否合理、控件摆放位置是否符合用户使用习惯、界面操作是否简单易懂、页面是否美观等,本节将从以下几个方面进行测试:

- (1) 界面能否在不同的浏览器上正常显示,常用的五大浏览器有 Chrome、IE、Safari、Opera、Firefox^[38]。
- (2) 界面在不同分辨率下能否正常显示。
- (3) 界面操作是否符合用户的常规习惯。
- (4) 界面展示是否存在缺陷,有无乱码出现。
- (5) 不同功能界面切换是否正常,按键是否存在冲突。
- (6) 界面导航栏、下拉列表、文件上传框、文本框、各种按钮等控件是否可以正常使用。

经常多次手工测试,界面在不同浏览器和分辨率下可以正常显示,界面控件可以正常使用,各个功能界面简洁易用、切换正常,没有出现字体乱码、文件上传或者下载失败等情况。

5.2 功能模块测试

功能模块测试主要是为了测试各个模块是否可以正常使用以及是否达到了预期效果。

5.2.1 PDF 阴影去除模块测试

点击文件上传控件,弹出文件选择框。当我们选择电脑里的 PDF 文件并点击确定后,界面提示文件上传成功,并且文件选择框下方的文件信息栏出现该文件的编号、名称、总页数等提示信息。当我们选择其他类型文件(非 PDF 文件),界面提示文件类型不符合要求,文件信息栏也不会出现该文件的任何信息。页面处理范围默认为第一页到最后一页,我们可以在范围框里自行输入所需处理的页码。当未点击操作栏里的开始按钮(三角图标)时,状态栏里的图标为未开始。当点击开始按钮后,状态栏里的图标变为正在处理,并且系统开始处理 PDF 文件。当状态栏里的图标变为成功时,系统处理 PDF 文件完毕。此时,可以点击

另存为栏里的保存按钮，把处理后的 PDF 文件保存到需要的位置。测试结果表明，PDF 文件里的阴影图像已经被去除。

5.2.2 PDF 文字提取模块测试

PDF 文字提取模块的测试步骤和 PDF 阴影去除模块的测试步骤基本相同，区别为需要点击下拉列表，选择需要生成的文件类型（Word 文件或者 TXT 文件）。我们分别测试这两个选项，测试结果表明，提取的文本内容在 Word 文件或者 TXT 文件中都能正确显示。

5.2.3 PDF 表格提取模块测试

PDF 表格提取模块的测试步骤和 PDF 阴影去除模块的测试步骤基本相同，区别为需要点击下拉列表，选择生成 Excel 文件里的单个工作表或者多个工作表。我们分别测试这两个选项，测试结果表明，提取的表格内容在单个工作表（所有提取的表格都在一张工作表）或者多个工作表（每个表格存放到一张工作表）中都能正确显示。

5.2.4 PDF 其他操作模块测试

点击 PDF 合并控件，界面切换到 PDF 合并功能界面。点击添加文件控件，弹出文件选择框。当我们选择电脑里的 PDF 文件并点击确定后，界面提示文件上传成功，并且界面中心的文件信息栏出现该文件的名称、大小信息。当我们选择其他类型文件（非 PDF 文件），界面提示文件类型不符合要求，文件信息栏也不会出现该文件的任何信息。上传多个 PDF 文件后，点击文件信息栏的上移或者下移控件，可以更改 PDF 文件的合并顺序。最后在文本框输入合并后的 PDF 文件名，点击开始合并控件。当合并完成后，界面出现文件下载控件，点击下载控件即可把合并后的 PDF 文件下载到相应位置。测试结果表明，多个 PDF 文件被正确合并。

点击 PDF 拆分控件，界面切换到 PDF 拆分功能界面。其他测试步骤和 PDF 合并测试步骤基本一致。区别为需要在两种输出格式中选择一种，分别为每一页生成一个 PDF 文件和按照区间拆分。分别测试这两种选项，按照区间拆分我们输入 2,5-10，表明拆分成两个 PDF 文件，一个 PDF 文件只有原始文件的第二页，另一个 PDF 文件包含原始文件的第五页到第十页。测试结果表明，PDF 文件被正确拆分。

点击 PDF 旋转控件, 界面切换到 PDF 旋转功能界面。其他测试步骤和 PDF 合并测试步骤基本一致。区别为需要在三种旋转角度中选择一种, 分别为顺时针旋转 90 度、逆时针旋转 90 度, 旋转 180 度。分别测试这三种旋转角度。测试结果表明, PDF 文件里的内容被正确旋转。

点击 PDF 添加水印控件, 界面切换到 PDF 添加水印功能界面。当我们上传了需要添加水印的 PDF 文件后, 有两种添加水印方式供我们选择, 分别为添加文字水印和添加图片水印。当选择添加文字水印时, 需要在文本框输入水印文字, 并且选择字体样式、颜色、大小、透明度、旋转角度、放叠层信息。当我们选择添加图片水印时, 需要上传水印图片。当我们上传非图片类型文件时, 系统会弹出文件类型错误提示框。上传水印图片后, 选择图片透明度、旋转角度、放叠层信息。测试结果表明, 文字水印或者图片水印都被正确添加到 PDF 文件中。

5.2.5 图片内容提取模块测试

点击文件上传控件, 弹出文件选择框。当我们选择电脑里的图片文件并点击确定后, 界面提示图片上传成功, 并且文件选择框下方的文件信息栏出现该图片名。下拉列表包含三个图片处理功能。选择下拉列表里的去除阴影, 系统会生成一个无阴影图片。选择下拉列表里的文字提取, 系统会生成一个包含图片中文字内容的 TXT 文件。选择下拉列表里的表格提取, 系统会生成一个包含图片中表格的 Excel 文件 (单个工作表)。测试结果表明, 这三个功能都能被系统正常处理。

5.2.6 用户管理模块测试

点击导航栏里的用户中心-评论, 系统切换到评论界面。在评论框里输入想要表达的文字, 点击发布, 即可在最新评论里找到刚才发布的评论。点击最热控件, 系统切换到热门评论界面, 评论按照点赞数排名, 自己也可以给某些评论点赞。

如果是系统管理员, 可以点击导航栏里的用户中心-使用统计, 系统切换到使用统计界面。界面存在三个下拉列表, 分别可以选择年、月、周。选择下拉列表里的一个具体周, 界面中心出现各模块使用次数统计图和各模块总共使用次数统计图。

点击导航栏里的用户中心-修改个人信息, 系统切换到修改个人信息界面。输入原始密码, 然后输入两次新密码, 点击确定按钮即可更改密码。如果原始密

码输入错误、两次新密码输入不一致、密码格式输入错误,系统都会出现相关提示。点击更换头像,即可上传新的头像图片。点击更换名称,即可更换心得网名。

测试结果表明,用户管理模块的相关功能都可以正常使用。

5.3 性能测试

在系统功能可以正常使用后,还需要对系统性能进行测试,保证多个用户可以同时使用本系统^[39]。测试机器显卡型号为 GeForce MX250,CPU 型号为 Inter(R) Core(TM) i7-8765U,内存容量为 16GB。

对于阴影去除模型,可以使用峰值信噪比(PSNR)和结构相似性(SSIM)两个指标对视觉质量进行定量比较来评价模型的性能效果。我们对系统使用的 BEDSR-Net 模型和四种有竞争力的方法在五个数据集上进行了比较,每个数据集的最佳分数用红色标记,次佳分数用蓝色标记。最终比较结果如下表 5-1、5-2 所示。

表 5-1 PSNR 比较结果

	SDSRD	RDSRD	Bako's dataset	Kligler's dataset	Jung's dataset
Bako	31.55	28.24	35.22	29.66	23.70
Kligler	22.03	22.53	26.50	26.45	24.45
Jung	17.06	14.45	13.88	19.21	28.49
ST-CGAN	39.38	30.31	29.12	25.92	23.71
BEDSR-Net	43.59	33.48	35.07	32.90	27.23

表 5-2 SSIM 比较结果

	SDSRD	RDSRD	Bako's dataset	Kligler's dataset	Jung's dataset
Bako	0.9658	0.8664	0.9823	0.9051	0.9015
Kligler	0.8435	0.7056	0.8381	0.8481	0.8332
Jung	0.8226	0.7054	0.8059	0.8724	0.9108
ST-CGAN	0.9834	0.9016	0.9600	0.9062	0.9046
BEDSR-Net	0.9935	0.9084	0.9809	0.9354	0.9115

由表中比较结果可知,我们系统使用的 BEDSR-Net 模型在大多数数据集上都达到了最好的效果。

对于文字识别模型,可以使用召回率、精确率、F1 值来评价模型性能效果。我们对系统使用的 TrOCR 模型和 SROIE 印刷体数据集排行榜上最先进的模型进行了比较,结果如表 5-3 所示。

表 5-3 SROIE 数据集上比较结果

Model	Recall	Precision	F1
TrOCR _{BASE}	96.37	96.31	96.34
TrOCR _{LARGE}	96.59	96.57	96.58
H&H Lab	96.35	96.52	96.43
MSOLab	94.77	94.88	94.82
CLOVA OCR	94.3	94.88	94.59

实验结果表明，TrOCR 模型凭借 Transformer 结构在打印体文字识别任务上超越了当前最先进的模型。

对于表格识别模型，可以使用 TEDS（基于树编辑距离的相似度）作为评价指标来比较模型性能效果。我们对系统使用的 PaddleOCR 中的表格识别模型和 PubTabNet 数据集（目前最大的公开表格识别数据集）排行榜上最先进的 EDD 模型^[40]进行了比较，结果如表 5-4 所示。

表 5-4 PubTabNet 数据集上比较结果

Method	TEDS(Tree-Edit-Distance-based Similarity)
EDD	88.3
Ours	93.32

实验结果表明，系统所采用的表格识别模型能够准确识别复杂的表格，TEDS 分数比目前最先进的表格识别模型高出 5.37%。

由于文件所占字节数较大，当多个用户同时上传文件进行功能调用时，系统的承受压力会变得很大。因此，我们也需要对系统进行压力测试，我们首先对系统的 PDF 文件上传接口进行测试，测试的 PDF 文件大小为 5.68M，测试结果如下图 5-1 所示。

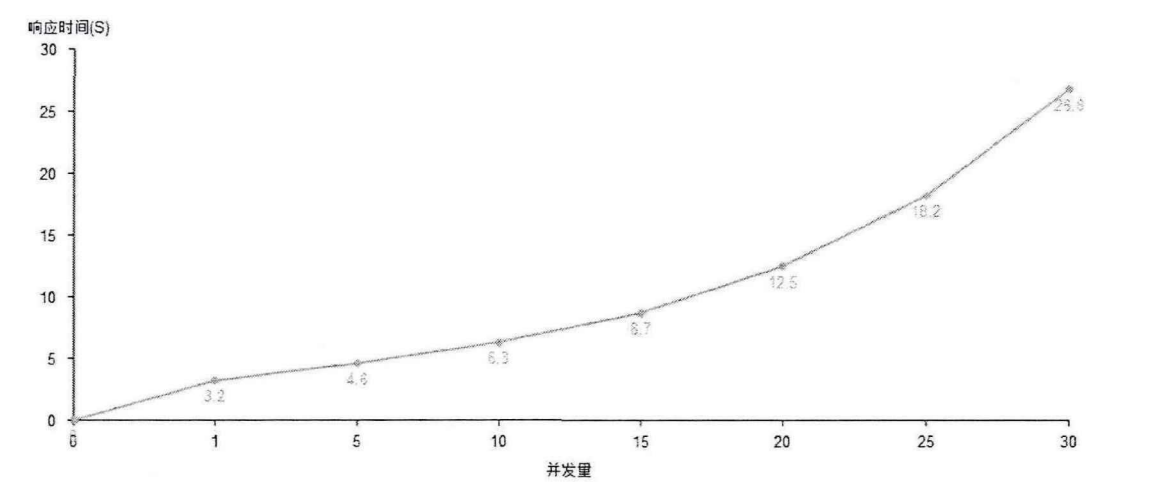


图 5-1 PDF 文件上传接口压测图

由上图 5-1 可以看出，在系统并发量低于 20 的时候，完成 PDF 文件上传所需要的时间平稳增加。并发量超过 20 的时候，所需的时间开始大幅度增加，系统性能开始下降。

之后，我们又对文字识别接口进行了测试，测试数据为第一步得到的 PDF 文件，转换范围为全部页面，生成结果为 TXT 类型文件，测试结果如下图 5-2 所示。

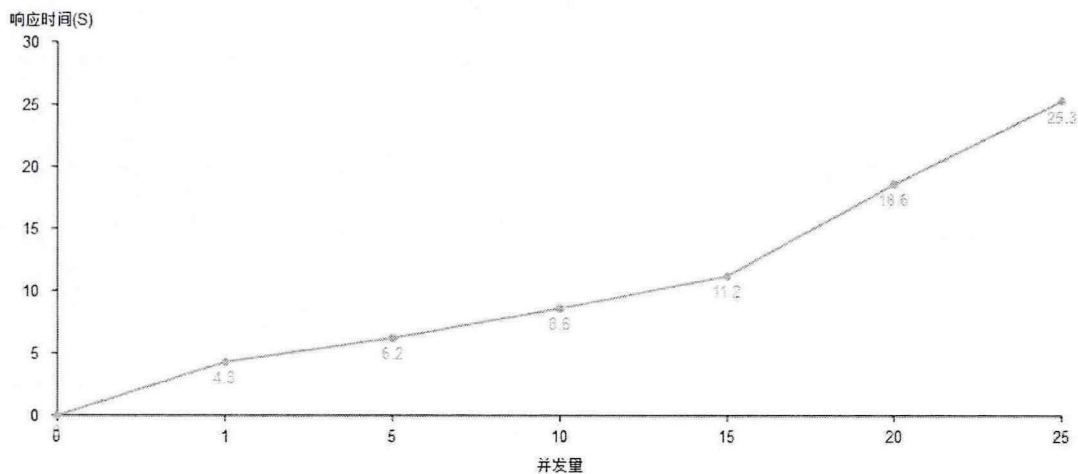


图 5-2 文字识别接口压测图

由上图 5-2 可以看出，在系统并发量低于 15 的时候，生成 TXT 文件所需要的时间平稳增加。并发量超过 15 的时候，所需的时间开始大幅度增加，系统性能开始下降。

系统接口的响应时间和所需要处理的文件大小有直接关系，在本测试环境中，用户上传 PDF 文件大小为 5.68M，调用 PDF 文字提取功能，系统的承载量在 20 个并发任务左右。

5.4 本章小节

本章说明了系统测试环境，详细描述了所需要测试的要点和操作步骤，对系统的界面、功能模块、性能进行了测试，达到了预期的效果。

第六章 总结和展望

6.1 论文总结

本文基于文档阴影去除、文字检测、文字识别、表格检测、表格识别方向的深度学习模型,结合深度学习框架 PyTorch、后端框架 Django、前端框架 Vue、关系型数据库 MySQL、非关系型数据库 Redis、工具库 pdfplumber 等技术,设计并实现了 PDF 内容提取系统。

本文的主要工作如下:

1. 从近年来人们用手机或者扫描仪读取纸质文档并生成 PDF 文件时存在的一些问题,引入了本系统的研究背景和意义。根据用户的实际需求和社会应用价值,确定了系统的研究目标和方向。根据系统的需求分析,完成了功能模块的划分,并给出了系统的总体设计。

2. 针对手机拍摄纸质文档得到的图像很容易产生阴影,但是市面上的 PDF 软件没有阴影去除功能这个问题。本系统使用了 BEDSR-Net 模型和相关技术实现了 PDF 文件内容阴影去除功能, BEDSR-Net 模型是第一个也是目前唯一一个专门为文档图像阴影消除而设计的深度学习模型。

3. 提出了文档文字图像数据集的制作方法:爬取互联网公开的 PDF 文件并随机选取其中的页面,由于这些 PDF 文件数字生成的,我们需要使用相关库把文件里的页面转为图像,然后根据文本行坐标裁剪图像即可得到高质量的文本行图像。对这些数据进行了图像增强,得到了更多类型的训练数据,提高了模型的泛化能力。

4. 根据数字 PDF 文件和图片 PDF 文件的区别,使用了不同的思路去实现文字提取功能和表格提取功能。根据用户的不同需求,系统可以只处理部分范围页面,并且生成不同格式的文件。文中给出了详细的处理流程图并且展示了最后的处理结果。

5. 实现了系统的前端页面,降低了系统使用门槛,让用户可以轻松的使用本系统。增加了一些常用的功能,例如 PDF 合并、PDF 拆分、PDF 旋转、PDF 加水印。为了更好的运营本系统和了解用户需求,增加了用户使用统计和用户评论功能。最后从系统的界面、功能、性能方面进行测试,并给出了测试结果。

6.2 工作展望

由于个人时间和精力有限,PDF 内容提取系统还存在一些问题,在应用性上还具有较大提升空间,现在对当前系统存在的不足进行梳理,未来我将会逐步完善。

1. 本系统针对数字 PDF 文件采用了 pdfplumber 工具库提取表格内容,但是该工具库对边框不完整、合并单元格较多的复杂表格处理效果不是很好。因此,计划在后期把数字 PDF 文件转化图片,像处理图片 PDF 文件那样使用深度学习技术提取表格内容。

2. 本系统大多数功能都需要处理文件,当用户上传文件较大时,系统接收、处理、生成文件都需要消耗大量时间,尤其并发量过高时,用户等待系统处理结果的时间会变得很长。后续计划对系统进行集群部署,然后使用 Nginx 作为负载均衡器,让用户的请求轮询访问到各个服务器。这样可以缓解单个服务器的压力,极大提升并发量、减少系统响应时间。

3. 由于经过阴影去除后的图像可以提升 OCR 性能,后续考虑在文字提取或者表格提取之前,先对图像做阴影去除。并且计划增加更多处理 PDF 文件的功能,例如 PDF 转 PPT、PDF 压缩、PDF 增加或去除密码、PDF 安全检测等,让 PDF 内容提取系统变得更加实用。

参考文献

- [1] 于丰畅,陆伟.基于机器视觉的PDF学术文献结构识别[J]. 情报学报,2019,38(4): 384-390.
- [2] 赵立忠. 图片版PDF制作方法多[J]. 电脑爱好者, 2013(8):1-2.
- [3] 杜文婉. 深度学习在上市公司年报中的预测与应用[D]. 大连理工大学, 2019.
- [4] Smith R . An Overview of the Tesseract OCR Engine[C]// International Conference on Document Analysis & Recognition. IEEE Computer Society, 2007.
- [5] Kim G , Hong T , Yim M , et al. Donut: Document Understanding Transformer without OCR[J]. 2021.
- [6] Finlayson G D , Drew M S , Lu C . Entropy Minimization for Shadow Removal[J]. International Journal of Computer Vision, 2009, 85(1):35-57.
- [7] Arbel E , Hel-Or H . Shadow Removal Using Intensity Surfaces and Texture Anchor Points[J]. IEEE Transactions on Pattern Analysis & Machine Intelligence, 2011, 33(6):1202-1216.
- [8] Lin Y H , Chen W C , Chuang Y Y . BEDSR-Net: A Deep Shadow Removal Network From a Single Document Image[C]// 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). IEEE, 2020.
- [9] 柳玉辉. 计算机文字识别的技术探讨与应用前景分析[J]. 软件工程师, 1999.
- [10] Casey R G , Nagy G . Advances in Pattern Recognition[J]. Scientific American, 1971, 224(4):56-71.
- [11] Li M , Lv T , Cui L , et al. TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models[J]. 2021.
- [12] Szegedy C , Ioffe S , Vanhoucke V , et al. Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning[J]. 2016.
- [13] He K , Zhang X , Ren S , et al. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification[C]// CVPR. IEEE Computer Society, 2015.
- [14] Graves A , Schmidhuber J . Framewise phoneme classification with bidirectional LSTM and other neural network architectures[J]. Neural Networks, 2005, 18(5-6):602-610.
- [15] Goodfellow I J , Pouget-Abadie J , Mirza M , et al. Generative Adversarial Networks[J]. Advances in Neural Information Processing Systems, 2014, 3:2672-2680.
- [16] 兰旭辉,熊家军,邓刚. 基于MySQL的应用程序设计[J]. 计算机工程与设计, 2004, 25(3):3.

- [17] 郎泓钰, 任永功. 基于Redis内存数据库的快速查找算法[J]. 2022(5).
- [18] 阮耀民. 基于Django的Python Web快速开发[J]. 2021.
- [19] 刘亚茹, 张军. Vue.js框架在网站前端开发中的研究[J]. 电脑编程技巧与维护, 2022(1):3.
- [20] Z Tian, W Huang, T He,等. Detecting Text in Natural Image with Connectionist Text Proposal Network[J]. 2016.
- [21] Ren S , He K , Girshick R , et al. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks[J]. 2015.
- [22] Shi B , Bai X , Belongie S . Detecting Oriented Text in Natural Images by Linking Segments[J]. IEEE Computer Society, 2017.
- [23] Liao M , Shi B , Bai X . TextBoxes++: A Single-Shot Oriented Scene Text Detector[J]. IEEE Transactions on Image Processing, 2018, 27(8):3676-3690.
- [24] Liao M , Shi B , Bai X , et al. TextBoxes: A Fast Text Detector with a Single Deep Neural Network[J]. 2016.
- [25] Baek Y , Lee B , Han D , et al. Character Region Awareness for Text Detection[C]// 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). IEEE, 2019.
- [26] Wei L , Cao L , D Zhao, et al. CRNN: Integrating classification rules into neural network[C]// Neural Networks (IJCNN), The 2013 International Joint Conference on. IEEE, 2013.
- [27] Li M , Lv T , Cui L , et al. TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models[J]. 2021.
- [28] Fetahu B , Anand A , Koutraki M . TableNet: An Approach for Determining Fine-grained Relations for Wikipedia Tables[J]. 2019.
- [29] Prasad D , Gadpal A , Kapadni K , et al. CascadeTabNet: An approach for end to end table detection and structure recognition from image-based documents[J]. IEEE, 2020.
- [30] 百度飞桨官网. 表格识别[EB/OL]. <https://github.com/PaddlePaddle/PaddleOCR/blob/release/2.2/ppstructure/table>.
- [31] Bako S , Darabi S , Shechtman E , et al. Removing Shadows from Images of Documents[C]// Asian Conference on Computer Vision. Springer, Cham, 2016.
- [32] Kligler N , Katz S , Tal A . Document enhancement using visibility detection[C]// 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). IEEE, 2018.
- [33] Jung S , Hasan M A , Kim C . Water-Filling: An Efficient Algorithm for Digitized Document Shadow Removal[C]// 2019.
- [34] Okuda T , Tanaka E , Kasai T . A Method for the Correction of Garbled Words Based on the Levenshtein Metric[J]. IEEE Transactions on Computers, 2009, C-25(2):172-178.

- [35] Liao M , Wan Z , Yao C , et al. Real-time Scene Text Detection with Differentiable Binarization[J]. 2019.
- [36] Shi B , Wang X , Lyu P , et al. Robust Scene Text Recognition with Automatic Rectification[C]// 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). IEEE, 2016.
- [37] 葛罗瑞亚·布埃诺·加西亚, 刘冰 译, 朱征宇 校. OpenCV图像处理 [Learning Image Processing with OpenCV][M]. 机械工业出版社, 2016.
- [38] 第七乐章. "漫"谈五大浏览器[J]. 计算机应用文摘, 2010(30):1.
- [39] 黄美锋, 张毅彬. 分布式应用软件接口测试分析技术[C]// 2008:3.
- [40] Xu Z , Shafieibavani E , Yepes A J . Image-Based Table Recognition: Data, Model, and Evaluation[M]. Springer, Cham, 2020.